

Agent Runtime / Sandbox

Agent 会话编排、沙箱隔离、凭据托管、网关治理和多 Agent 协作。

39 篇 · 2026-10-04

目录

1. Agent Runtime / Sandbox 项目地图
2. agent-sandbox
3. OpenKruise Agents
4. agentgateway
5. Agent Runtime / Sandbox 细分选型地图
6. MCP Server / Tool Gateway 对比地图
7. Agent Runtime Substrate
8. Agent Substrate 架构与设计思路分析
9. Agent 凭据隔离
10. agent-sandbox 架构与设计思路分析
11. AgentCube
12. AgentCube 架构与设计思路分析
13. AgentScope
14. AgentScope Runtime
15. AgentScope Runtime 架构与设计思路分析
16. AgentScope 架构与设计思路分析
17. AI Gateway
18. Envoy AI Gateway
19. Envoy AI Gateway 架构与设计思路分析
20. HiClaw
21. HiClaw 架构与设计思路分析
22. Higress
23. Higress 架构与设计思路分析
24. Inference Routing
25. kgateway
26. kgateway 架构与设计思路分析
27. LLM 网关与路由层对比
28. NemoClaw

- 29. NemoClaw 架构与设计思路分析
- 30. Open Cowork
- 31. Open Cowork 架构与设计思路分析
- 32. OpenKruise Agents 架构与设计思路分析
- 33. OpenShell
- 34. OpenShell 架构与设计思路分析
- 35. Plano
- 36. Plano 架构与设计思路分析
- 37. substrate
- 38. vLLM Semantic Router
- 39. 声明式 Agent 管理

Agent Runtime / Sandbox 项目地图

这页把当前知识库里和 Agent runtime、sandbox、会话编排、安全控制面相关的项目放到同一张工程地图里。核心结论：这些项目都在解决“Agent 怎么安全、可恢复、可观测地长期运行”，但抽象层不一样。

```
Developer / user API
  ↓
Agent app framework / CLI / SDK
  ↓
session orchestration / agent control plane
  ↓
sandbox lifecycle primitive
  ↓
runtime enforcement: container / supervisor / proxy / policy
  ↓
network, credentials, storage, inference, tool access
```

通用 Agent Session 生命周期

```
Create / Claim
  ↓
Provision or WarmPool adoption
  ↓
Ready + route + identity
  ↓
Running session
  ├── tool/model/network request → policy → external service
  ├── idle timeout → suspend / hibernate
  ├── checkpoint → snapshot / PVC / workspace
  ├── resume → restore route + identity
  └── expiry / delete → revoke → cleanup / retain data
```

这个方向不能只看“能不能启动一个容器”。Agent runtime 的特殊性在于：它是长寿命、有状态、会执行不可信输入、会持有工具能力、会访问模型和外部网络，并且经常需要人类中途介入。真正的系统边界包括生命周期、身份、凭据、网络、文件、推理路由、工具权限和恢复。

一句话分层

项目	一句话定位	抽象层
agent-sandbox	K8s SIG Apps 的 Sandbox CRD，把单个有状态 Agent 容器建模成 K8s 一等资源	Sandbox 基础设施原语
openkruis-e-agents	OpenKruise 的 Agent sandbox lifecycle platform，把 SandboxSet/Claim、E2B API、Envoy 路由和 runtime/CSI/identity 扩展组合起来	K8s sandbox 平台层
agentcube	基于 agent-sandbox 的 serverless session orchestration layer，给 Code Interpreter / AgentRuntime 提供 HTTP/SDK 调用入口	会话编排层
OpenShell	NVIDIA 的安全私有 Agent runtime，Gateway 控制面 + sandbox Supervisor + OPA/Z3 policy proxy	安全 runtime / enforcement
NemoClaw	OpenShell sandbox 内 always-on Agent 的 host-side CLI 控制面，编排 onboarding、provider、policy 和 agent setup	Host-side 编排层
HiClaw	K8s operator + Matrix IM + Higress 网关的多 Agent 协作平台	多 Agent 平台 / 协作层
AgentScope	Python 多 Agent 应用框架，事件流 ReAct loop + toolkit/MCP/skill + workspace/offload + FastAPI service	应用框架 / 服务层
agentgateway	面向 LLM / MCP / A2A 的 Rust L7 数据面 + Gateway API 控制面	Agent 网络 / 工具网关

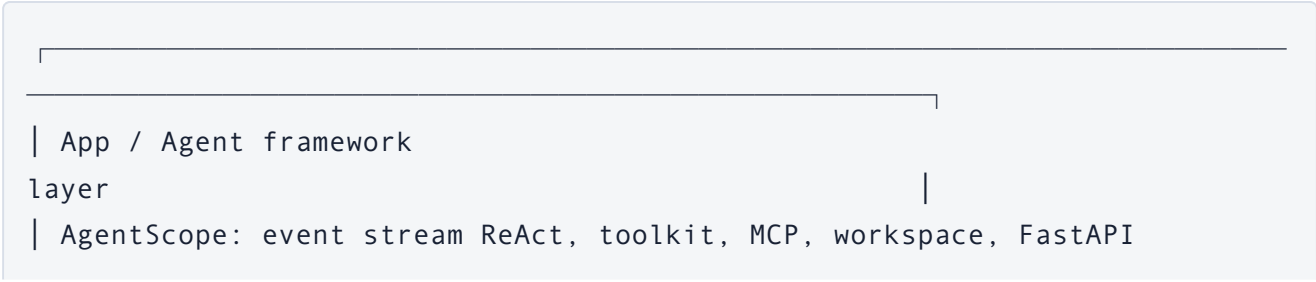
横向对比

维度	agent-sandbox	openkruis-e-agents	agentcube	OpenShell	NemoClaw	HiClaw	AgentScope	agentgateway
主问题	安全运行一个有状态 Agent 容器	K8s 上运营 E2B-like sandbox 服务	把 Sandbox 变成可调用 session	sandbox 内 runtime enforcement	OpenShell onboarding + agent setup	多 Agent 运维与协作	Agent app 构建与服务化	AI 协议流量治理
用户入口	K8s CRD / Go/Python SDK	E2B API + K8s CRD	HTTP invocation / Python SDK /	CLI / SDK / TUI	nemocl原因 / nemohermes CLI	Matrix / CLI / CRD	Python SDK / FastAPI	Gateway API / xDS / UI

维度	agent-sandbox	openkruse-agents	agentcube	OpenShell	NemoClaw	HiClaw	AgentScope	agentgateway
			Dify / LangChain					
控制面	K8s controller-runtime	sandbox-manager + agent-sandbox-controller	Router + WorkloadManager + Redis	Gateway object store + compute runtime	Host-side FSM + OpenShell adapters	K8s operator + embedded/in-cluster mode	FastAPI service + storage / session manager	Go controller + ADS server
数据面	Pod + headless Service + PVC	Envoy ext_proc / Go filter + Sandbox Pod	Sandbox Pod / PicoD / reverse proxy	Supervisor + restricted child + policy proxy	OpenShell sandbox + OpenClaw/Hermes	Worker / Manager containers + Matrix rooms	Agent event loop + workspace	Rust proxy for LLM/MCP/A2A
生命周期	Sandbox 0/1 replicas, shutdownTime, WarmPool	Sandbox Set pool、Claim、pause/resume、clone/checkpoint	session create/route/GC, WarmPool claim	create/delete/watch/reconcile/resume	onboard resume/fresh, registry lock	CR reconcile worker/team/human/manager	session run serialization, continuation events	listener/route/backend config hot update
隔离机制	委托 K8s runtimeClass/NetworkPolicy/securityContext	K8s runtime + agent-runtime/CSI/traffic-proxy 扩展	继承 agent-sandbox + Router JWT	Supervisor/proxy/seccomp/netns/TLS/OPA	继承 OpenShell policy/shields	容器隔离 + Higress 凭据隔离	Workspace Local/Docker/E2B	CEL RBAC, mTLS / HBO NE, guardrails

维度	agent-sandbox	openkruse-agents	agentcube	OpenShell	NemoClaw	HiClaw	AgentScope	agentgateway
凭据模型	不直接托管凭据	E2B API key storage + identity token propagation	Router→Pic oD JWT, WorkloadManager auth	Gateway 托管 provide r credentials	Gateway 是 凭据 system-of- record	Higress 托管真实凭 据, Worker 只持 consumer key	Credential/ session / workspace service	backend credentials + policy/ filter
状态	K8s API + PVC	K8s API + route registry + key storage + memberlist	Redis/ ValKey session registry + K8s	SQLite / Postgres object store	~/ .nemo claw JSON + Gateway state	etcd/ kine + Matrix + MinIO + gateway	storage / session / workspace	xDS stores + control plane
典型强项	K8s-native、简单、WarmPool	CRD + E2B API + Envoy route + runtime 扩展一体化	产品化 invocation API	安全 enforcement 边界最 细	Onboarding 和 provider/ policy 编排	IM-first 协作和 凭据零 暴露	Agent app 编 程体验	LLM/ MCP/ A2A 统一 网关
典型代价	只提供底层原语	系统面 宽, API/ roadmap 仍在快 速演进	依赖 agent- sandbox API 演进	系统复 杂度高	host-side 状 态机复杂	学习曲 线和容 器开销	不是强 sandbox x 原语	不直 接运 行 Agent

分层图谱



```
service |
| nanobot: lightweight personal agent loop, channels, providers,
skills |
```

↓

```
| Multi-agent / session product
layer |
| HiClaw: Worker/Team/Human/Manager CRD + Matrix +
Higress |
| OpenKruise Agents: SandboxSet/Claim + E2B API + Envoy
route |
| AgentCube: AgentRuntime/CodeInterpreter + Router +
WorkloadManager |
| NemoClaw: OpenShell onboarding + OpenClaw/Hermes
setup |
```

↓

```
| Sandbox / runtime primitive
layer |
| agent-sandbox: Sandbox/Claim/Template/WarmPool → Pod/PVC/
Service |
| OpenShell: Gateway desired state → driver → Supervisor/proxy
enforcement |
```

↓

```
| Network / credential / tool gateway
layer |
| agentgateway: LLM/MCP/A2A proxy, Gateway API, CEL, HBONE,
xDS |
| Higress/OpenShell gateway: provider credentials and inference
routing |
```

关键关系：

- [agentcube](#) 明确站在 [agent-sandbox](#) 之上：前者做 session API，后者做 Sandbox 生命周期。
- [openkruise-agents](#) 站在 [agent-sandbox](#) 相邻但更平台化的位置：它把 SandboxSet/Claim、E2B API、Envoy 路由、agent-runtime、CSI 和 identity token 放到同一套 K8s 发行版里。
- [NemoClaw](#) 明确站在 [OpenShell](#) 之上：前者做 onboarding 和 agent setup，后者做 sandbox runtime 和 enforcement。
- [HiClaw](#) 和 [agent-sandbox](#) 是互补关系：HiClaw 管多 Agent 协作，agent-sandbox 可以成为底层隔离原语。
- [AgentScope](#) 和 [agent-sandbox](#) 处在相邻问题域：AgentScope 的 Workspace 能接 Local/Docker/E2B，agent-sandbox 是更云原生的执行环境原语。
- [agentgateway](#) 不运行 Agent，但控制 Agent 访问模型、MCP、A2A 的网络边界。

工程剖面

[agent-sandbox](#)：Sandbox 作为 K8s 原语

[agent-sandbox](#) 的核心贡献是把 Agent runtime 从普通 Deployment/StatefulSet 里抽出来。Agent sandbox 的语义是：

- 单实例，replicas 只能 0/1。
- 有稳定身份，自动创建 headless Service。
- 有持久存储，PVC 生命周期不随 Pod 自动删除。
- 可以暂停/恢复，scale to 0/1。
- 可以过期，shutdownTime 是声明式生命周期。
- 可以 WarmPool 预热，然后通过 Claim 领养。

它避免把安全机制做成私有实现，而是把隔离委托给 K8s 原语：runtimeClassName、gVisor/Kata、NetworkPolicy、securityContext、serviceAccount。这个设计的工程价值是可组合：任何上层平台都可以继承 K8s 生态能力。

风险也清楚：它只提供底层原语，不负责多 Agent 协作、工具协议、凭据网关、HTTP invocation 或 Agent app framework。这些必须由上层补齐。

[openkruise-agents](#)：把 Sandbox 原语扩展成 E2B-like 平台

[openkruise-agents](#) 的关键不是再造一个最小 Sandbox CRD，而是把 Agent sandbox 生命周期的产品面补齐：

E2B API / K8s CRD



```
sandbox-manager
  ↓
SandboxClaim / SandboxSet / Sandbox
  ↓
agent-sandbox-controller
  ↓
Envoy route registry → Sandbox Pod IP:port
```

它把 warm pool 明确建模成 `SandboxSet`，把用户请求建模成 `SandboxClaim`，把 API 入口做成 `E2B-compatible / sandboxes / / snapshots`，再通过 `Envoy ext_proc` 或 `Go filter` 把请求路由到具体 `sandbox`。这个组合适合做自托管 E2B-like workspace 服务。

风险在于系统面比 [agent-sandbox](#) 大很多：key storage、route registry、memberlist peer sync、dynamic CSI mount、identity token propagation、checkpoint/hibernation 和 runtime sidecar 都需要分别验证。选型时应把它看成平台工程项目，而不是单一 controller。

[agentcube](#)：把 Sandbox 包成可调用会话

[agentcube](#) 的定位比 [agent-sandbox](#) 高一层。它关心开发者如何通过 HTTP/SDK 调用一个 Code Interpreter 或 AgentRuntime，而不是让用户直接写 Sandbox CR。

关键路径：

```
Client invocation
  ↓
Router checks x-agentcube-session-id
  ↓
WorkloadManager creates Sandbox or SandboxClaim
  ↓
agent-sandbox starts/adopts Pod
  ↓
Router reverse-proxies to PodIP/entrypoint
  ↓
Redis/ValKey tracks session TTL and activity
```

它最重要的架构判断是不重写 `Sandbox controller`。`WorkloadManager` 只把 `AgentCube` 的 `AgentRuntime / CodeInterpreter` 翻译成 [agent-sandbox](#) CRD，底层 lifecycle、WarmPool adoption、Pod/PVC/Service 都交给 `agent-sandbox`。

安全上，PicoD 用 Router-signed JWT 替代 SSH；这简化 SDK，但 Router 变成强信任边界，后续需要 auth、mTLS、RBAC、审计一起兜住。

[OpenShell](#): Gateway desired state, Supervisor runtime enforcement

[OpenShell](#) 的分层边界最安全工程化:

```
Gateway owns desired state
  ↓
compute driver provisions sandbox
  ↓
Supervisor owns runtime enforcement
  ↓
policy proxy controls process/network/inference egress
```

Gateway 保存 sandbox、provider、policy、settings、inference、sessions 等状态；sandbox 内 Supervisor 才真正执行网络、进程、文件、TLS、provider credential 注入和 proxy enforcement。这个边界成立，因为只有 sandbox 内能看见进程身份、socket、binary hash、TLS 流和运行时凭据。

OpenShell 的关键不是“有 sandbox”，而是把 Agent egress 变成可审计、可裁决、可提案的策略流。普通网络访问走 OPA；inference.local 是特殊本地能力，代理到 provider，但 Agent 不直接持有真实 key。

[NemoClaw](#): OpenShell 上的 host-side Agent 编排

[NemoClaw](#) 是 OpenShell 的产品化编排层，不是另一个推理或 sandbox 引擎。它把用户真正要完成的步骤串起来：

- preflight 检查 Docker/OpenShell/GPU/DNS。
- 启动或复用 named OpenShell gateway。
- 选择 provider/model/credential route。
- 创建或复用 sandbox。
- 安装/同步 OpenClaw 或 Hermes runtime config。
- 应用 baseline policy 和 presets。
- 编译 messaging channel manifest。

它的核心是可恢复 onboard FSM：host-side JSON registry、session state、全局 lock、resume/fresh/non-interactive。这个模式适合复杂本地开发工具，但也带来状态恢复、部分写入和版本迁移成本。

[HiClaw](#): Agent 是 CR，也是 IM 用户

[HiClaw](#) 的设计很鲜明：把 Agent 运维做成 K8s operator，同时把协作面放到 Matrix IM。

```
Worker CR
  ↓
controller reconcile
  ↓
```

```
Matrix user + room
  ↓
Higress consumer key
  ↓
Worker container
  ↓
Human / Manager / Workers collaborate in rooms
```

这解决两个常被 Agent framework 忽略的问题：

- 人类如何自然介入：直接进 Matrix 房间。
- Agent 凭据如何隔离：Worker 只拿 Higress consumer key，真实 provider/API key 在网关侧。

它的代价是平台复杂度高。每个 Worker 是容器、Matrix 用户、gateway consumer 三位一体；Provisioner 同时编排 Matrix/Higress/Credentials/Manager bootstrap，横向拆分压力已经出现。

[AgentScope](#)：应用框架里的 runtime 边界

[AgentScope](#) 不提供强 sandbox 原语，但它定义了生产 Agent 应用的运行时边界：

- AgentEvent / Msg 是 SDK、SSE、存储和 UI 的统一协议。
- ReAct loop 把人类确认和外部执行建模成 continuation event。
- Tool execution 被拆成权限/上下文写入和 raw I/O。
- Workspace 承接 tools、MCP、skills 和 context/tool-result offload。
- FastAPI service 每轮从 storage/session/workspace 重新组装 Agent。

这类框架更适合应用服务，而不是底层隔离。它可以把 Docker/E2B/未来 agent-sandbox 作为 Workspace backend，把自身保持在 Agent 编排与事件流层。

[agentgateway](#)：Agent 访问外界的网络控制点

[agentgateway](#) 解决的是 Agent 流量，而不是 Agent 生命周期。它把 LLM API、MCP tools、A2A 通信统一到一个 Rust L7 数据面，控制面复用 Gateway API/xDS。

对 runtime/sandbox 体系来说，它的价值是：

- LLM provider 统一协议和 guardrails。
- MCP tools federation + CEL RBAC。
- A2A Agent Card URL rewrite 和代理。
- HBONE/mTLS/telemetry/access log。
- 用声明式 CRD 管路由、策略和后端。

如果 [agent-sandbox](#) 是“Agent 跑在哪里”，[agentgateway](#) 就是“Agent 能访问什么、怎么访问、怎么审计”。

核心难点

1. 生命周期不是 Pod 生命周期

Agent sandbox 的生命周期包含：创建、恢复、暂停、续期、过期、热领养、GC、startup resume、长任务 offload。这比普通 Pod 的 Running/Terminated 更复杂。

- [agent-sandbox](#) 用 shutdownTime 和 0/1 replicas 表达暂停/过期。
- [openkruise-agents](#) 用 SandboxSet / SandboxClaim 表达 pool 与领取，用 E2B API 管 pause/resume/clone/checkpoint。
- [agentcube](#) 用 Redis sorted set 管 session TTL 和 last activity。
- [OpenShell](#) 用 Gateway store + driver watch/reconcile/resume。
- [NemoClaw](#) 用 host-side FSM 和 registry 做 onboarding resume。

结论：Agent runtime 必须显式建模“会话”和“工作负载”的差异。

2. 凭据不能进 Agent 进程

Agent 会执行模型生成的工具调用，也会读取用户输入。真实 API key、GitHub PAT、云 AK 如果进入 Agent 进程，就会被 prompt injection 放大。

成熟方向都是网关托管：

- [HiClaw](#)：真实 key 在 Higress，Worker 只持 consumer key。
- [OpenShell](#) / [NemoClaw](#)：Gateway 是 provider credential system-of-record，sandbox 通过 inference.local 使用模型。
- [agentgateway](#)：后端 credentials 与 policy/filter 在网关侧。

这和 [agent-credential-isolation](#) 是同一条设计线。

3. 网络策略需要进程级语义

K8s NetworkPolicy 只能管 Pod 维度，Agent sandbox 内可能有多个进程：agent、tool、shell、browser、package manager。[OpenShell](#) 通过 proxy + /proc/net/tcp + binary identity 把网络裁决细化到进程/二进制身份，这是更细粒度的 runtime enforcement。

对于大多数平台，第一阶段可以用 NetworkPolicy + egress proxy；高风险场景才需要 OpenShell 这种 Supervisor/proxy 模型。

4. WarmPool 是性能优化，也是所有权协议

预热不是“提前启动 Pod”这么简单。关键难点是 ready sandbox 被 claim 领养时，ownerReference、labels、status、Pod exclusivity 必须一致。

[agent-sandbox](#) 的 SandboxClaim adoption 协议是底层实现；[agentcube](#) 只在 CodeInterpreter warmPoolSize 上暴露能力。这是合理分工：底层保证所有权不变量，上层只表达“我需要低冷启动”。[openkruise-agents](#) 则把 warm pool 升级成平台级 SandboxSet + SandboxClaim 协议，并在 claim 时叠加 inplace update、dynamic CSI mount、runtime init 和 identity token。

5. 人在回路不是 UI 附加项

多 Agent 系统需要人类在任务中途介入、批准、观察和接管。

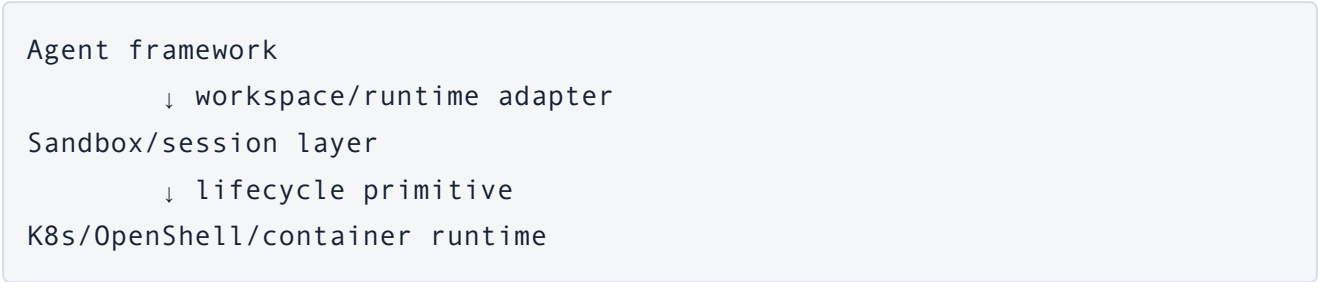
- [HiClaw](#) 用 Matrix 房间把人类和 Agent 放在同一个协作平面。
- [AgentScope](#) 用 RequireUserConfirmEvent 和 continuation event 把人类确认放进事件流。
- [OpenShell](#) 的 policy proposal 默认人工审批。

人在回路应该是协议和状态机能力，不应该只是一个前端按钮。

6. App framework 和 sandbox runtime 要分层

AgentScope、nanobot 这类应用框架解决“如何思考、调用工具、持久化上下文”；agent-sandbox/OpenShell 解决“如何安全运行”。把两层混在一起会导致框架既要懂 ReAct，又要懂容器、网络、凭据、恢复，复杂度失控。

比较稳的组合是：



设计分型

分型	代表项目	设计重心	适合场景
Sandbox 原语型	agent-sandbox	CRD、Pod/PVC/Service、WarmPool、K8s 原语	云原生 Agent 执行环境底座
Session 编排型	agentcube	HTTP invocation、session registry、Router/WorkloadManager	Code interpreter / serverless agent runtime
安全 runtime 型	OpenShell	Gateway desired state、Supervisor、policy proxy	高风险 autonomous agent、凭据隔离、网络审计
Host-side 编排型	NemoClaw	onboarding FSM、provider/policy/agent setup	把安全 runtime 包成可用产品

分型	代表项目	设计重心	适合场景
多 Agent 平台型	HiClaw	CRD + IM + 网关凭据 + Worker/Team	企业多 Agent 协作和运维
应用框架型	AgentScope , nanobot	ReAct loop、event stream、tools、workspace	构建 Agent 应用或个人 Agent
网络网关型	agentgateway	LLM/MCP/A2A 流量、policy、telemetry	Agent 外部访问治理

选型建议

目标	优先看	工程关注点
想把 Agent 容器建模成 K8s 原语	agent-sandbox	Sandbox/Claim/WarmPool、PVC 生命周期、NetworkPolicy
想做 Code Interpreter / AgentRuntime 会话服务	agentcube	Router/WorkloadManager、session registry、WarmPool adoption
想研究最细的安全 sandbox runtime	OpenShell	Gateway/Supervisor 分界、policy proxy、inference.local
想把 OpenShell 包成用户可用 CLI workflow	NemoClaw	onboard FSM、provider/policy/messaging manifest
想做多 Agent 协作平台	HiClaw	CRD、Matrix、Higress、credential isolation
想做 Agent 应用服务	AgentScope	AgentEvent、tool permission、workspace/offload、FastAPI
想治理 Agent 的 LLM/MCP/A2A 流量	agentgateway	Gateway API、xDS、CEL、MCP federation、guardrails

相关页面

- [agent-sandbox](#)
- [agentcube](#)
- [HiClaw](#)
- [agentgateway](#)
- [declarative-agent-management](#)
- [agent-credential-isolation](#)
- [cloud-native-security](#)
- [ai-agent-plugin-patterns](#)

- mcp

agent-sandbox

kubernetes-sigs/agent-sandbox 是 K8s SIG Apps 官方孵化的 Sandbox CRD + controller。把 AI Agent runtime 那种"长寿命、有状态、单实例、可暂停、有稳定身份"的容器形态建模成第一类 K8s 资源——比 Deployment（无状态副本）和 StatefulSet（编号 Pod）都更精准。隔离机制完全委托给标准 K8s 原语（gvisor / kata-containers / network-policy），controller 只做生命周期编排。

详细架构见 [src-agent-sandbox-architecture](#)。

Sandbox 生命周期图



关键事实

- 仓库: [kubernetes-sigs/agent-sandbox](#)
- 当前核验: GitHub API 显示 main 最近 push 2026-06-11, 最新稳定 tag v0.4.6, 另有 v0.5.0rc1
- 本 wiki 架构分析版本: v0.4.5+11 (HEAD e1d8898, 2026-04-23 左右)
- 主要语言: Go 1.26.2, 模块 sigs.k8s.io/agent-sandbox
- 4 个 CRD: 核心 Sandbox (agents.x-k8s.io) + 扩展 SandboxClaim / SandboxTemplate / SandboxWarmPool (extensions.agents.x-k8s.io)
- 治理: K8s SIG Apps 孵化项目, 遵循 [Kubernetes Resource Model](#) 范式
- SDK: Go + Python (codegen 生成) + 标准 K8s clientset/informer/lister
- examples 目录当前超过 20 个: quickstart / hello-world / kata-gke / openclaw / hermes / langchain / jupyter / vscode / chrome / HPA / Kueue / Cilium/NetworkPolicy / MCP / Ray / Python SDK 等

与 [HiClaw](#) 的关系

互补不竞争——agent-sandbox 是基础设施层（提供"安全运行 1 个有状态 Agent 容器"原语），HiClaw 是应用层（提供"管 N 个 Agent 协作 + IM 平面 + 凭据网关"）。理论上 HiClaw 的 Worker runtime 完全可以跑在 agent-sandbox 之上。

与 [HiClaw](#) 的完整对比

维度	agent-sandbox	HiClaw
层级	Runtime substrate / CRD 原语	多 Agent 协作平台
核心对象	Sandbox、Claim、Template、WarmPool	Worker、Team、Human、Manager
协作模型	不管 IM / team / manager	Matrix 房间 + Manager/Worker/Human
凭据模型	由 PodTemplate / 用户环境决定	Gateway consumer key + 真凭据托管
隔离模型	委托 gVisor/Kata/NetworkPolicy/PodTemplate	依赖容器后端，可进一步叠 agent-sandbox
启动优化	WarmPool / Template / suspend-resume	Worker lifecycle + runtime package
典型用户	平台工程师、runtime 基座维护者	Agent 平台/协作产品维护者

GKE Kata walkthrough 速记

1. 准备支持 Kata/gVisor 的 GKE 或兼容集群。
2. 安装 agent-sandbox CRD/controller。
3. 创建带 runtimeClass / securityContext / volume / network policy 的 SandboxTemplate。
4. 可选创建 SandboxWarmPool，预热若干 Pod。
5. 用户创建 SandboxClaim，controller 从 WarmPool 领取或新建 Sandbox。
6. Sandbox Ready 后，通过 service / SDK / router 访问容器内 agent 或工具。
7. 空闲后 scale to zero / suspend，保留 PVC 状态，后续 resume。

这个流程说明它的关键不是“创建 Pod”，而是把单实例、有状态、可暂停、可领取的 Agent 容器变成可声明和可复用的资源。

v0.5 / beta 路线

roadmap 当前显示几条主线：

- API 与 runtime 解耦，走 portable backend / common proto；
- SandboxTemplate / WarmPool rolling update；
- router 成为 first-class 项目组件；

- auto suspend/resume、scale-to-zero、TFFI 延迟优化；
- 智能 WarmPool 选择和 claim latency 优化；
- alpha → beta API versioning；
- TypeScript SDK、MCP server、Ray / LangChain / CrewAI / kAgent 集成。

这意味着 v0.5 之后的重点会从“CRD 能跑”走向“多后端、低延迟、SDK、router、生产指标”。

examples 导读

类别	examples
入门	quickstart,hello-world-sandbox,python-sdk-quickstart
隔离 / runtime	kata-gke-sandbox,chrome-sandbox,sandboxed-tools,sandbox-ksa,policy
Agent / framework	openclaw-sandbox,hermes-agent,langchain,gemini-cu-sandbox,code-interpreter-agent-on-adk
扩缩 / 调度	hpa-swp-scaling,kueue-agent-sandbox>manual-pdb
网络策略	composing-sandbox-nw-policies,mcp-server-sandbox
工具型环境	jupyterlab,vscode-sandbox,aio-sandbox,analytics-tool,python-runtime-sandbox,ray-integration

OpenKruise Agents

OpenKruise Agents 是 OpenKruise 面向 AI Agent sandbox lifecycle management 的 Kubernetes 原生平台。它把 Sandbox / SandboxSet / SandboxClaim / SandboxTemplate CRD、E2B-compatible API、Envoy 路由扩展、agent-runtime sidecar、identity token 和动态 CSI mount 组合成一套“在 K8s 上运营 Agent sandbox”的控制面。

详细架构见 [src-openkruise-agents-architecture](#)。

关键事实

- 仓库: [openkruise/agents](#)
- 当前核验: GitHub API 显示默认分支 master, 最近 push 2026-06-12, 约 202 stars / 78 forks, Apache-2.0, 主语言 Go
- 本 wiki 分析版本: HEAD 0e58df8 (2026-06-12)
- 核心入口: agent-sandbox-controller、sandbox-manager、traffic-extension、sandbox-gateway
- 关键 API: Sandbox、SandboxSet、SandboxClaim、SandboxTemplate
- 产品入口: E2B-compatible / sandboxes、/ snapshots、pause/resume/connect/timeout/debug 等 API
- 典型场景: code interpreter、cloud desktop、Claude Code / OpenClaw sandbox、RL/HITL/open-world training、长寿命 workspace

架构定位

E2B 请求到 Sandbox 的流程

```
E2B API / CRD request → sandbox-manager
├─ create / claim / clone / checkpoint
└─ route + identity + key storage
    ↓
    SandboxClaim / SandboxSet / Sandbox
    ↓
    agent-sandbox controller
    ↓
    Pod + PVC + agent-runtime / CSI
    ↓
    Envoy ext_proc → Sandbox Pod IP:port
```

OpenKruise Agents 处在 [agent-sandbox](#) 和 [agentcube](#) 中间偏平台化的一层：

- 比 [agent-sandbox](#) 更完整：不仅有 sandbox lifecycle primitive，还把 warm pool、claim、E2B API、路由、identity、CSI 和 runtime sidecar 接起来。
- 比 [agentcube](#) 更底层：不主打 AgentRuntime / CodeInterpreter invocation 产品抽象，而是提供更通用的 sandbox 管理、clone/checkpoint、route 和 E2B 兼容面。
- 比 [substrate](#) 更 K8s CRD 化：Substrate 聚焦 actor/worker multiplexing 和 wake routing；OpenKruise Agents 聚焦可声明、可预热、可暂停/恢复、可路由的 sandbox。
- 可作为 Agent framework 的远端 workspace backend：AgentScope、coding agent、browser/desktop agent 可以把它当作 E2B-like 执行环境。

核心组件

组件	作用
agent-sandbox-controller	K8s controller-runtime 控制面，管理 Sandbox、SandboxSet、SandboxClaim、SandboxUpdateOps、SecurityTokenRefresh。
sandbox-manager	E2B-compatible HTTP API 控制面，负责 claim/clone/pause/resume/delete checkpoint、API key storage、team permission 和 route sync。
traffic-extension	Envoy external processing gRPC 服务，把 sandbox 请求改写为 original-dst 路由。
sandbox-gateway	Envoy Go HTTP filter 版本的数据面扩展，解析 sandboxID/port 并写 dynamic metadata。
sandboxcr infra	Infrastructure / Sandbox 抽象的 K8s CRD 实现，把 API 请求落到 SandboxClaim/Sandbox/PVC/Pod。
agent-runtime / identity / CSI	sandbox 内 runtime 初始化、短期 token 传播、动态 volume mount 的扩展点。

关键判断

- Warm pool 是控制面协议：SandboxSet 保持可用库存，SandboxClaim 显式领取；这比简单 Pod pool 更适合并发、超时、失败保留和滚动更新。
- E2B API 降低接入成本：上层 SDK 可以用 familiar API 创建、暂停、恢复、连接 sandbox；平台侧仍保留 K8s CRD/GitOps/RBAC 能力。
- 路由不走 Service per sandbox：Envoy ext_proc / Go filter 用 sandboxID 查 route registry，把流量导到 Pod IP:port，适合大量短会话/长会话混合的 sandbox。
- 安全边界要继续补齐：identity provider、API key storage、token propagation、runtimeClass、NetworkPolicy 和 audit 是生产采用前必须验证的部分。

什么时候优先看它

- 需要在 K8s 上搭建 E2B-like sandbox service;
- 希望同时拥有 CRD 管理面和 SDK/API 产品面;
- 需要 warm pool 降低 Agent / Code Interpreter 冷启动;
- 需要 pause/resume、checkpoint/clone、动态存储挂载、路由和多租户 namespace/team;
- 想把 AI Agent workspace 从本地/Docker/E2B SaaS 迁到自有 Kubernetes。

什么时候不适合

- 只需要一个本地 Agent 框架或 Python workflow runtime;
- 只想学习最小 sandbox CRD，不想同时理解 manager/gateway/identity/CSI;
- 对强 runtime enforcement 的要求高于生命周期管理，此时还要叠加 cloud-native-security、OPA/egress gateway、gVisor/Kata/seccomp 等机制;
- 当前业务不能接受快速演进 API 和 roadmap 中尚未完成的 storage/network/runtime/scheduling/observability 能力。

agentgateway

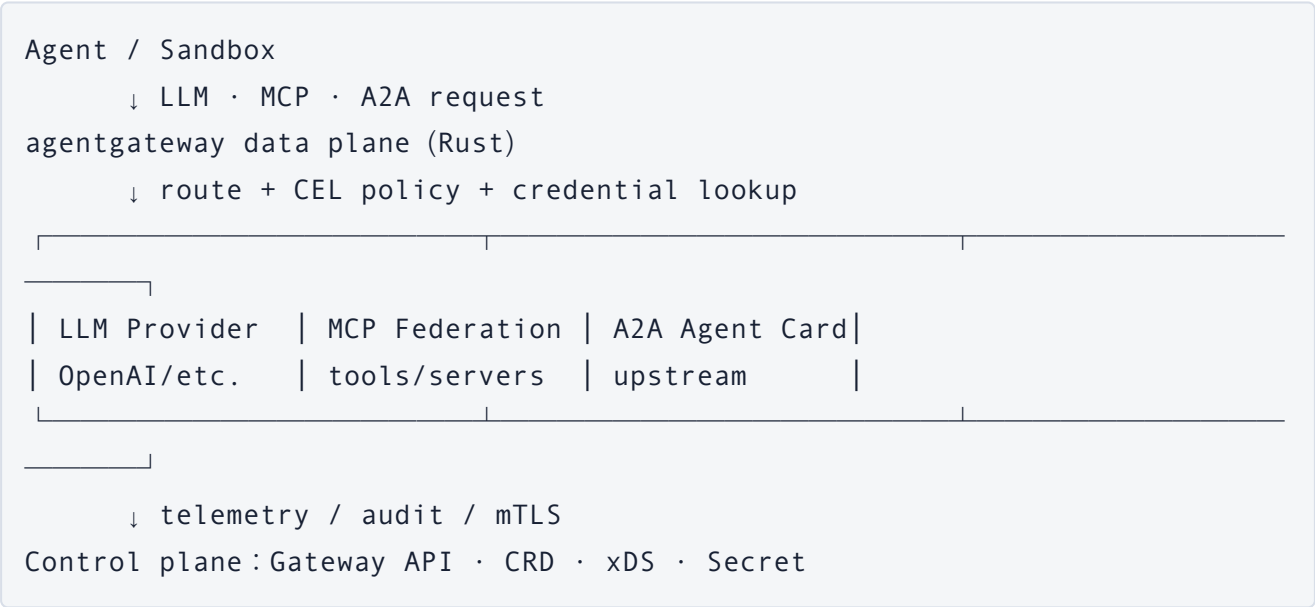
Solo.io / Istio 系出品的 AI-native L7 网关（Apache 2.0）。把三种 AI 流量统一在一个 Rust 数据面里：

- **LLM Gateway**: OpenAI / Anthropic / Gemini / Bedrock / Vertex / Azure / Copilot 七家 provider 统一为 OpenAI 兼容 API + guardrails
- **MCP Gateway**: stdio / HTTP Streamable / SSE 三种 transport + OpenAPI 自动转 MCP 工具 + 多 MCP server federation
- **A2A Gateway**: Google Agent-to-Agent 协议代理，通过重写 Agent Card 让所有调用走向 gateway

架构骨架

- **控制面 (Go)** : Gateway API + 自家 CRD (AgentgatewayPolicy / AgentgatewayBackend / AgentgatewayParameters) , KRT 反应式表, xDS Delta ADS 分发
- **数据面 (Rust)** : tokio + hyper + rustls, proxy/gateway → http/route → llm|mcp|a2a → upstream, CEL 做策略 IR, HBONE 跟 Istio mesh 互操作
- **UI (TypeScript)** : Next.js 管理界面

Agent 出口请求流程



核心设计哲学

1. 沿用 Istio 基建做 AI Gateway: Gateway API + KRT + xDS + HBONE 全套复用，只把数据面换成专为 AI 协议优化的 Rust 代理
2. 三协议共享同一条 pipeline: Route / Policy / Backend / CEL 求值统一，差别只在 Backend.kind 决定走哪个 provider 适配器

3. 凭据托管：LLM provider 真凭据放在 backend secret，Agent 只面对 gateway 自身认证，参见 [agent-credential-isolation](#)
4. 跟 [agent-sandbox](#) 互补：前者出口流量治理，后者运行时隔离，并集 = AI 工作负载完整治理面

详见 src-agentgateway-architecture。

LLM provider 适配器对照

Provider	请求侧职责	响应侧职责	典型差异
OpenAI / OpenAI-compatible	作为内部标准格式，基本透传	返回 OpenAI-compatible SSE / JSON	其他 provider 多向它收敛
Anthropic	OpenAI messages → Anthropic messages / tools	Anthropic event stream → OpenAI chunk	thinking/tool schema / stream event 名称不同
Gemini / Vertex	OpenAI chat → generateContent / Vertex endpoint	Gemini candidates → OpenAI choices	project/location/auth 和多模态字段不同
Bedrock	OpenAI chat → Bedrock invoke，叠加 AWS SigV4	Bedrock body/stream → OpenAI-compatible	AWS auth、model id、区域化 endpoint
Azure OpenAI	OpenAI schema + Azure deployment endpoint	Azure OpenAI response → OpenAI-compatible	deployment 名称、api-version、endpoint path
Copilot	适配 GitHub Copilot provider 入口	统一成 OpenAI-compatible	认证和可用模型集合不同

agentgateway 的关键不是“支持更多 provider”本身，而是把 provider 差异压到 `AIProvider / Provider trait` 后面。上层 `HTTPRoute`、`Policy`、`Backend`、`CEL`、`telemetry` 不需要知道具体 provider 细节。

MCP federation 与 CEL RBAC

MCP Gateway 把多个 MCP server 聚合到一个入口。设计重点是命名空间和授权：

- transport 层支持 stdio / HTTP Streamable / SSE，上游协议差异由 `mcp/upstream` 吸收；
- federation 时要避免工具名冲突，通常需要按 server 或 route 分命名空间；
- 权限不应散落在工具实现里，而应在 gateway route / policy 层用 cel 表达式做统一判定；
- agent 只拿 gateway 侧身份，后端 MCP server 的真实凭据仍由 gateway 管。

这和 [HiClaw](#) 的 worker consumer key 模式一致：Agent 能调用哪些工具，由网关侧 consumer、route 和 policy 决定，而不是由 prompt 里的自觉性决定。

和同类 AI Gateway 的边界

项目	主要定位	agentgateway 的差异
LiteLLM	Python LLM proxy / provider adapter	agentgateway 更偏 Rust 数据面 + Gateway API/xDS/mesh 集成
OpenRouter	托管模型路由服务	agentgateway 是可自托管 infra 组件，控制面和策略在用户集群里
Portkey	托管/企业 LLM gateway	agentgateway 更云原生，强调 CRD、mesh、MCP/A2A
Envoy AI Gateway	Envoy Gateway 上的 GenAI gateway	agentgateway 不复用 Envoy 数据面，而是自写 Rust 数据面并同时覆盖 LLM/MCP/A2A

部署 walkthrough 速记

1. 安装 Gateway API CRD 与 agentgateway 自家 CRD。
2. 部署 controller，它 watch Gateway / HTTPRoute / AgentgatewayPolicy / AgentgatewayBackend。
3. 创建 GatewayClass 与 Gateway，由 AgentgatewayParameters 控制 dataplane 部署参数。
4. 创建 AgentgatewayBackend，把 AI/MCP/A2A 后端和 secret 挂进去。
5. 用 HTTPRoute.backendRef 指向 backend，并用 AgentgatewayPolicy 加鉴权、限流、转换或 CEL RBAC。
6. dataplane 通过 xDS 增量接收配置，开始处理 LLM / MCP / A2A 流量。

如果需要运行时隔离，把 Agent 容器放在 [agent-sandbox](#) / [agentcube](#) 一类 substrate 上；如果需要 LLM serving endpoint picking，把入口和 gateway-api-inference-extension / llm-d 对齐。

Agent Runtime / Sandbox 细分选型地图

已有 [agent-runtime-sandbox-project-map](#) 做了完整分层。这页面向选型：你需要的是 sandbox primitive、E2B-like sandbox 平台、session API、安全 runtime、多 Agent 平台，还是 AI gateway。

GitHub 当前核验

截至 2026-06-15 通过 GitHub API 重新核验：

项目	仓库	最近 push	stars	主语言	当前定位
agent-sandbox	https://github.com/kubernetes-sigs/agent-sandbox	2026-06-10	2.8k	Go	K8s-native isolated stateful singleton workload
openkruise-agents	https://github.com/openkruise/agents	2026-06-12	202	Go	K8s-native agent sandbox lifecycle platform
agentcube	https://github.com/volcano-sh/agentcube	2026-06-08	147	Go	Sandbox session orchestration
OpenShell	https://github.com/NVIDIA/OpenShell	2026-06-11	7k	Rust	Safe private runtime for autonomous agents
NemoClaw	https://github.com/NVIDIA/NemoClaw	2026-06-11	21k	TypeScript	OpenShell 内运行 Hermes/OpenClaw
HiClaw	https://github.com/agentscope-ai/HiClaw	2026-06-10	4.8k	Go	Matrix + K8s 的 collaborative multi-agent OS
AgentScope	https://github.com/agentscope-ai/agentscope	2026-06-09	26k	Python	可观测、可理解、可信任的 Agent 构建框架
agentgateway	https://github.com/agentgateway/agentgateway	2026-06-10	3.2k	Rust	AI agents / MCP servers 的 agentic proxy

分层选择

你要解决的问题	首选	不适合
K8s 上安全运行一个有状态 Agent 容器	agent-sandbox	需要完整产品 API、IM 协作或工具治理

你要解决的问题	首选	不适合
自托管 E2B-like sandbox service	openkruise-agents	只想要最小 CRD 或不能接受快速演进的 manager/gateway/runtime 组合
把 sandbox 包成 session/invocation API	agentcube	不使用 Kubernetes 或不想引入 Redis/session registry
本地/私有强安全 Agent runtime	OpenShell	只需要普通容器隔离
给 OpenShell 配套 agent onboarding	NemoClaw	只想写一个 Agent 应用框架
多 Agent 协作 + 人在回路	HiClaw	不想引入 Matrix / K8s operator
Python Agent 应用服务化	AgentScope	需要强 sandbox 原语
LLM/MCP/A2A 流量治理	agentgateway	需要运行 Agent 进程本身

架构边界



[agent-sandbox](#) 处在 sandbox lifecycle primitive；[openkruise-agents](#) 处在 sandbox platform layer，把 CRD、E2B API、Envoy route、runtime/CSI/identity 扩展组合起来；[OpenShell](#) 处在 runtime enforcement；[agentgateway](#) 处在 network/tool gateway；[HiClaw](#) 和 [agentcube](#) 是上层产品化编排；[AgentScope](#) 是应用框架层。

按问题定位项目

我要解决什么问题？

— 写 Agent 应用 / event loop

→ AgentScope

— 提供 session / invocation API

→ AgentCube

— 管理 K8s sandbox 生命周期

→ agent-sandbox

— 做 E2B-like 平台

→ OpenKruise Agents

└─ 做进程/网络/凭据强制隔离	→ OpenShell
└─ 做多人协作与人在回路	→ HiClaw
└─ 管理 LLM/MCP/A2A 出口	→ agentgateway

关键对比

维度	agent-sandbox	openkruise-agents	OpenShell	agentcube	HiClaw	agentgateway
控制面	K8s controller	sandbox-manager + K8s controllers	Gateway desired state	Router + WorkloadManager	K8s operator + Matrix/Higress	Gateway API/xDS
数据面	Pod/PVC/Service	Envoy ext_proc / Go filter + Sandbox Pod	supervisor + policy proxy	Sandbox pod + PicoD	Worker containers + Matrix rooms	Rust L7 proxy
状态	K8s API + PVC	K8s API + route registry + key storage	object store + sandbox state	Redis/ValKey session registry	etcd/kine + Matrix + MinIO	route/backend/policy stores
凭据	交给上层	E2B API key storage + security token propagation	gateway 托管 provider key	Router JWT / workload auth	Higress 托管真实凭据	backend secret / policy
用户入口	CRD / SDK	E2B API + CRD	CLI / SDK	HTTP / Python SDK / Dify	Matrix / CRD / CLI	CRD / Gateway / UI

采用建议

- 做“Agent 运行在 K8s 里的基础设施”：先看 [agent-sandbox](#)；需要 E2B-compatible API、warm pool、路由和 runtime 扩展时看 [openkruise-agents](#)；需要 invocation/session 产品 API 时再看 [agentcube](#)。
- 做“安全私有 Agent 运行时”：先看 [OpenShell](#)，再看 [NemoClaw](#) 如何编排。
- 做“团队协作 Agent 平台”：[HiClaw](#) 的 Matrix-first 协作面比普通 Agent framework 更贴近人类介入。
- 做“工具/模型访问治理”：[agentgateway](#) 应作为 sandbox 外侧的流量控制点。

MCP Server / Tool Gateway 对比地图

MCP 生态已经从“给 Agent 接几个工具”扩展成三层：SDK/framework、具体工具 server、gateway/proxy/data plane。本页把 FastMCP、官方/垂直 MCP server 和 agentgateway/Plano 放在一起看。

GitHub 当前核验

截至 2026-06-11 通过 GitHub API 重新核验：

项目	仓库	最近 push	stars	主语言	定位
FastMCP	https://github.com/PrefectHQ/fastmcp	2026-06-06	25k	Python	Pythonic MCP servers/clients
GitHub MCP	https://github.com/github/github-mcp-server	2026-06-10	30k	Go	GitHub 官方 MCP server
Playwright MCP	https://github.com/microsoft/playwright-mcp	2026-06-10	33k	TypeScript	浏览器自动化 MCP server
kubectl MCP	https://github.com/rohitg00/kubectl-mcp-server	2026-04-08	908	Python	Kubernetes MCP server
agentgateway	https://github.com/agentgateway/agentgateway	2026-06-10	3.2k	Rust	LLM/MCP/A2A agentic proxy
Plano	https://github.com/katanemo/plano	2026-06-09	6.5k	Rust	AI-native proxy and data plane

分层

层	代表	解决的问题
MCP SDK/framework	FastMCP, MCP Python SDK	快速写 server/client，处理 schema、transport、lifecycle
Tool server	GitHub MCP, Playwright MCP, kubectl MCP	把 GitHub、浏览器、K8s 等具体能力暴露给 Agent
Gateway/proxy	agentgateway , Plano	federation、鉴权、RBAC、observability、routing、policy

选型

需求	选择
快速给内部 API 写 MCP server	FastMCP
让 Agent 操作 GitHub issue/PR/repo	GitHub MCP
让 Agent 控浏览器做验证/抓取	Playwright MCP
让 Agent 读 K8s 集群状态	kubectl MCP，但要强 RBAC 和只读优先
多 MCP server federation + policy	agentgateway
Agentic app data plane / smart routing	Plano

架构差异

维度	FastMCP	GitHub MCP	Playwright MCP	kubectl MCP	agentgateway / Plano
抽象	SDK/ framework	domain server	browser server	K8s server	gateway/proxy
主要风险	schema/鉴权 自建	token 权限过宽	浏览器可执行外部 站点动作	集群权限过 宽	策略和路由复杂
运维边界	app 进程	server + GitHub token	browser runtime	kubeconfig / RBAC	数据面 + 控制面
适合规模	单 server 到 中型	team / repo workflow	自动化测试/浏览器 任务	集群诊断	多 server / 多 Agent / 多租户

避坑条件

- MCP server 默认不是安全边界；凭据、RBAC、allow-list 要外置治理。
- Browser/K8s/GitHub 这类高权限工具必须最小权限启动。
- 多 server 组合时，不要让 Agent 直接持有所有 token；应通过 [agent-credential-isolation](#) 和 gateway 管控。
- “支持 MCP”不等于可观测；生产环境要记录 tool call、参数、结果、token 和审批。

Agent Runtime Substrate

Agent runtime substrate 指承载大量 agent-like workload 的底层运行基座：worker/actor 生命周期、状态保存、唤醒路由、沙箱隔离、快照恢复、并发密度和调度策略。

为什么单独成概念

传统 [agent-sandbox](#) 解决“一个 Agent 会话如何安全运行”，而 substrate 解决“一批 Agent 会话如何高密度、可恢复、可路由地运行”。[openkruise-agents](#) 则把这条线推向 K8s 平台化：用 SandboxSet/Claim 管 warm pool 和领取协议，用 E2B API 暴露产品入口，用 Envoy route registry 把请求导向具体 sandbox。[src-substrate-architecture](#) 把这个方向推进到 WorkerPool/ActorTemplate、Redis/ValKey actor state、Envoy 唤醒路由、gVisor snapshot；[agentscope-runtime](#) 则代表 framework app 到 Agent-as-a-Service 的服务化路线。

从应用到执行环境

```
Agent App / ReAct Loop
  ↓
Session API / Invocation
  ↓
Workload Manager / Router
  ↓
Sandbox Claim / Warm Pool
  ↓
Pod + RuntimeClass + PVC + NetworkPolicy
  ↓
gVisor / Kata / Container / VM
```

项目对比

项目	层级	重点
agent-sandbox	sandbox primitive	单会话隔离和有状态容器
openkruise-agents	sandbox lifecycle platform	SandboxSet/Claim warm pool + E2B API + Envoy route + runtime/CSI/identity 扩展
agentcube	session orchestration	Router + WorkloadManager + WarmPool
Substrate	runtime substrate	actor/worker multiplexing + snapshot/wakeup

项目	层级	重点
agentscope-runtime	Agent-as-a-Service	Agent app 服务化和部署器

选型提示

如果问题是安全隔离，看 sandbox；如果问题是 Kubernetes 上的 E2B-like sandbox 平台，看 [openkruise-agents](#)；如果问题是高密度运行、唤醒和恢复，看 substrate；如果问题是把 framework app 暴露成服务，看 Agent-as-a-Service runtime。

Agent Substrate 架构与设计思路分析

原文: raw/substrate-architecture-analysis.md · 仓库: <https://github.com/agent-substrate/substrate> · 分析版本 HEAD a3f4474

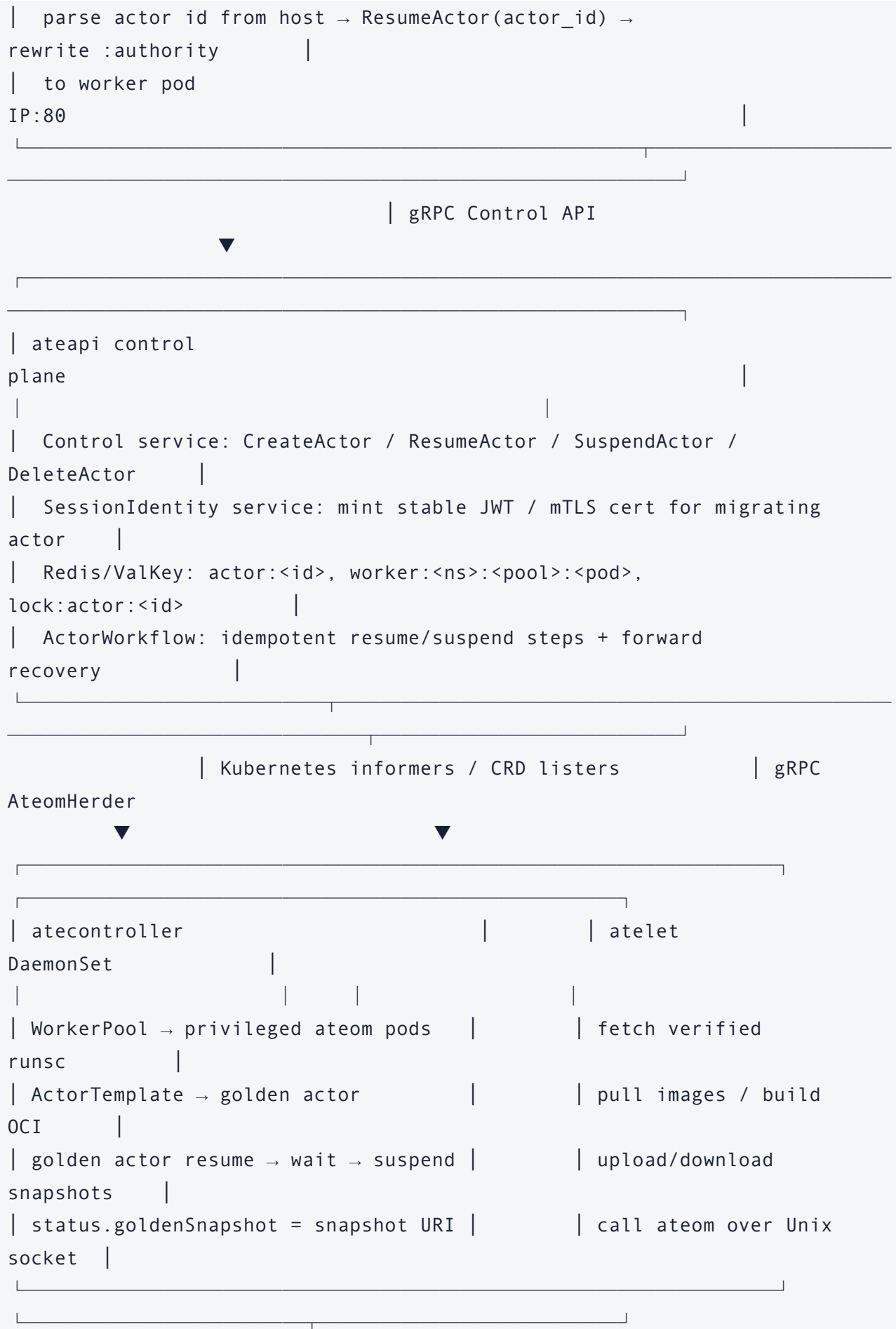
一句话定位

Agent Substrate 是一个建立在 kubernetes 之上的高密度 [Agent Runtime / Sandbox](#) substrate: Kubernetes 只负责声明式容量和基础设施, Actor 的高频生命周期、resume/suspend、worker 分配和路由唤醒由 Substrate 自己的 Redis/ValKey 控制面、Envoy ext_proc router、atelet node supervisor 和 gVisor atecom worker 承担。

它补的是 [agent-sandbox](#) 和 [agentcube](#) 之间更底层、更激进的一层: 不是“创建一个有状态 sandbox”, 而是把大量 idle actor multiplex 到较少 warm worker 上, 通过 golden snapshot、runtime snapshot、DNS/router 唤醒和 gVisor restore 降低激活延迟。官方 architecture 文档也明确标注“Much of this architecture is aspirational”, 所以要把愿景和当前代码实现分开看。

核心架构图







```

| ateom-gvisor worker
pod
|
| privileged pod, one active actor at a
time
| link-local veth: pod netns ateom0 ↔ actor netns
eth0
| nftables DNAT/MASQ compatibility
bridge
| runc create/start/checkpoint/restore pause + app
containers

```



```

| Object storage
snapshots
|
| gs:// or s3://
prefix
| checkpoint.img.zstd / pages.img.zstd /
pages_meta.img.zstd

```

模块分层

层 / 模块	职责
CRD / public API	定义 WorkerPool、ActorTemplate 和 actor lifecycle gRPC API; K8s CRD 承载低频配置, gRPC 承载高频 actor 操作。
K8s controller	把 WorkerPool 物化为 privileged ateom Deployment; 把 ActorTemplate 驱动成 golden actor → golden snapshot → Ready。

层 / 模块	职责
Control plane / DB	对外提供 Control / SessionIdentity gRPC; 用 Redis/ValKey 存 actor/worker 状态和 lock; 用 workflow step 实现 resume/suspend forward recovery。
Node supervisor	每节点/每 worker 管理 runsc、镜像拉取、OCI bundle、snapshot 上传下载, 并转调 ateam。
Sandbox herder	privileged worker pod 内的 gVisor 控制进程, 串行调用 runsc create/start/checkpoint/restore, 管理 actor netns/veth/nftables。
Router / DNS	用 CoreDNS stub domain + Envoy xDS + ext_proc, 把 actor hostname 转成 ResumeActor 调用和 worker pod IP 转发。

关键边界: kubernetes API 不存每个高频 actor transition; WorkerPool/ActorTemplate 是声明式配置, Actor/Worker 的运行状态在 Redis/ValKey, 真正 runsc 操作在 worker pod 本地完成。

关键数据流

ActorTemplate 到 golden snapshot

```

ActorTemplate CR created
|
▼
atecontroller ActorTemplateReconciler
|   PhaseInitial
▼
ateapi.CreateActor(goldenActorID)
|
▼
status.phase = ResumeGoldenActor
|
▼
ateapi.ResumeActor(goldenActorID)
|   no actor snapshot + no golden snapshot
▼
atelet.Run → ateam.RunWorkload → runsc create/start
|
▼
wait 20s
|
▼

```

```

ateapi.SuspendActor(goldenActorID)
|
▼
atelet.Checkpoint → ateam.CheckpointWorkload → upload checkpoint files
|
▼
ActorTemplate.status.goldenSnapshot = snapshot URI
ActorTemplate.status.phase = Ready

```

请求唤醒和路由

```

HTTP request
Host: <actor-id>.actors.resources.substrate.ate.dev
|
▼
CoreDNS stub domain routes to atenet-router
|
▼
Envoy RequestHeaders ext_proc
|
▼
parseActorID(host)
|
▼
singleflight ResumeActor(actor-id)
|
▼
ateapi resume workflow
|
├─ LoadActorForResume
├─ AssignWorker
├─ CallAteletRestore / Run
└─ FinalizeRunning
|
▼
actor.ateom_pod_ip returned
|
▼
mutate :authority to <worker-ip>:80
|

```

▼
Envoy dynamic_forward_proxy forwards request

Resume / Suspend workflow

```
ResumeActor(actor-id)
  |
  ▼
  Acquire lock: lock:actor:<id>
  |
  ▼
  Load actor + ActorTemplate
  |
  ▼
  Find existing assigned worker or random free worker
  |
  ▼
  Update worker(actor_id=...) then actor(status=RESUMING, worker fields)
  |
  ▼
  if actor.last_snapshot: atelet.Restore(last_snapshot)
  else if template.goldenSnapshot and !boot: atelet.Restore(goldenSnapshot)
  else: atelet.Run(template spec)
  |
  ▼
  Update actor(status=RUNNING)

SuspendActor(actor-id)
  |
  ▼
  Acquire lock: lock:actor:<id>
  |
  ▼
  Load actor + ActorTemplate
  |
  ▼
  Mark actor SUSPENDING + in_progress_snapshot
  |
  ▼
  atelet.Checkpoint(snapshot URI)
```



```
Free worker, move in_progress_snapshot → last_snapshot,  
clear worker fields, status=SUSPENDED
```

设计决策与哲学

- K8s 管低频配置，Substrate 管高频 actor 状态：WorkerPool/ActorTemplate 是 CRD，actor/worker 运行态进入 Redis/ValKey。Redis store 注释明确说明跨 slot 不能原子更新 actor 和 worker，所以系统必须接受 forward recovery。
- Golden snapshot 是冷启动优化根：ActorTemplate 会生成一个 golden actor，启动后暂停成 snapshot；后续 actor 可从 golden snapshot 恢复，而不是每次重新跑初始化。
- Resume/suspend 采用 client-driven forward recovery：每个 workflow step 都能判断是否已完成，失败后靠客户端重试继续推进；每个 actor 还有 lock:actor:<id> 防同 actor 并发操作。
- worker 是 warm capacity，不是 actor 本体：WorkerPool 控制 privileged atoom pods 的数量；Actor 只是逻辑记录，运行时才绑定某个 worker pod。
- gVisor/runsc 是当前 snapshot 执行后端：atelet 负责准备 runsc、OCI bundle 和 snapshot 文件；ateom-gvisor 串行调用 runsc 的 create/start/checkpoint/restore。
- 网络唤醒还是兼容桥阶段：当前路由按 actor DNS 唤醒后把请求改写到 worker IP:80；ateom 用 veth + nftables 做 actor ingress/egress 兼容，后续仍需要更严格的 gateway/policy 层，和 [agentgateway](#) 一类项目会形成互补。

关键组件深入解读

ateapi ActorWorkflow

ActorWorkflow 是当前源码里最能体现 Substrate 设计哲学的部分。它不是写一个大函数完成 resume/suspend，而是把流程拆成可跳过的步骤：resume 依次加载 actor、分配 worker、调用 atelet restore/run、最终置 running；suspend 依次加载 actor、标记 suspending、调用 atelet checkpoint、释放 worker 并提交 snapshot。

这个拆法服务于两个现实约束：actor 与 worker 状态跨 Redis Cluster key，不可能用一个强事务同时修改；调用 atelet/ateom/runsc 是外部副作用，失败可能发生在任意位置。因此 workflow 采用“状态已写入就能被下一次请求识别并继续”的模型。

atelet + ateam-gvisor

atelet 和 ateam 的分工很清楚：atelet 是 node supervisor，接收 ateapi 的 Run/Checkpoint/Restore 请求；ateom 是 worker pod 内的 gVisor herder，真正执行 runsc。atelet 负责下载/校验 runsc、拉 OCI image、展开 rootfs、生成 OCI config、下载/上传 checkpoint zstd 文件。ateom 负责创建 interior network namespace、建立 veth、安装 nftables、执行 runsc create/start/checkpoint/restore。

当前实现假设一个 worker pod 同时只运行一个 active actor；ateom 的 mutex 和 nftables 表清理逻辑都体现了这一点。

相关页面

- [agent-runtime-sandbox-selection-map](#)
- [agent-runtime-sandbox-project-map](#)
- [agent-sandbox](#)
- [agentcube](#)
- [kubernetes](#)
- [agentgateway](#)

Agent 凭据隔离

让 AI Agent 永远拿不到真实凭据（LLM API key / GitHub PAT / OSS AK），它只持一个网关侧颁发的 consumer key，所有外部访问经网关鉴权和路由。

HiClaw 通过 [Higress AI Gateway](#) 实现这个模式：

- 用户的真 API key 只存 Higress 的 secret 里
- 每个 Agent（[Worker](#) / Manager）被分配一个 Higress consumer + key
- Agent 调 LLM / OSS / MCP server 都经过 Higress，consumer key 决定能访问哪些路径
- 抗 prompt injection：即便 Agent 被劫持，攻击者拿到的也只是可被即时 revoke 的 consumer key——而不是真凭据

为什么重要

社区 skill 市场（如 [skills.sh](#) 上 80,000+ 社区作品）能否被企业放心装载，取决于这一点——传统框架（autogen / langgraph / crewai）让 Agent 直接持真凭据，prompt injection = 凭据失窃。

模式拆解

```
Agent runtime
| only has consumer key / short-lived token
▼
AI Gateway / Tool Gateway
| owns real provider keys, GitHub PAT, OSS AK, MCP credentials
| applies authn/authz/rate limit/audit/policy
▼
LLM provider / MCP server / storage / code host
```

关键点不是“把 key 放到另一个地方”这么简单，而是把权限决策从 prompt 可影响的 Agent 进程移到网关控制面：

- Agent 只看到统一 endpoint 和可撤销身份；
- 真凭据只在 gateway/backend secret/credential provider 内部使用；
- route / policy / consumer 决定 Agent 能访问哪些模型、工具、bucket、repo；
- audit log 记录的是“哪个 Agent 身份调用了哪个能力”，而不是一堆共享 API key。

代表实现

项目	隔离方式	适合借鉴点
HiClaw	Worker / Manager 只持 Higress consumer key，真实 LLM / OSS / MCP 凭据在网关侧	多 Agent 协作平台的默认安全边界
agentgateway	AgentgatewayBackend 保存 provider/backend secret，LLM/MCP/A2A 统一走 gateway policy	AI Gateway 作为出口流量和工具访问控制面
openkruise-agents	E2B API key storage、namespace-scoped team、identity provider 和 security token propagation，把 sandbox API 身份与运行时 token 分开	K8s sandbox platform 的 API key / short-lived token / runtime propagation 边界
loongsuite-pilot	不托管业务凭据，但在 telemetry 上统一执行 content policy / secret mask	观测链路里的隐私边界

凭据轮转

凭据轮转应该发生在网关侧，而不是要求 Agent 重新学习/保存真实 key：

- 1. credential provider 或 secret manager 生成/刷新真实 provider 凭据；
- 2. gateway 更新 backend secret 或 consumer key 映射；
- 3. Agent 侧只需要继续用自己的 consumer key，或接收一个短期 token；
- 4. 旧 consumer key 可按 Agent、Team、workspace 维度 revoke；
- 5. audit log 用于确认旧 key 不再被使用。

HiClaw 的 RefreshCredentials 思路属于这个模式：Worker 身份稳定，真实凭据和 STS 权限可在外部滚动。

网关单点风险

凭据隔离把风险从“每个 Agent 都可能泄露真 key”收敛为“网关是强信任边界”。需要配套：

- 网关高可用，避免所有 Agent 出口依赖单 Pod；
- RBAC / namespace / consumer policy 分层，避免一个 consumer key 访问全部能力；
- secret manager 或 KMS 托管真凭据，避免明文落盘；
- access log + anomaly detection，发现异常工具调用或模型调用；
- break-glass 机制，网关不可用时明确哪些能力允许降级，哪些必须 fail-closed。

与插件设计原则的关系

这对应 ai-agent-plugin-patterns 中的“宿主权限边界”和“协议通道纪律”：插件、skill、MCP server 可以扩展 Agent 能力，但不应把真实凭据直接暴露给 Agent prompt 或第三方插件代码。

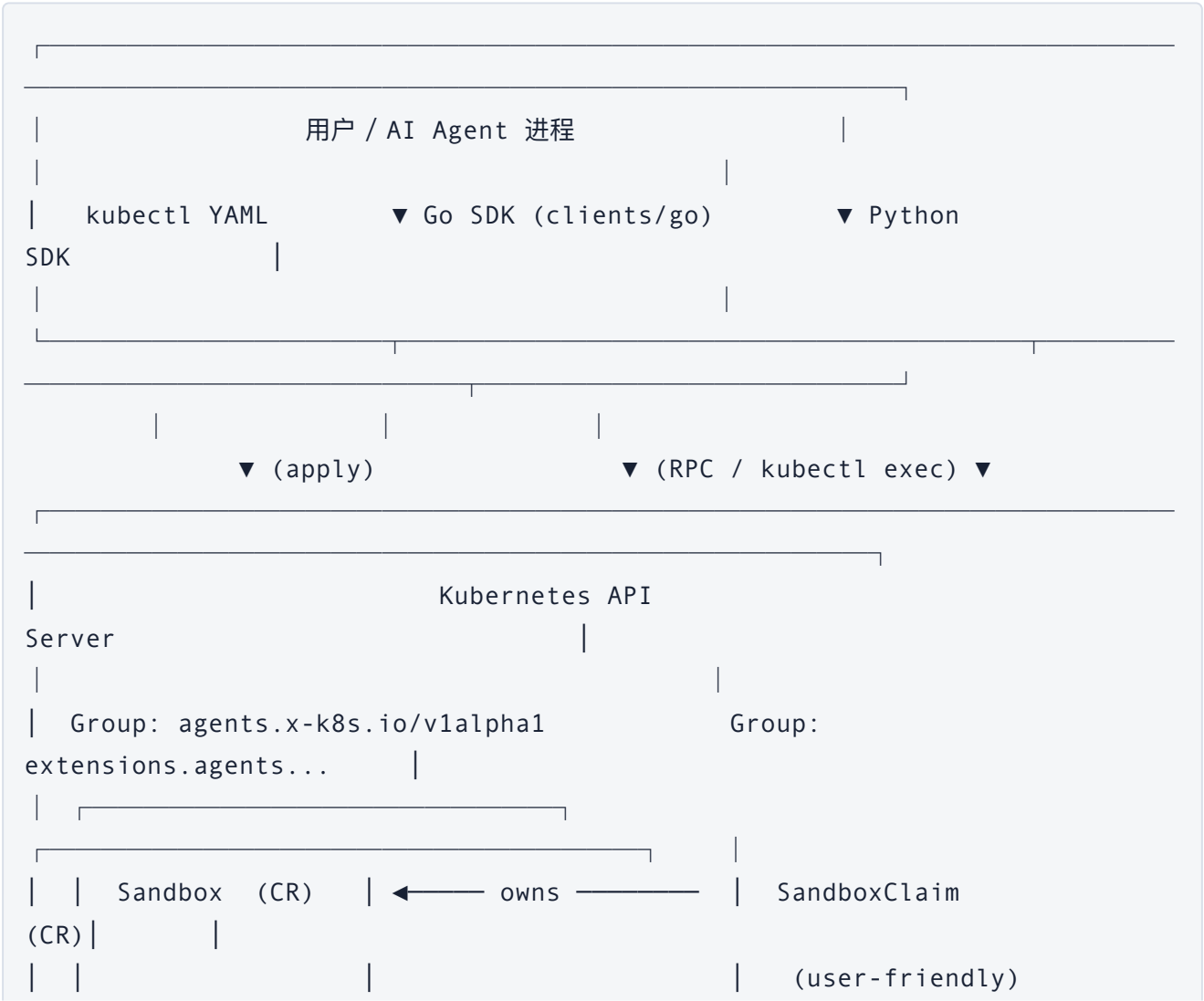
agent-sandbox 架构与设计思路分析

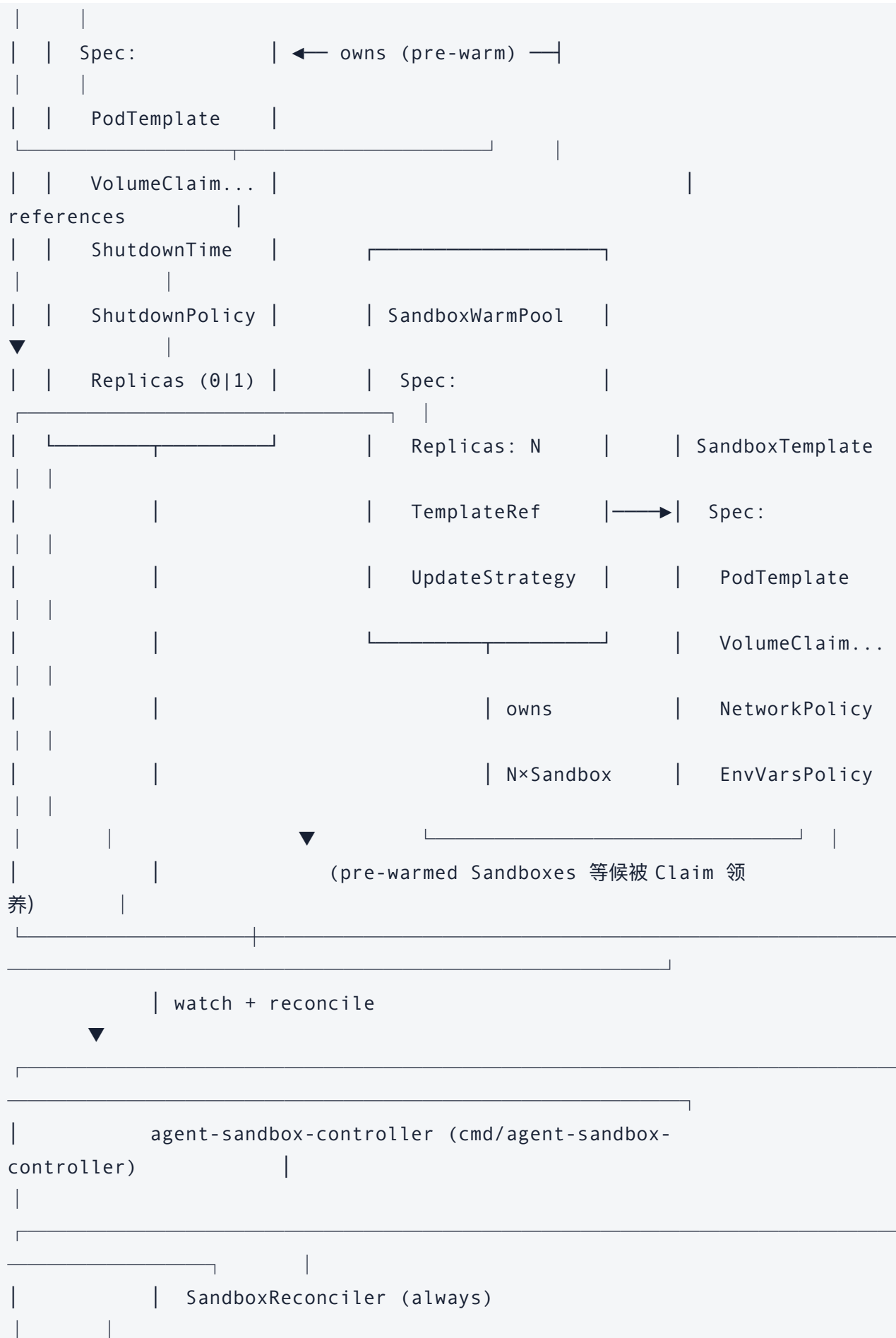
原文: raw/agent-sandbox-architecture-analysis.md · 仓库: [kubernetes-sigs/agent-sandbox](#) · 分析版本 v0.4.5+11 (HEAD e1d8898)

一句话定位

kubernetes-sigs/agent-sandbox 是 K8s SIG Apps 官方孵化的 Sandbox CRD 与 controller，把 AI Agent runtime 那种"长寿命、有状态、单实例、可暂停、有稳定身份"的容器形态建模成第一类 K8s 资源——比 Deployment（无状态副本）和 StatefulSet（编号 Pod）都更精准。核心手段：1 个 Sandbox CR = 1 个 Pod + 1 个 headless Service + 持久 PVC + 可选 Template/Claim/WarmPool 三件套，隔离机制（gVisor/Kata/NetworkPolicy）完全委托给标准 K8s 原语，controller 只做生命周期编排。

核心架构图







模块分层

层	职责
CRD 定义层	4 个 CRD 的 Go 类型 + kubebuilder 标注 → <code>controller-gen</code> 自动生成 YAML。Spec/Status 即 API 契约。
Reconciler 层	controller-runtime 风格的 4 个 reconciler。watch + owns + 幂等 reconcile。Claim/WarmPool/Template 通过 <code>--extensions=true</code> 开关启用。
运行时支持	绝对时间 + TTL 过期计算、Prometheus 指标、per-template-hash 的 FIFO 领养队列（O(1) WarmPool adopt）。
入口二进制	单二进制 <code>agent-sandbox-controller</code> ，flags 控制 extensions 开关、leader election、并发度、OTel tracing、pprof。
Client SDK	Go / Python / 标准 K8s clientset。Go SDK 提供远程 exec、文件传输、gateway 隧道、追踪注入——给 AI Agent runtime 调用 Sandbox 用。
打包	Helm chart（CRDs / RBAC / Deployment / metrics Service）+ 单独的 <code>k8s/crds/</code> 给手工 <code>kubectl apply</code> 。
示例与集成	17 个 <code>examples/</code> ：hello-world / kata-gke / openclaw / hermes / langchain / jupyter / vscode / chrome / HPA / Kueue / Cilium。"读完 examples 就会用"。

分层关键约束：api/ 与 extensions/api/ 是被 K8s API server 直接读的契约，只能改不能删；internal/ 严格私有；controllers/ 不允许 import extensions/（核心稳定、扩展演进的边界）。

关键数据流



```

| Reconcile(sandbox) - controllers/sandbox_controller.go |
|   ├── checkSandboxExpiry          (line 1003)         |
|   ├── reconcilePVCs              (line 853) → 创建 PVC |
|   ├── reconcilePod               (line 531)           |
|   |   ├── 如果有 SandboxPodNameAnnotation:           | |
|   |   |   ├── 走"warm pool 领养"分支, 复用现有 Pod   |
|   |   |   └── 否则 Create Pod {sandbox.Name}         |
|   ├── reconcileService           (line 398) → 创建   |
|   |   └── headless Service (ClusterIP=None)         |
|   ├── 算 Conditions (Ready/Finished)                 |
|   └── Update Status + requeueAfter(min(timeLeft, 2s)) |

```



```

| Pod 起来 → 容器内 Agent runtime (OpenClaw/Hermes/...) |
|
| AI Agent 进程 ← SDK (gateway/tunnel/exec) → 外部 caller
| (在 gVisor / Kata 内运行, 宿主机内核被隔离)

```

| t = ShutdownTime (绝对时间)



```

| handleSandboxExpiry (line 922) |
|   ├── ShutdownPolicy=Retain (default): 删 Pod+Svc, |
|   |   保留 Sandbox CR (status 显示 expired) |
|   └── ShutdownPolicy=Delete: 整个 Sandbox 也删掉 |

```

SandboxClaim 快/慢路径

```
user kubectl apply SandboxClaim
|
▼
SandboxClaimReconciler.getOrCreateSandbox
(sandboxclaim_controller.go:320)
|
├─▶ 【快路径】adoptSandboxFromCandidates (line 719)
|   │   query SimpleSandboxQueue[templateHash]
|   │   找到 1 个 WarmPool 已就绪的 Sandbox
|   │   │   └─ patch: 清掉 WarmPool 的 OwnerReference
|   │   └─ patch: 把 Claim 设为新 owner    ≈ 0 启动延迟
|
└─▶ 【慢路径】createSandbox (line 385)
    没找到 / WarmPoolPolicy=none / 自定义 env (跟 warmpool 不兼容)
    └─ 读 SandboxTemplate
    └─ 复制 PodTemplate + VolumeClaimTemplates
    └─ 注入 claim 的额外 env / labels
        └─ SetControllerReference(claim, sandbox)
        └─ Create Sandbox CR    ≈ Pod 冷启动延迟 (秒级)
```

设计决策与哲学

- 为什么不复用 Deployment / StatefulSet: AI Agent runtime 既不像 Deployment 那样无状态可副本 (Agent 有会话状态), 也不像 StatefulSet 那样需要编号有序 (每个 sandbox 是独立用户实例)。需要的是 "1 个 stable identity + 持久存储 + 可调度销毁 + 可暂停可恢复" 的单实例语义——Pet 概念在云原生的回归, 但这次明确为 AI Agent runtime 量身定做。replicas: 0|1 强制约束就是 "暂停 = scale to 0、恢复 = scale to 1" 语义的承载。
- 隔离机制全部委托标准 K8s 原语: controller 不强制任何隔离手段——gVisor / Kata / securityContext / serviceAccount / seccomp 都在用户填写的 PodTemplate.Spec 里, 原样透传。controller 唯一强势注入的是 (1) headless Service 强制走 DNS (多 sandbox 才能各自有稳定身份), (2) Template 级的默认 NetworkPolicy (deny RFC1918 内网 + deny 云元数据服务)。这种 "机制由 K8s 提供、策略由用户挑选" 的分工, 让它能兼容 GKE Kata、EKS Firecracker、本地 gVisor 任何隔离基座。这条与 AI Agent 外挂设计原则里 "隔离用现成机制、不重新发明" 完全吻合。

- **WarmPool + OwnerReference 转移实现 ~0 启动延迟**：单纯 Sandbox 冷启动慢（镜像拉取 + Kata/gVisor 初始化数秒到数十秒）。WarmPool 维护 N 个 pre-warmed Sandbox，Claim 来了不是新建 Pod，而是把 Ready 的 Pod 改归属者。SimpleSandboxQueue 是 per-template-hash 的线程安全 FIFO，O(1) 找候选。sandboxclaim_pod_exclusivity_test.go 这个测试名直白地说明 handoff 历史上踩过"1 个 Sandbox 对 2 个 Pod / 幽灵 Pod"的坑。
- **声明式 NetworkPolicy 而非隐式 firewalling**：Template 级别声明 Managed/Unmanaged。Managed 默认就是严格 deny——明确把"AI Agent 跑用户输入的代码必须默认禁掉内网访问 + 云元数据访问"做成系统约束，而不是文档里的最佳实践。
- **PVC 不被 Sandbox 标为 owned**：Sandbox 删除时 PVC 不级联回收，保留用户数据——"数据生命周期 ≠ 工作负载生命周期"做成了第一类设计。代价是孤儿 PVC 需要外部清理策略。
- **绝对时间 ShutdownTime，无心跳续约**：expiry 是声明式的，客户端要长寿命就主动 patch shutdownTime。controller 只做"现在 ≥ shutdownTime 就清理"的判断。避免 server 端维护"上次心跳时间"的状态，但代价是客户端要负责续期。
- **CRD validation 而非 webhook**：没有 admission webhook，全靠 OpenAPI v3 schema validation + controller 幂等 reconcile。少一个组件 = 少一个故障点——典型 SIG-style 的"简单优先"。

关键组件深入解读

SandboxReconciler 的三阶段子资源 reconcile

Reconcile() 主流程的工程亮点不在于复杂，而在于幂等三步永远按同一顺序：reconcilePVCs → reconcilePod → reconcileService。每一步内部都是 "Get → Create or Adopt or Validate ownership → Patch labels/annotations" 的三步式幂等。Service 一定是 headless (ClusterIP: None)，selector 用 agents.x-k8s.io/sandbox-name-hash=<hash> (hash 防止 sandbox 名过长触碰 K8s label 63 字符上限)。这导致每个 Sandbox 自动有 DNS A record: {sandbox}.{ns}.svc.{clusterDomain}——成为 Sandbox "稳定身份"承诺的实现基础。

requeueAfter = max(timeLeft, 2s) 是个被反复琢磨的小细节：既避免热循环打爆 apiserver，又保证即将过期的 Sandbox 准时被清理，不依赖 informer event。

SandboxClaimReconciler 的 adoption 协议

这是 extensions 子系统最有意思的代码，~60 行实现了 "WarmPool → Claim 的所有权热迁移"：

1. 从 SimpleSandboxQueue 拿到候选 Sandbox 名（已 Ready 的）；
2. 乐观锁 patch：用 OptimisticLockErrorMsg resourceVersion 保护，避免两个并发 Claim 抢同一个 Sandbox；
3. 清掉 WarmPool 标识标签；
4. OwnerReferences 置换：移除 WarmPool 的 controller ref，加 Claim 的 controller ref——后续删 WarmPool 不会再级联删这个 Sandbox；

5. Claim status 记录 AdoptedSandboxName，下次 reconcile 走 short-circuit。

关键不变量是 4 + 5 必须原子（同一个 patch 请求里）。这就是 sandboxclaim_pod_exclusivity_test.go 守护的不变量。

与同类对比

维度	agent-sandbox	HiClaw	普通 K8s Deployment
抽象层级	基础设施（Sandbox 原语）	应用层（Manager + Workers 协作）	基础设施（无状态副本）
单实例 + 稳定身份	✅（核心设计目标）	⚠️（Worker 之间通过 Matrix 房间）	❌
持久存储	✅	✅（MinIO + PVC）	❌
多 Agent 编排	❌（单 sandbox，多 sandbox 靠上层）	✅（Team/Manager CRD）	❌
WarmPool 预热	✅	❌	❌
隔离机制	委托 K8s（gVisor/Kata 任选）	Higress 网关 + 凭据隔离	取决于 runtimeClass
治理	K8s SIG Apps 官方孵化	阿里 Higress 团队开源	K8s core

定位结论：agent-sandbox 与 [HiClaw](#) 互补不竞争。前者提供"安全运行 1 个有状态 Agent 容器"的基础设施；后者提供"管 N 个 Agent 协作 + IM 平面 + 凭据网关"的应用层。理论上 HiClaw 的 Worker runtime 完全可以跑在 agent-sandbox 之上——一个用 Sandbox 做隔离底座，一个用 Worker CRD 做协作上层。这种分层正是云原生生态的健康样态。

相关页面

- [HiClaw](#) — 应用层多 Agent 协作平台（与本项目互补）
- k8s-operator — 本项目所遵循的 controller pattern
- [declarative-agent-management](#) — 用 K8s CRD 声明式管理 AI Agent 的方法论
- [agent-credential-isolation](#) — 凭据零暴露设计模式（HiClaw 用，本项目不直接处理）
- ai-agent-plugin-patterns — AI Agent 外挂的 9 条设计原则

AgentCube

AgentCube 是 Volcano 社区面向 AI Agent / Code Interpreter 的 Kubernetes 原生会话编排层。它不是替代 [agent-sandbox](#)，而是在 agent-sandbox 的 Sandbox / SandboxClaim / SandboxTemplate / SandboxWarmPool 之上，补上应用级会话、HTTP 入口、代码执行 API、SDK 入口和生命周期回收。

详细架构见 [src-agentcube-architecture](#)。

关键事实

- 仓库: [volcano-sh/agentcube](#)
- 当前核验: GitHub API 显示 main 最近 push 2026-06-12, 已有 tag v0.1.0
- 本 wiki 架构分析版本: HEAD 208da32 (2026-06-01)
- 阶段: 已从 Proposal 进入早期 release, 但 API 仍需按上游 agent-sandbox 演进持续核验
- 主要语言: Go 1.24.4, 模块 [github.com/volcano-sh/agentcube](#)
- 核心依赖: [sigs.k8s.io/agent-sandbox v0.1.1](#) (当前 main 的 go.mod 仍如此)
- 2 个 AgentCube CRD: AgentRuntime 与 CodeInterpreter
- 主要组件: Router、WorkloadManager、PicoD、Redis / ValKey session registry

架构定位

AgentCube 把 agent-sandbox 的底层运行时原语包装成更贴近 AI 应用的会话模型:

- AgentRuntime 面向长期运行的 Agent 服务, 声明 targetPort、PodTemplate 和 session timeout。
- CodeInterpreter 面向代码执行工作负载, 声明执行镜像、端口、session timeout、最大会话时长、warm pool 规模和鉴权模式。
- Router 是数据平面, 根据 x-agentcube-session-id 查询 Redis / ValKey, 把请求反向代理到已存在的 Sandbox, 必要时调用 WorkloadManager 创建新 session。
- WorkloadManager 是控制平面, 负责创建 Sandbox 或 SandboxClaim, 等待 Ready、做 endpoint TCP probe、写入 session 映射并执行 GC。
- PicoD 是轻量 HTTP daemon, 提供 /api/execute 与 /api/files, 用 Router 签发的 JWT 做访问控制, 替代传统 SSH 进入容器。

与 [agent-sandbox](#) 的结合

AgentCube 复用了 agent-sandbox 的隔离、Pod 生命周期和 WarmPool 能力, 但把它们隐藏在更高层 API 后面:

- 冷启动路径: AgentRuntime / CodeInterpreter -> WorkloadManager -> Sandbox -> Pod。
- 预热路径: CodeInterpreter.warmPoolSize -> SandboxTemplate + SandboxWarmPool -> 首次请求创建 SandboxClaim -> 领取预热 Sandbox。

- 会话路径: Router 从 session registry 解析 sessionId -> endpoint, 后续请求不再走 K8s reconcile, 而是直接代理到 PicoD 或 Agent 服务。

这个分层让 agent-sandbox 继续专注"安全、有状态、可回收的单实例容器", AgentCube 则负责"用户请求如何变成一个可复用的 AI / Code session"。

agent-sandbox API 兼容性

AgentCube 当前已发布 v0.1.0, 但仍依赖 sigs.k8s.io/agent-sandbox v0.1.1。这意味着它和当前 agent-sandbox v0.4.6 / v0.5.0rc1 之间可能存在 API 追赶成本:

- SandboxTemplate / SandboxWarmPool 字段可能需要适配新版本 rolling update / backend 解耦;
- claim / warm pool 的 status 和 owner reference 语义可能变化;
- router/session registry 依赖 Sandbox Ready、endpoint、port 等状态字段, 升级时要重点回归;
- AgentCube 的 CodeInterpreter.warmPoolSize 是对 agent-sandbox WarmPool 的上层封装, 上游 API 变动会直接影响这一功能。

选型上, AgentCube 当前适合作为"agent-sandbox 上层应用 API"的设计参考; 生产采用前应先验证它是否已跟上目标 agent-sandbox 版本。

CodeInterpreter warm pool 请求时序



```
wait Sandbox Ready + endpoint TCP probe
|
▼
write sessionId → endpoint into Redis / ValKey
|
▼
Router reverse proxy → PicoD `/api/execute` / `/api/files`
```

这条路径把高频请求和低频 K8s reconcile 分开：首次 miss 需要控制面介入，后续同一 session 直接由 Router 查 registry 并代理到 PicoD。

AgentCube 架构与设计思路分析

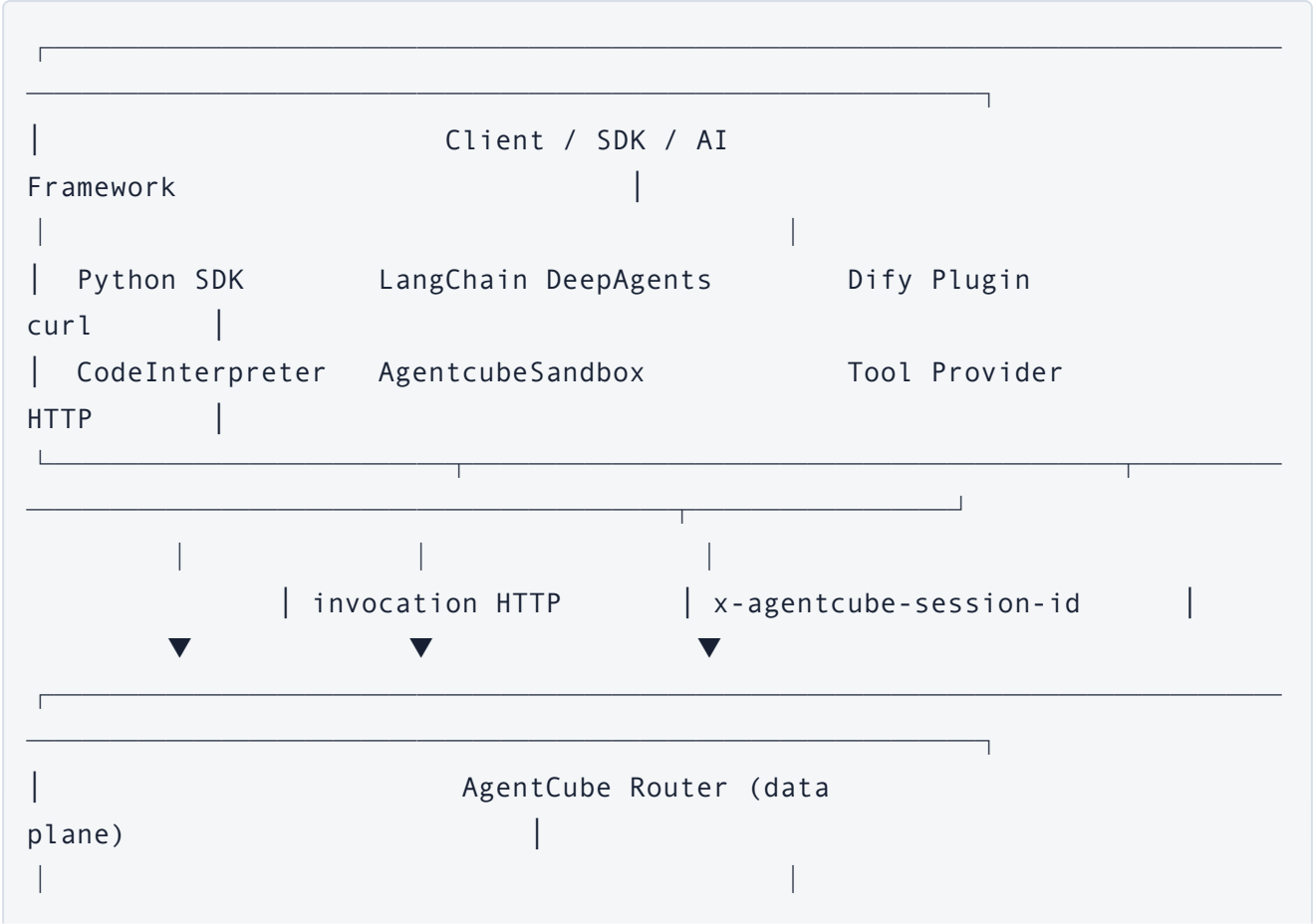
原文: raw/agentcube-architecture-analysis.md · 仓库: <https://github.com/volcano-sh/agentcube> · 分析版本 HEAD 208da32 (2026-06-01)

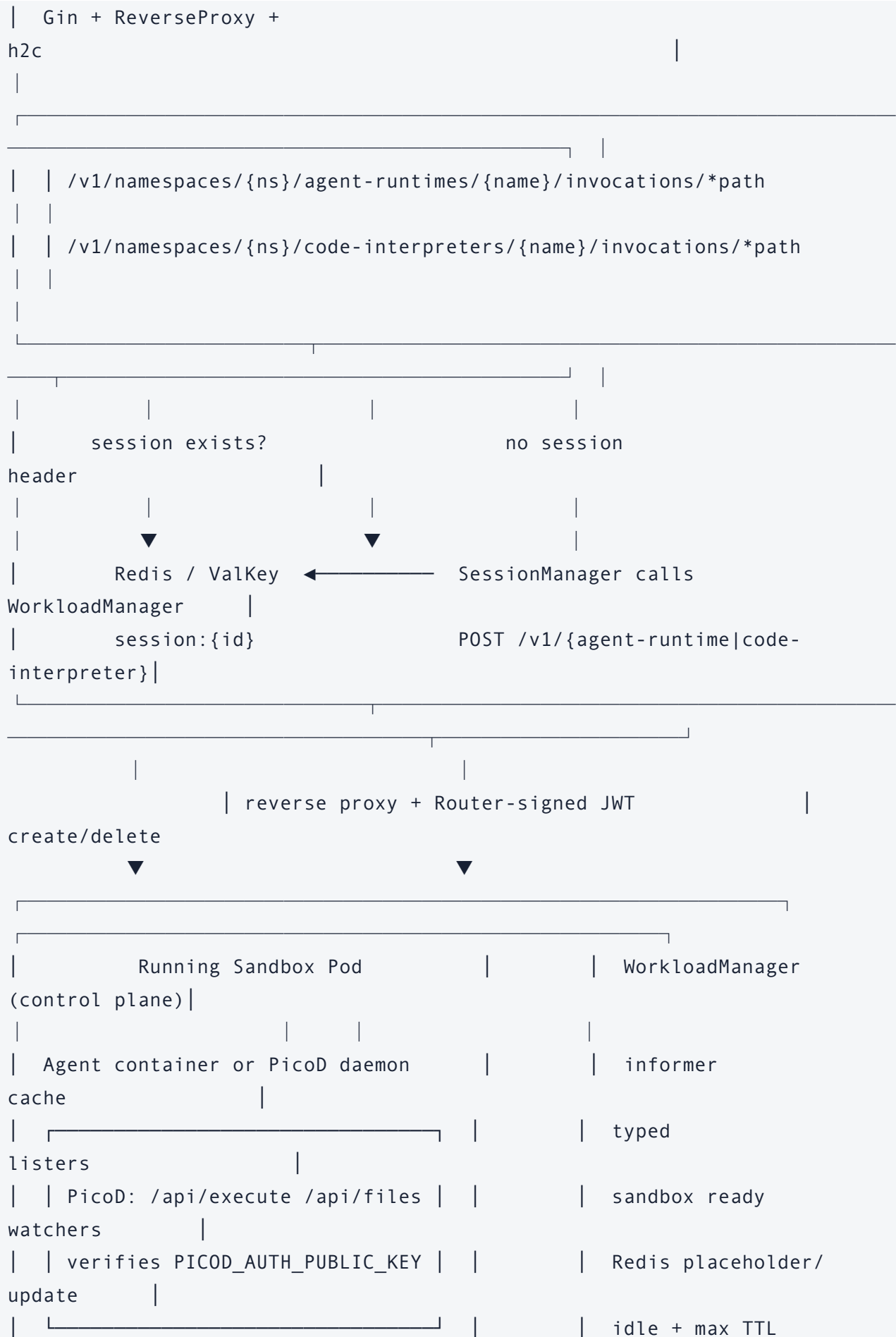
一句话定位

volcano-sh/agentcube 是 Volcano 社区面向 AI Agent / Code Interpreter 工作负载的 **serverless session orchestration layer**: 上层用 AgentRuntime / CodeInterpreter 两个 CRD 表达"可被调用的 agent/runtime 模板", 中层用 Router + WorkloadManager + Redis/ValKey 管 session、路由、GC 和预热, 下层直接复用 [agent-sandbox](#) 的 Sandbox / SandboxClaim / SandboxTemplate / SandboxWarmPool 来真正创建隔离执行环境。

它和 [agent-sandbox](#) 不是竞争关系, 而是更明确的上下层关系: [agent-sandbox](#) 提供"一个稳定、有状态、可预热/可领取的 Sandbox 原语", AgentCube 把这个原语包装成"开发者能通过 HTTP/SDK 调用的 AgentRuntime / CodeInterpreter 会话服务"。

核心架构图







模块分层

层 / 模块	职责
AgentCube API 层	定义 AgentRuntime / CodeInterpreter 两个面向用户的 CRD。前者保留完整 PodSpec，后者收敛成 image/resources/runtimeClass/authMode/warmPoolSize。
数据面 Router	暴露 invocation API，按 x-agentcube-session-id 做 session affinity；没 session 时调用 WorkloadManager 创建；有 session 时查 Redis/ValKey；最后反向代理到 sandbox endpoint。

层 / 模块	职责
控制面 WorkloadManager	读 AgentCube CRD，构造 agent-sandbox CR，等待 Sandbox Ready，探测 entrypoint，写 session store，并周期性 GC。
Sandbox 内 runtime	PicoD 用轻量 HTTP API 替换 SSH：执行命令、上传/下载/列文件、workspace path jail、JWT 验证、32MB body limit。
状态存储	Redis / ValKey 抽象；session:{id} 保存 SandboxInfo，两个 sorted set 分别索引 ExpiresAt 和 LastActivityAt。
安全与身份	Router→PicoD 用 RS256 JWT；Router→WorkloadManager 可用 SA token 或 SPIFFE/mTLS；mTLS 文件来源可来自 SPIRE / cert-manager / 静态 Secret。
SDK / 集成	Python SDK、LangChain Deep Agents backend、Dify plugin、kubectl 风格 CLI，把底层 session API 包成开发者接口。
部署与 E2E	Helm 安装 Router/WorkloadManager/CRDs/SPIRE；E2E 覆盖 CodeInterpreter、WarmPool、 agent-sandbox 安装和 session 生命周期。

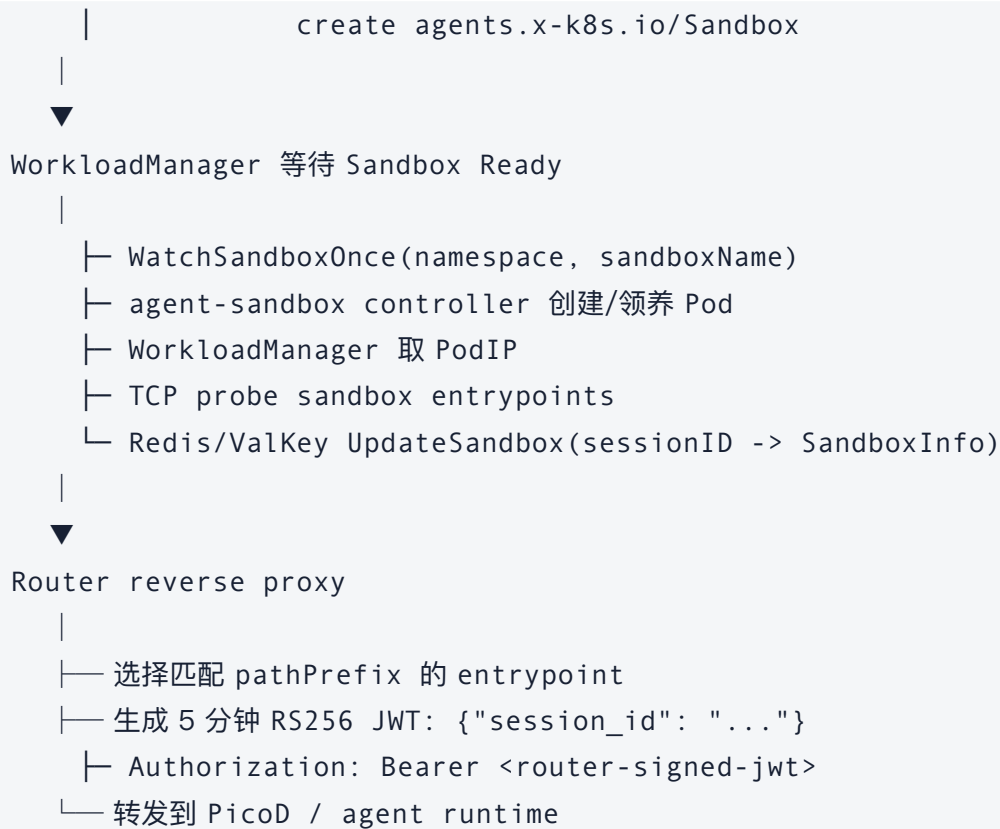
关键数据流

无 session 的首次调用

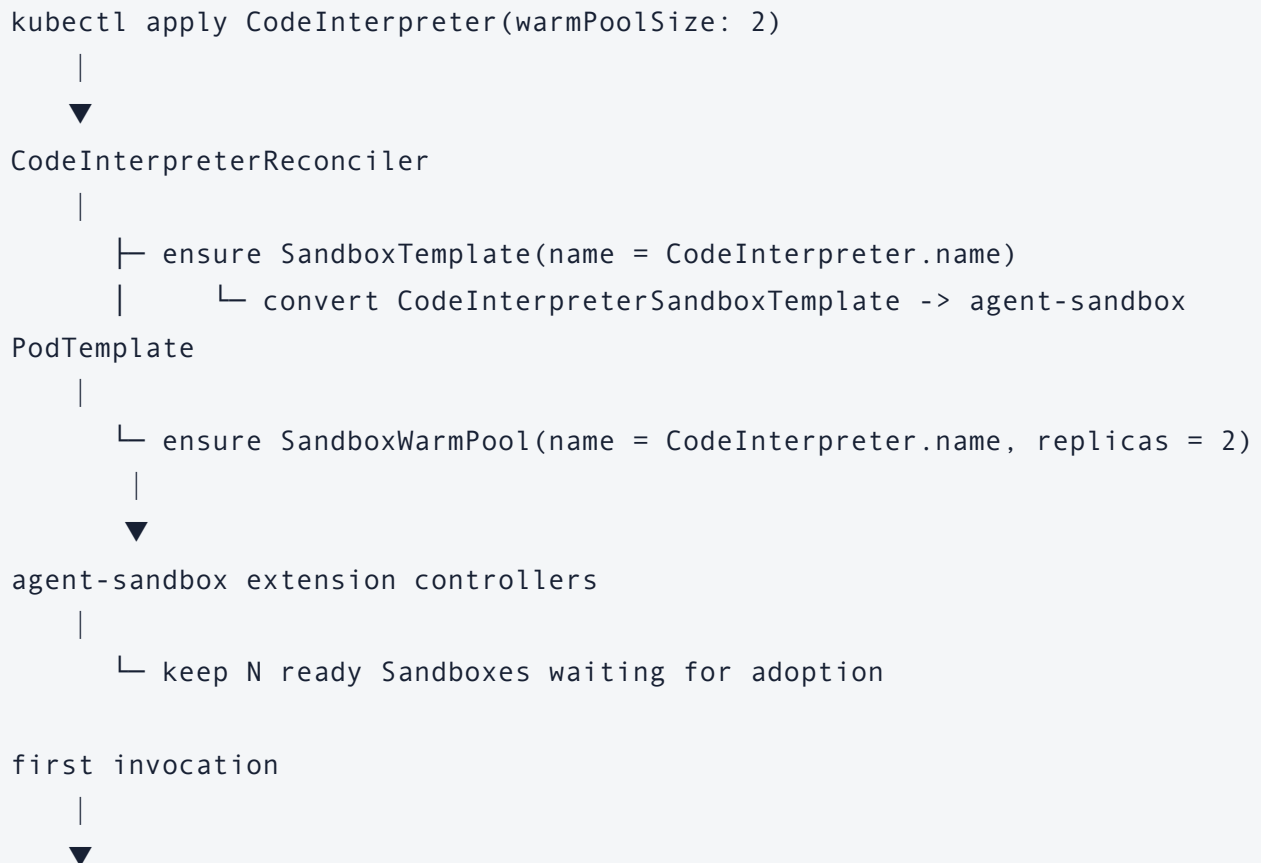
```

Client / SDK
|
|   | POST /v1/namespaces/default/code-interpreters/my-interpreter/
invocations/api/execute
|   | (no x-agentcube-session-id)
▼
AgentCube Router
|
|   └ SessionManager.GetSandboxBySession(sessionID="")
|   |
|   |   ▼
|   |   | WorkloadManager POST /v1/code-interpreter
|   |   |
|   |   |   └ informer lister 读取 CodeInterpreter CR
|   |   └ 生成 sessionID + sandboxName
|   └ 如果 warmPoolSize > 0:
|       |   create extensions.agents.x-k8s.io/SandboxClaim
|   └ 否则:

```



CodeInterpreter WarmPool 路径



```

WorkloadManager buildSandboxByCodeInterpreter
|
├─ create SandboxClaim(templateRef = CodeInterpreter.name)
├─ entry.Kind = SandboxClaim
└─ store session placeholder
|
▼
agent-sandbox SandboxClaimReconciler
|
├─ pick ready Sandbox from WarmPool
├─ transfer ownerReference to Claim
└─ annotate SandboxPodName
|
▼
WorkloadManager sees Sandbox Ready -> PodIP -> entypoints -> session store

```

设计决策与哲学

- 把 [agent-sandbox](#) 当 substrate，不重新实现 Sandbox controller: go.mod 直接依赖 sigs.k8s.io/agent-sandbox v0.1.1, WorkloadManager 构造 agents.x-k8s.io/Sandbox 和 extensions.agents.x-k8s.io/SandboxClaim。AgentCube 自己碰 Pod reconcile 细节，专注 session API、路由、GC、SDK 和集成。
- 两个 CRD 对应两类 AI workload: AgentRuntime 允许完整 PodSpec，适合长会话 agent、volume、credential、sidecar; CodeInterpreter 收敛成 image/resources/runtimeClass/authMode/warmPoolSize，适合执行不可信代码。也因此只有 CodeInterpreter 一等支持 warmPoolSize，因为 code interpreter 更需要低冷启动延迟。
- Router / WorkloadManager 分平面: Router 是高频数据面，负责 session header、反向代理、路径匹配和并发限制; WorkloadManager 是低频控制面，负责 kubernetes API、informer、Sandbox 创建和 GC。这样 Router 可以水平扩展，状态放进 Redis/ValKey，而不是把请求路径绑死在 controller 进程内。
- 创建先写 placeholder，再等 Ready，再 update store: WorkloadManager 在真正创建 K8s 资源前先 StoreSandbox placeholder，失败时 rollback K8s 资源和 store。成功路径必须等 Ready、拿到 PodIP、并 TCP probe endpoint 后才把 store 更新为可路由信息。
- PicoD 用 HTTP + JWT 替代 SSH: PicoD 是 sandbox 内最薄的执行面，POST /api/execute 执行命令，/api/files 做文件操作。当前代码已经从早期"客户端私钥签名"演进到"Router 持有 RSA 私钥、PicoD 校验 Router 公钥"的模型: Router 创建 picod-router-identity Secret, WorkloadManager 从 Secret 缓存 public key，再注入 PICOD_AUTH_PUBLIC_KEY。这降低 SDK 复杂度，但也让 Router 成为更强的信任边界。

- mTLS 是可选增强，不阻断低延迟路径：pkg/mtls 支持文件热加载证书和 SPIFFE ID 校验，Helm 可以通过 SPIRE sidecar 给 Router / WorkloadManager 注入 SVID。Router→WorkloadManager 可 mTLS，Router→PicoD 当前仍主要走 JWT，以避免每个短会话 sandbox 的 TLS 握手成本。

与 agent-sandbox 的结合方式

维度	agent-sandbox	AgentCube
抽象层	基础设施原语：Sandbox / SandboxClaim / SandboxTemplate / SandboxWarmPool	产品化会话层：AgentRuntime / CodeInterpreter + Router + SDK
用户入口	kubectl apply Sandbox / SandboxClaim 或 SDK 直接管 Sandbox	HTTP invocation API、Python SDK、LangChain/Dify/CLI
生命周期语义	1 个 Sandbox = 1 个有状态隔离 Pod，可暂停/恢复/过期	1 个 session = 1 个 Sandbox，附加 session header、activity、TTL、GC
Warm Pool	提供 WarmPool 和 Claim adoption 机制	CodeInterpreter.spec.warmPoolSize 驱动创建 Template/WarmPool，首次调用时创建 Claim
路由	headless Service / Pod 访问由上层决定	Router 根据 SandboxInfo.EntryPoints 反向代理到 PodIP:port
状态存储	K8s API / Status	Redis/ValKey session registry，跨 Router 副本共享
隔离策略	委托 K8s runtimeClassName / NetworkPolicy / securityContext	在 CRD 模板中暴露 runtimeClassName / resources，默认 image 可跑 PicoD
安全边界	运行时隔离与 K8s 原语	Router JWT、可选 WorkloadManager auth、可选 SPIFFE/mTLS

结合后的层次可以概括成：



▼
agent-sandbox CRDs own lifecycle mechanics

|

▼
Kubernetes Pod / RuntimeClass / NetworkPolicy / Service

最重要的设计收益是：AgentCube 没有把"AI Agent 平台"做成另一个私有调度器，而是把 AI Agent 的会话和工具调用语义加在 [agent-sandbox](#) 上。底层依然能继承 [agent-sandbox](#) 的 gVisor/Kata/NetworkPolicy/WarmPool 能力，上层则补齐了 HTTP/API/SDK/Redis session registry 这些 agent 应用真正需要的东西。

关键组件深入解读

WorkloadManager：从 AgentCube CRD 到 agent-sandbox CRD

核心转换在 pkg/workloadmanager/workload_builder.go。buildSandboxByAgentRuntime 读取 AgentRuntime，复制 spec.podTemplate.spec，归一化空 runtimeClassName，生成 sessionID 和随机后缀 sandboxName，然后构造 [agent-sandbox](#) 的 Sandbox。这条路径完全保留用户的 PodSpec，适合通用 agent runtime。

buildSandboxByCodeInterpreter 更专门：先检查 authMode=picod 时 public key 是否已缓存，然后为没有 ports 的 CodeInterpreter 默认补 / + 8080/HTTP。如果 warmPoolSize > 0，它不直接创建完整 Sandbox，而是创建 SandboxClaim(templateRef = codeInterpreterName)；如果没有 WarmPool，则构造带 PicoD image/env/resources 的直接 Sandbox。

风险与演进信号

- 项目仍在 alpha / proposal 到 release 的过渡期：README 还写 Early Design，release 文档写 v0.1.0。字段命名也有历史痕迹，比如 AgentRuntime.spec.targetPort vs CodeInterpreter.spec.ports，podTemplate vs template。
- Router 是强信任边界：当前 Router 持有 PicoD 私钥并能为任何 session 签执行请求。相比"客户端私钥不出本机"，这更容易用，但要靠 Router auth、mTLS、RBAC、审计来补强。
- WorkloadManager 目前承担两种职责：既是 HTTP control plane，又跑 controller-runtime manager 和 GC。规模变大后可能会拆成 API server / reconciler / GC 三个部署单元。
- agent-sandbox 版本偏早：依赖 v0.1.1，而本 wiki 之前分析的是 v0.4.5+11。AgentCube 当前锁定的 API 可能需要跟随 [agent-sandbox](#) 后续 v1alpha1 演进。

相关页面

- [agent-sandbox](#)
- [kubernetes](#)
- [declarative-agent-management](#)

- [agent-credential-isolation](#)

AgentScope

AgentScope 2.0 是面向生产化多 Agent 应用的 Python 框架。它把 Agent 推理循环、模型 provider、工具 / mcp / skill、workspace、权限、人类确认、长任务 offload 和 FastAPI 服务层拆成可组合的异步组件。详见 [src-agentscope-architecture](#)。

架构边界

AgentScope 的核心协议是 `Agent.reply_stream()` 事件流，而不是只返回最终文本。模型输出、thinking、tool call、tool result、确认请求和外部执行结果都会进入 `AgentEvent / Msg`，这让 SDK、SSE、存储和前端可以共享同一套执行语义。

关键设计

- Agent 只依赖模型、toolkit、workspace、permission 和 middleware 抽象。
- ReAct loop 把“等待人类确认/外部执行”作为一等状态，可以暂停后恢复。
- Workspace 是工具资源和上下文外溢边界，可接 Local / Docker / E2B。
- 慢工具可以 offload 到后台，完成后再把结果注入后续 reasoning。
- 服务层用 FastAPI / SessionManager / storage 把 agent app 多租户化。

选型判断

需要写多 Agent 应用和服务化 agent app 时看 AgentScope。需要只做记忆组件看 ReMe 或 mem0；需要 Kubernetes 运行时部署看 [agentscope-runtime](#)；需要强 sandbox 隔离看 [agent-sandbox](#) / [openshell](#)。

AgentScope Runtime

AgentScope 生态的生产化 Agent-as-a-Service runtime，把 AgentApp、Runner、protocol adapters、sandbox manager 和 K8s/serverless deployers 组合起来。详见 [src-agentscope-runtime-architecture](#)。

架构边界

它处在 framework app 与生产 runtime 之间：比 [agent-sandbox](#) 更上层，比完整 managed teammate 产品更底层。仓库 README 已提示能力并入 AgentScope 2.0，所以更适合作为架构迁移参考。

AgentApp 执行流程

```
AgentApp / Agent definition
    ↓
Runner: session + event loop
    ↓
Protocol Adapter: HTTP / SSE / RPC
    ↓
Workspace / Tool / MCP
    ↓
Sandbox Manager 或 K8s/Serverless Deployer
    ↓
AgentEvent / stream / persistence
```

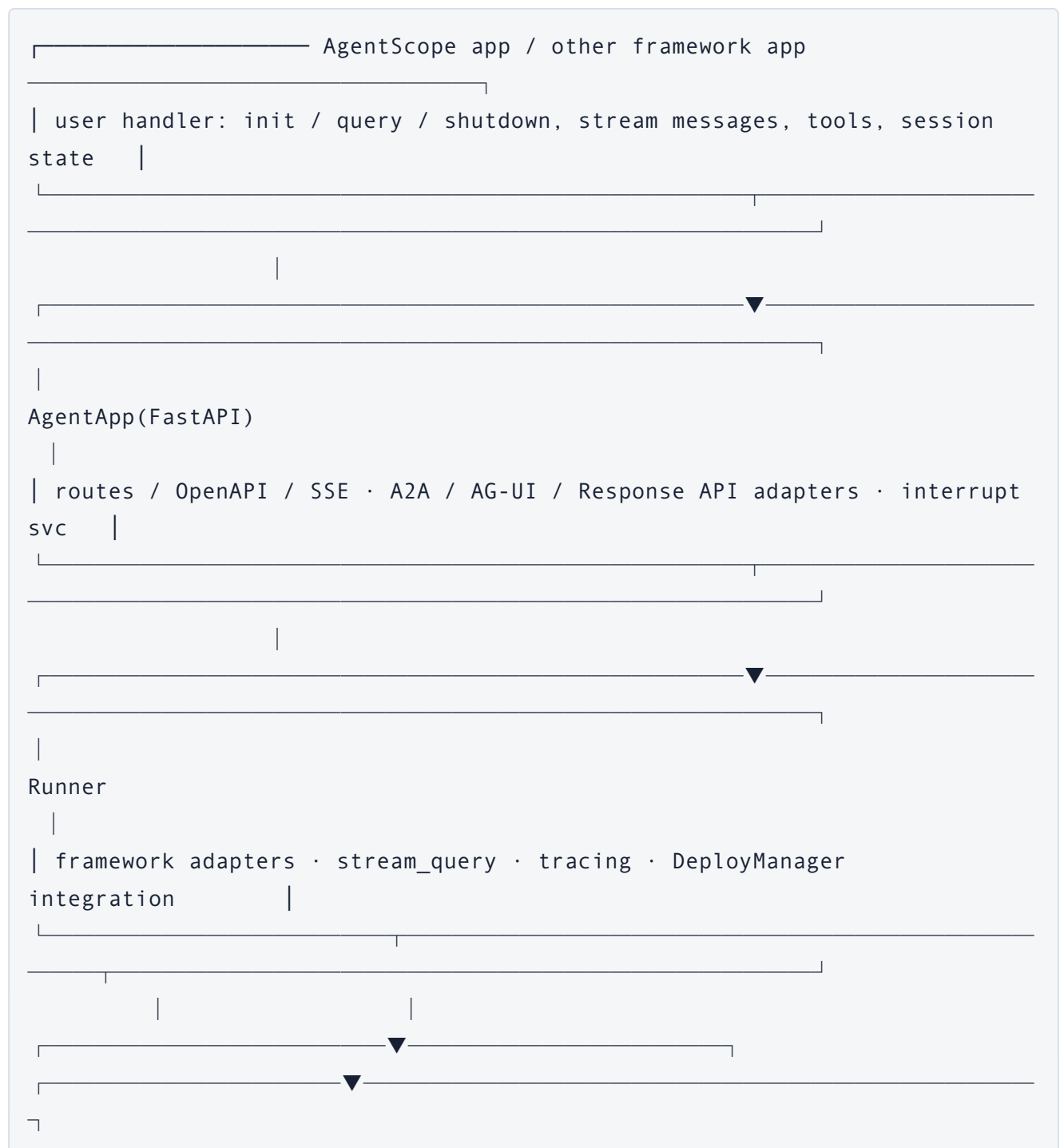
选型判断

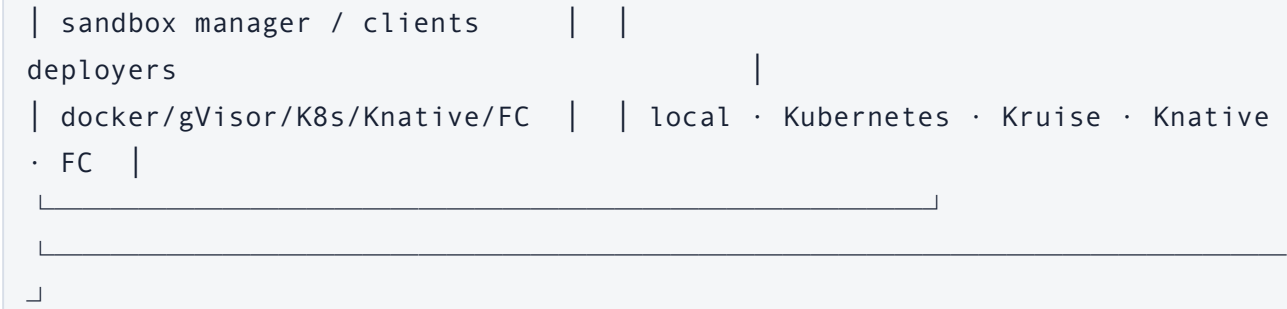
- 要理解 AgentScope app 如何服务化：看 AgentScope Runtime。
- 要理解底层容器/沙箱隔离：看 [agent-sandbox](#) 和 [agent-runtime-substrate](#)。
- 要理解多 Agent 协作产品层：看 [agentcube](#)、HiClaw、Multica 类项目。

AgentScope Runtime 架构与设计思路分析

agentscope-ai/agentscope-runtime 是 AgentScope 生态把 framework app 包装成生产 API/runtime 的过渡层: AgentApp(FastAPI) 暴露 streaming endpoint, Runner 统一 agent handler 生命周期, deployers 把本地、Kubernetes、Knative/FC 等环境抽象成可部署服务, sandbox manager 给工具调用提供隔离执行入口。README 明确提示这些能力已经并入 AgentScope 2.0, 仓库更像生产 runtime 设计参考。

核心架构图





模块分层

层/目录	责任
src/agentscope_runtime/engine/app/agent_app.py	AgentApp 直接继承 FastAPI，负责 endpoint、OpenAPI schema 注入、protocol adapters、分布式 interrupt service 和中间件。
src/agentscope_runtime/engine/runner.py	核心执行器，管理 query_handler/init_handler/shutdown_handler，把 handler 输出统一成 streaming event。
engine/deployers/**	把 runtime 投递到 local、Kubernetes、Knative、Kruise、FC、ModelStudio 等环境。
sandbox/** + common/container_clients/**	sandbox registry、manager server、workspace/storage mixin，以及 docker/gVisor/K8s/Knative/FC client。

关键数据流

- 请求进入 /process 或 A2A/Response API adapter，AgentApp 反序列化成 AgentRequest，交给 Runner.stream_query。
- Runner 检查 framework type 和 health，再调用用户注册的 query handler；同步、异步、generator、async generator 输出都被包成统一 stream。
- 工具调用通过 adapter 转成 runtime tool；需要隔离时走 sandbox client/manager，manager 再落到 Docker、gVisor、K8s、serverless 容器。

设计决策

- 把 Agent app 做成 FastAPI 子类，而不是另起 RPC 框架，换来 OpenAPI、中间件、生命周期和生态兼容。
- protocol adapter 是外壳，Runner 是内核；A2A/AG-UI/Response API 可以并存。
- sandbox 和 deployer 分离：前者处理工具调用隔离，后者处理整个 agent service 如何运行。

对比定位

和 [src-agentscope-architecture](#) 相比，它不是 agent 编程框架，而是服务化/部署/隔离壳；和 [agent-sandbox](#) 相比，它不把 workload 建模成 K8s CRD，而是在 Python runtime 内部暴露 Agent-as-a-Service；和 Agent Substrate 相比，它偏应用 runtime，不做高密度 actor/worker multiplexing。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

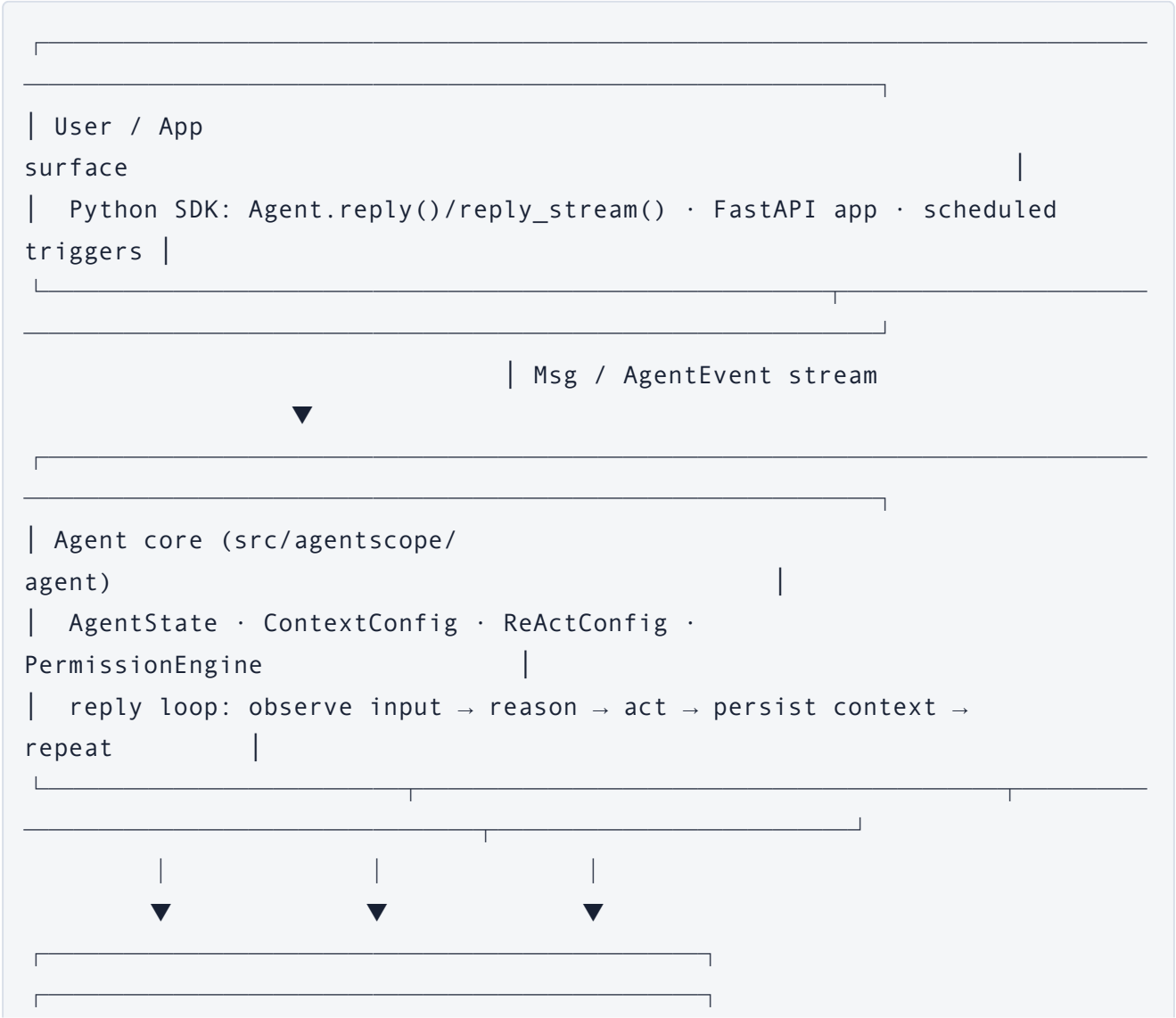
AgentScope 架构与设计思路分析

原文: raw/agentscope-architecture-analysis.md · 仓库: [agentscope-ai/agentscope](#) · 分析版本 main HEAD e129177 (2026-06-01)

一句话定位

AgentScope 2.0 是一个面向生产化多 Agent 应用的 Python 框架：它把 Agent 推理循环、模型 provider、工具 / mcp / skill、workspace、权限、人类确认、长任务 offload 和 FastAPI 服务层拆成可组合的异步组件。它的核心不是固定编排 DSL，而是以 `Agent.reply_stream()` 为中心的事件流，把 ReAct 过程中的模型输出、tool call、tool result、确认请求和外部执行结果统一建模成可持久化的 `AgentEvent / Msg`。

核心架构图





模块分层

层 / 模块	职责
公共 SDK / Agent 核心	管理 session state、上下文、ReAct 循环、事件流和最终消息聚合。
模型抽象与 provider	把统一 Msg / content blocks 转成各 provider API 请求，把流式 / 非流式响应转回 ChatResponse。

层 / 模块	职责
工具、MCP、skill	注册 Python tools、MCP tools、agent skills，生成 JSON schema，执行工具并归一成 ToolChunk / ToolResponse。
权限与确认	在工具执行前做 deny/ask/allow、模式检查、危险路径检查，并通过事件请求用户确认或外部执行。
Workspace / offload	为每个 session 提供本地 / Docker / E2B 执行环境，暴露 tools/MCP/skills，并持久化压缩上下文和超长工具结果。
FastAPI app/service	多租户、多 session 服务层；负责 agent 组装、SSE 事件输出、存储、调度、后台任务。
长任务与计划任务	把慢工具 offload 到后台，结果完成后重新注入上下文；用 APScheduler 驱动定时 agent run。

这个分层的关键约束是：Agent 只依赖抽象接口，不直接关心 provider SDK、Docker/E2B 细节、Redis 存储或 FastAPI 请求对象。服务层通过 `get_agent()` 每次从 `storage/session/workspace` 重新组装一个 Agent，Agent 内部再用统一事件流表达执行过程。

关键数据流





next reasoning iteration or final Msg

服务化路径是在这个本地流外面包一层持久化和 SSE:

```
HTTP POST /chat
|
▼
FastAPI router
|   encode AgentEvent as "data: <json>"
▼
ChatService.stream_chat
|
|   └─ build ToolOffloadMiddleware
|   └─ get_agent(storage, workspace_manager, user, agent, session)
|       └─ load AgentRecord / SessionRecord / AgentState
|       └─ instantiate model and optional fallback model
|       └─ get per-session workspace
|       └─ build Toolkit(tools + skills + MCPs)
|
|   └─ SessionManager.run(session_id) serializes the run
|   └─ persist input Msg
|   └─ stream Agent.reply_stream() events
|   └─ reconstruct AssistantMsg by append_event()
|   └─ persist reply Msg + updated AgentState
```

设计决策与哲学

- 事件流是核心协议，而不是“最终文本”：模型调用、文本 / thinking / data / tool-call delta、tool result、reply end 都是事件；这让 SDK、SSE、存储和前端 UI 可以共享同一种流式语义，也适合做 agent-memory 或观测流水线。
- ReAct 循环把“等待人类/外部系统”作为一等状态：确认和外部执行通过 `RequireUserConfirmEvent` / `RequireExternalExecutionEvent` 暂停，再用 `continuation event` 恢复，比阻塞式输入更适合 Web 服务。
- 工具执行被拆成“验证/权限/上下文写入”和“原始 I/O”两层：permission 语义在 Agent 内部，工具 I/O 通过 `Toolkit.call_tool()` 和 middleware 拦截；这和 [agent-credential-isolation](#)、ai-agent-plugin-patterns 里强调的边界治理一致。

- **Workspace** 是工具资源和上下文外溢的边界：Local / Docker / E2B workspace 给同一个 Agent 暴露 tools、mcp、skills 与 offload，和 [agent-sandbox](#) 的“执行环境原语”在问题域上相邻，但 AgentScope 更偏应用框架。
- 慢工具不取消，而是 offload 后再注入：服务层把超时工具转为后台任务，先给模型 synthetic result，任务完成后再把结果作为 hint 注入下一次 reasoning，避免单个慢 I/O 卡死 SSE 回复。

核心组件

Agent

Agent 构造时接收模型、toolkit、middlewares、state、context/react config 和 offloader。核心 `_reply_impl()` 先区分新输入和 continuation event，再进入 `cur_iter < max_iters` 的 ReAct loop：需要 reasoning 时准备 system prompt、summary、context、tool schemas 并调用模型；模型产出 tool calls 后按 sequential/concurrent batch 执行；如果触发用户确认或外部执行，就返回等待消息，让后续事件恢复同一个 reply。

App Service

FastAPI chat router 只负责把 `ChatService.stream_chat()` 的 `AgentEvent` 编码成 SSE。
`ChatService` 才是服务端运行入口：它构造 `ToolOffloadMiddleware`，调用 `get_agent()` 从 storage、session、credential、workspace 组装 Agent，在 `SessionManager.run(session_id)` 中串行运行同一 session，并把输入消息、reply 消息、更新后的 `AgentState` 写回 storage。

相关页面

- mcp
- agent-memory
- ai-agent-plugin-patterns
- [agent-credential-isolation](#)
- claude-code
- [agent-sandbox](#)
- nanobot

AI Gateway

AI Gateway 是面向 LLM/MCP/Agent 流量的入口治理层，通常处理 provider 适配、认证、凭据托管、rate limit、routing、redaction、guardrails、telemetry 和 policy。

当前项目族

项目	定位
agentgateway	LLM/MCP/A2A 三协议 Rust 数据面 + Gateway API 控制面
envoy-ai-gateway	Envoy Gateway 上的 GenAI provider translator / extproc / policy
kgateway	Gateway API / Envoy xDS / API+AI Gateway 能力分支
Higress	AI Native API Gateway，和 HiClaw/凭据托管强相关
Plano	AI-native proxy/data plane，偏 model routing / guardrails / agent orchestration
gateway-api-inference-extension	Gateway API 的 inference endpoint picking 协议与 InferencePool API
semantic-router	按 intent、difficulty、risk、capability 和 system state 选择 model/recipe 的控制层
routellm	用 strong/weak model threshold 做质量-成本路由并提供离线评测
llm-d-router	面向 InferencePool endpoint 的 EPP，按 KV、load、priority、health 做 pod/worker 选择

和 inference routing 的关系

[inference-routing](#) 是 AI Gateway 的一个子问题：请求应该去哪个模型、哪个 endpoint、哪个 pod。AI Gateway 还要处理凭据、安全、协议转换和治理。

不要把“网关”理解成一个单一组件：Envoy AI Gateway 偏 edge/provider governance，Semantic Router/RouteLLM 偏 model or capability selection，Gateway API Inference Extension 偏标准协议，llm-d Router 偏 K8s inference endpoint picking。完整对比见 [src-k8s-gateway-routing-comparison](#)。

Envoy AI Gateway

Envoy AI Gateway 是基于 Envoy Gateway 的开源 GenAI traffic gateway，用于把应用请求接入多个 hosted provider 或 self-hosted model serving cluster。

解决的问题

企业需要统一处理 provider 认证、顶层路由、global rate limit、协议和失败边界，同时又希望把自托管集群内部的细粒度 endpoint picking 交给专用 inference router。

核心架构

README 采用 two-tier gateway: Tier One 负责 authentication、top-level routing 和 global rate limiting; Tier Two 负责 self-hosted model access，并可接 endpoint picker / LLM-aware routing。它支持 OpenAI、Azure OpenAI、Gemini、Bedrock、Anthropic 等 provider 方向。

边界与选型

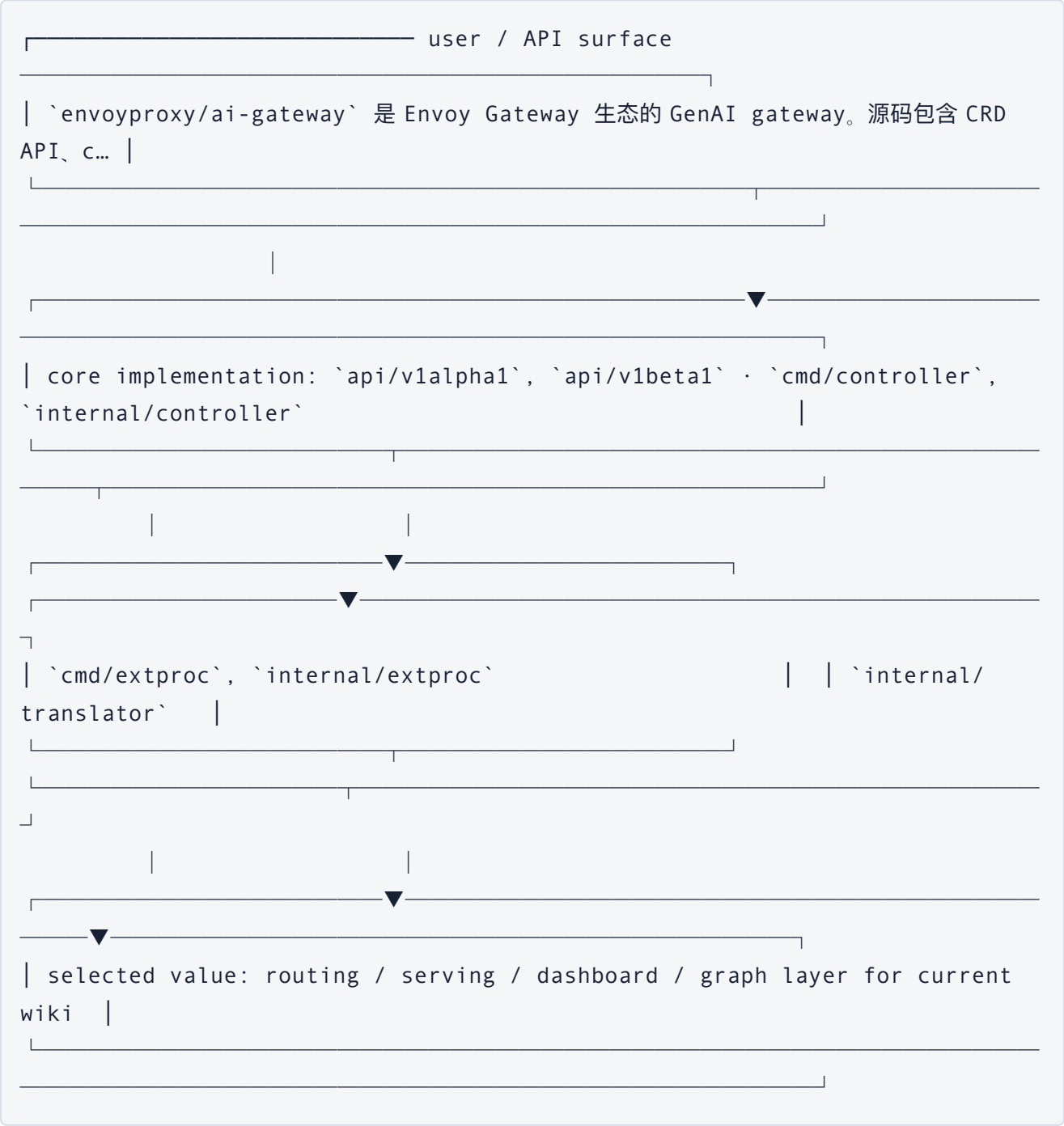
它是 edge/provider governance，不是 P/D worker router，也不是模型质量路由算法。典型组合是 Envoy AI Gateway → [semantic-router](#)（可选）→ llm-d-router / gateway-api-inference-extension → model server。

详见 [src-ai-gateway-architecture](#)、[src-k8s-gateway-routing-comparison](#)。

Envoy AI Gateway 架构与设计思路分析

envoyproxy/ai-gateway 是 Envoy Gateway 生态的 GenAI gateway。源码包含 CRD API、controller、extproc、provider translators、backend auth、body/header mutator、rate limit、redaction、MCP proxy、metrics/tracing 和大量 data-plane/e2e tests。它解决的是 LLM/API 访问治理，不是模型 serving engine。

核心架构图



模块分层

层/目录	责任
api/v1alpha1, api/v1beta1	AI Gateway CRD。
cmd/controller, internal/controller	控制面。
cmd/extproc, internal/extproc	Envoy External Processing 数据面。
internal/translator	OpenAI/Bedrock 等 provider 翻译。

关键数据流

1. Gateway/HTTPRoute 流量进入 Envoy。
2. extproc 根据 AI Gateway 配置改写/校验/路由请求。
3. translator 处理不同 provider 协议差异，auth/ratelimit/redaction 处理治理。

设计决策

- 建立在 Envoy Gateway 上，复用成熟 L7 控制面。
- 把 provider protocol translation 当核心能力，最近 Bedrock translator 修复就是例子。
- MCP proxy 表明它不只管 LLM，也在扩展到 tool gateway。

对比定位

和 agentgateway 相比，Envoy AI Gateway 更 Envoy Gateway/CRD-first；和 Higress/kgateway 相比，它更专注 GenAI，而不是通用 API gateway。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

HiClaw

阿里 Higress 团队 (agentscope-ai) 出品的开源 K8s 原生多 Agent 协作平台。把"管 AI Agent 大军"问题完全套进 K8s 范式: 4 个 CRD (Worker / Team / Human / Manager) 声明 Agent, controller 把每个 Agent 落成"容器 + Matrix IM 用户 + Higress consumer"三位一体, 人类与 Agent 在同一组 Matrix 房间里协作, 真实 LLM 凭据托管在 Higress 网关, Worker 只持 consumer key 实现 [凭据零暴露](#)。

详细架构与设计哲学见 [src-hiclaw-architecture](#)。

关键事实

- 仓库: [agentscope-ai/HiClaw](#)
- 当前核验: GitHub API 显示 main 最近 push 2026-06-10, 最新 tag v1.1.2
- 本 wiki 架构分析版本: v1.1.0 (2026-04-24, HEAD e21ac83)
- 主要语言: Go (controller, 153 文件) + Python (worker runtimes: QwenPaw / Hermes)
- 同栈: [Higress AI Gateway](#) · Tuwunel (Matrix 服务端) · MinIO · Element Web · k3s-io/kine (嵌入式 etcd)
- Worker runtime 三选一: OpenClaw (claude-code 系) / QwenPaw (基于 Qwen Code) / Hermes (autonomous coding agent) ; 当前默认 QwenPaw

从 0 到一个 Worker 跑起来

HiClaw 的最小闭环不是“启动一个 Python Agent”, 而是创建一套协作身份和凭据边界:

1. 安装 HiClaw: 本地 Docker 模式走 installer, K8s 模式走 Helm chart。
2. 准备 LLM 配置: `credentials.llmProvider`、`credentials.defaultModel`、`credentials.llmApiKey`、`credentials.llmBaseUrl`。
3. 启动控制面: controller 创建/管理 CRD, 并连接 Matrix、gateway、storage。
4. 创建 Human / 管理员身份, 进入 Element Web 或其他 Matrix client。
5. Manager Agent 通过 Matrix 收到“创建 Worker”的请求。
6. Manager 调 `hiclaw worker create ...`, controller 写入 Worker CR。
7. Worker reconciler 创建 Matrix user/room、gateway consumer key、对象存储权限和 Worker 容器。
8. Worker 容器启动后进入 Matrix 房间, 后续人类、Manager、Worker 都在 IM 协作平面里交流。

这条路径体现 [declarative-agent-management](#): 用户声明 Worker, controller 负责把容器、IM 身份、网关凭据和存储权限收敛到期望状态。

Helm 组件配置

当前 Helm chart 的主要组件来自 `helm/hiclaw/values.yaml` 与 `templates`:

组件	values 路径	作用
controller	controller.*	CRD reconciler、worker lifecycle、gateway/credential/storage 编排；可配 replica、resources、workerBackend、uninstall hook
manager	manager.*	是否创建 Manager CR、Manager runtime/model/image/resources
matrix	matrix.*	Tuwunel 或 Synapse；managed/existing；控制 Matrix server、持久化和服务端口
gateway	gateway.* / higress.*	managed Higress 或 existing AI Gateway；配置公网 URL、APIG 信息、Higress subchart
storage	storage.*	managed MinIO 或 existing OSS；bucket、endpoint、MinIO 资源和持久化
credentialProvider	credentialProvider.*	云上 APIG/OSS 场景下签发 STS / RAM role token
secrets	credentials.*	Matrix admin、LLM provider、默认模型和 base URL

本地 kind/minikube 默认更偏 managed Tuwunel + managed Higress + managed MinIO；云上 ACK/ACS 场景可以把 gateway/storage 切到 existing provider。

与 claude-mem 的区别

维度	HiClaw	claude-mem
目标	管理多个 Agent 的协作、身份、容器、房间、凭据	给 Claude Code 增加长期记忆
状态单位	Worker/Team/Human/Manager CR	用户 prompt、tool log、memory fact
控制面	Kubernetes controller + Matrix + gateway	Claude Code hooks + memory server/storage
安全重点	Agent 只持 consumer key，凭据在 gateway	记忆采集、压缩、检索和上下文注入
适合问题	多 Agent 运维、人在回路、协作拓扑	单 Agent 跨会话记忆

二者可组合：HiClaw 负责“Agent 如何被创建、协作和持权”，记忆系统负责“Agent 如何记住长期事实”。

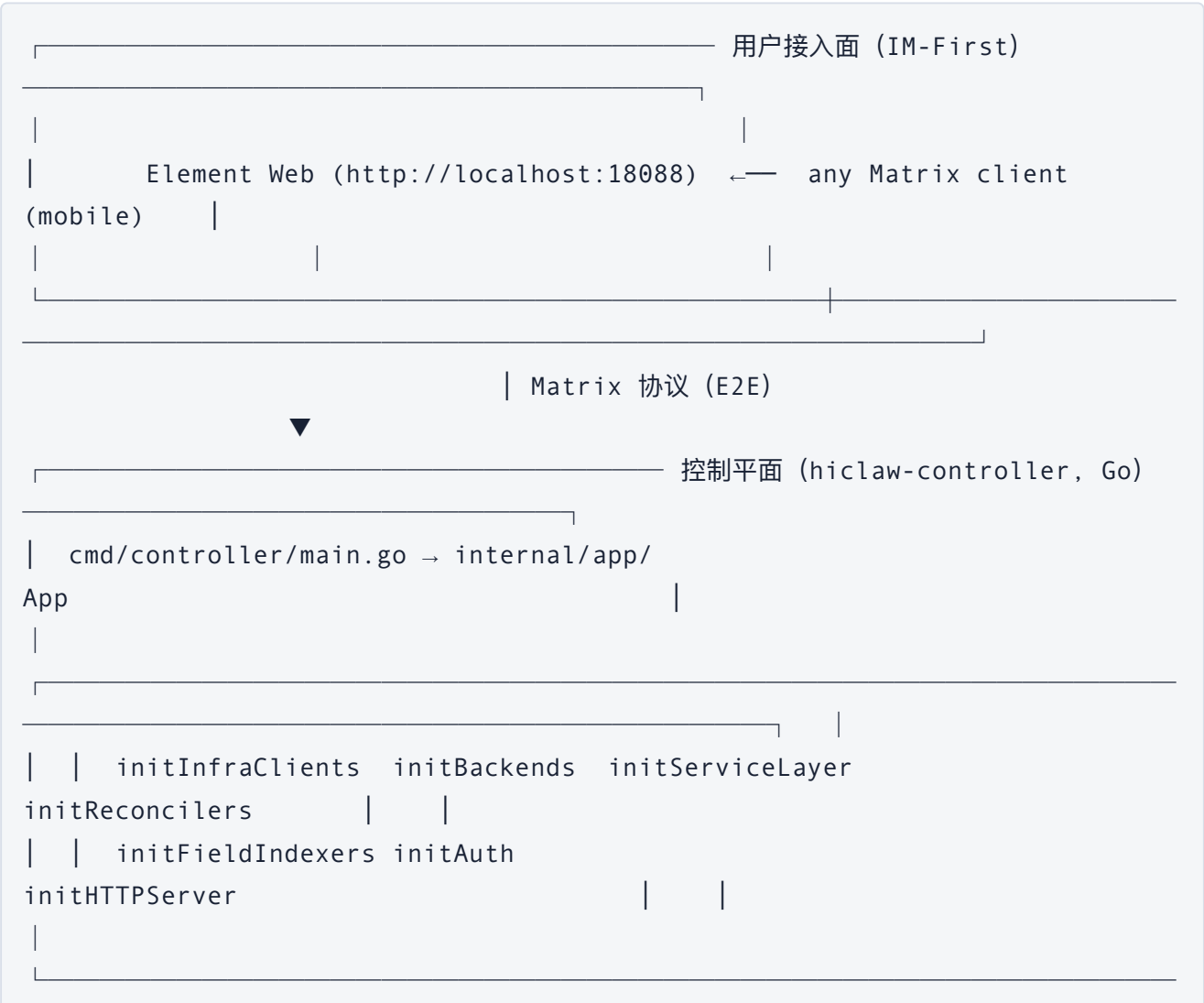
HiClaw 架构与设计思路分析

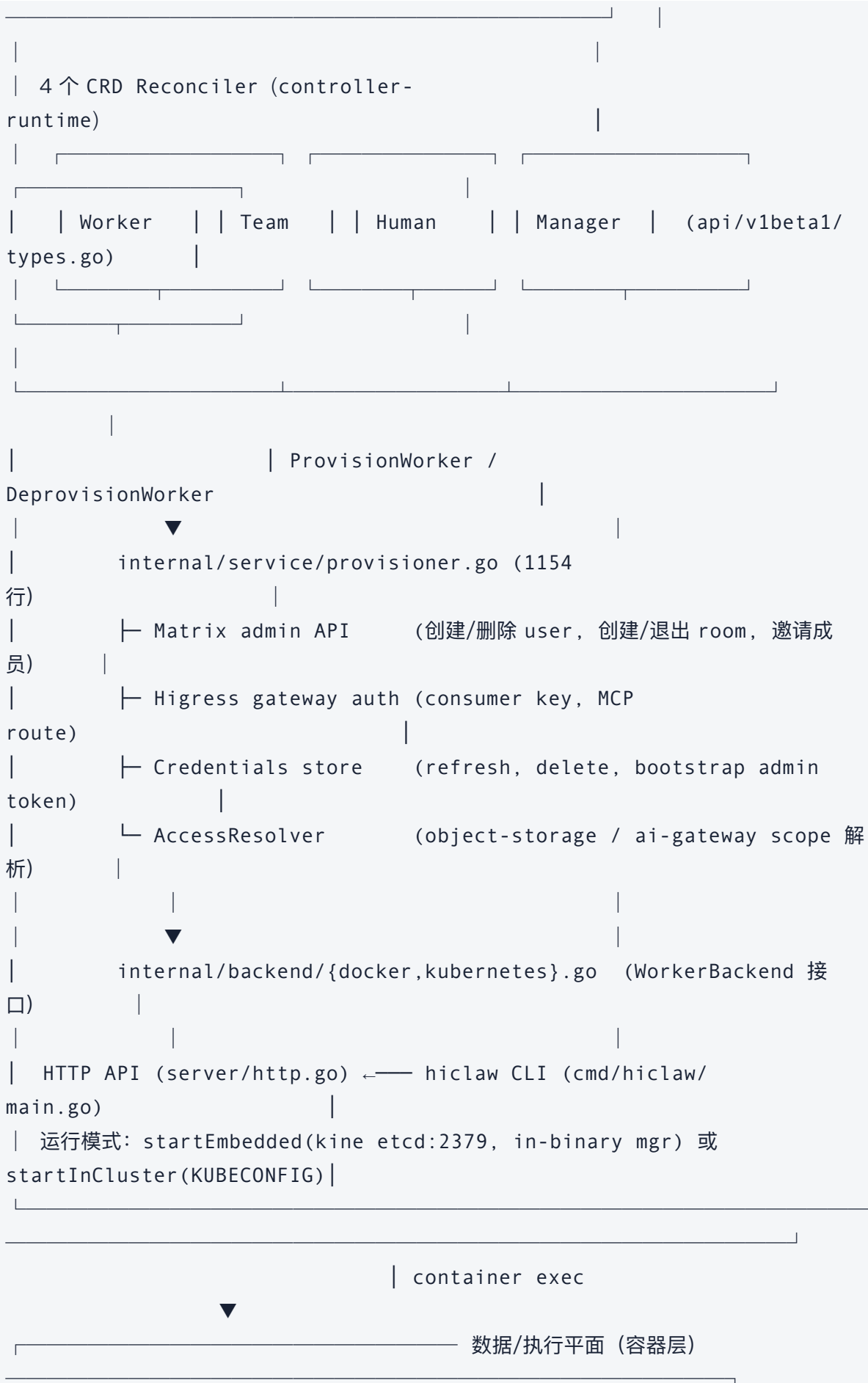
原文: raw/hiclaw-architecture-analysis.md · 仓库: [agentscope-ai/HiClaw](#) · 分析版本 v1.1.0 (HEAD e21ac83)

一句话定位

[HiClaw](#) 是把 k8s-operator 模式套到 AI Agent 运维上的多 Agent 协作平台——用 4 个 CRD (Worker / Team / Human / Manager) 声明 Agent, controller 把每个 Agent 落成"容器 + Matrix IM 用户 + [Higress](#) consumer"三位一体; 人类、Manager、Workers 全在同一组 Matrix 房间里协作; 真实 LLM API key 永远不落到 Worker, 只发 consumer key ([凭据零暴露](#))。一行 curl | bash 装到 Docker, 或 helm install 到 K8s, 二者形态对齐。

核心架构图







Helm chart (helm/hiclaw/) 在 K8s 上把上面 4 个组件一并部署
install/hiclaw-install.sh / .ps1 在 Docker 单机上等价部署

四个 CRD

CRD	表达什么	关键 Spec 字段
Worker	单个 AI Agent	model · runtime (openclaw/copaw/hermes) · image · soul · skills · mcpServers · package (file:///http(s):///nacos://) · expose · accessEntries · state (Running/Sleeping/Stopped)
Team	一组 Worker + Leader + 房间策略	leader · workers · peerMentions · channelPolicy · admin
Manager	系统唯一的管家 Agent	model / runtime / soul / skills (含 12 个管理 skill)
Human	管理员的 Matrix 身份	matrixUserID · channel scope

模块分层（简版）

层	职责
CRD 定义	api/v1beta1/types.go, 仅依赖 k8s.io/apimachinery, 不引用任何 internal 包
应用骨架	internal/app/App 用 7-step init 串行装配, 扩展只需 append {name, initFn}
CR Reconciler	Worker / Team / Human / Manager 4 套, 仅做编排, 绝不直接调外部 API
服务编排	internal/service/provisioner.go (1154 行, 脏活集中地, reconcile 的大半在这里)
容器后端	WorkerBackend 接口 + docker.go / kubernetes.go 两个实现, 按 config 选
基础设施集成	internal/{matrix,gateway,oss,credprovider,accessresolver}/ 分别封装 Tuwunel / Higress / MinIO / 阿里云凭据 / 模板解析
运行模式	startEmbedded (k3s-io/kine SQLite-backed etcd 在 127.0.0.1:2379 + 内嵌 mgr) 或 startInCluster (标准 KUBECONFIG)

层	职责
Manager Agent	manager/agent/ 容器化 orchestrator, 12 个管理 skill + workers-registry.json, 通过 hiclaw CLI 操作 controller
Worker runtime	OpenClaw (claude-code 系, Dockerfile only) · QwenPaw (Python, copaw/) · Hermes (Python, autonomous coding)
部署	helm/hiclaw/ (K8s) / install/hiclaw-install.{sh,ps1} (Docker 单机), 二者部署形态对齐

关键数据流：建一个 Worker 端到端

管理员 (Element Web 中)

| 发送 Matrix 消息: "帮我建一个 Go 代码评审 Worker"



Manager Agent 容器 (manager/agent)

| 加载 worker-management skill (TOOLS.md 路由)

| 决定参数 (model / runtime / soul / skills / mcp-servers)

| 调 hiclaw CLI: `hiclaw worker create --name go-review ...`



hiclaw CLI (cmd/hiclaw/main.go) — HTTP POST —▶ controller server/http.go



写入 Worker CR

(embedded: kine sqlite / in-

cluster: etcd)

┌──────────┐ controller-runtime watch 触发



worker_controller.Reconcile()



└─[1] Provisioner.ProvisionWorker(ctx, req):

| Matrix admin API → 创建 user @go-review:hiclaw.local

| Matrix admin API → 创建 1on1 房间, 邀请管理员 + Manager

| Higress apig API → 注册 consumer + 颁发 consumer key

| AccessResolver → 解析 AccessEntries 模板 (\${self.name}...)



→ 调 credprovider 申请 MinIO 桶权限

| Credentials store → 写入 secret + 设 refresh 时刻



└─[2] WorkerBackend.CreateWorker(spec):

```

|         docker.go 或 kubernetes.go (依 config)
|     → 把 PodTemplate + 环境变量 + soul/identity/skills 卷挂上
|     → 启动 Worker 容器
|
|     └─[3] member_reconcile.go:
|         把 Worker Matrix user 加入相关 Team 房间 (如有)
|
|     └─[4] 更新 Worker.Status:
|         Phase = Running
|         MatrixUserID = "@go-review:hiclaw.local"
|         RoomID      = "!abc...:hiclaw.local"
|         ContainerState = "running"
|         ExposedPorts  = [...]    ← 若 Spec.Expose 有端口

```



Worker 容器启动后:

- Python runtime (copaw/hermes): bridge.py 用 consumer key 登录 Matrix, 订阅自己的房间
- OpenClaw runtime: 宿主 Claude Code 自带 hook 完成同等动作



管理员立刻在 Element Web 看到 @go-review 出现并打招呼; 后续的代码评审请求可直接 @

稳态通信信道

```

Worker → LLM:      Worker 容器 —consumer key→ Higress AI Gateway —真实 API
key→ LLM provider
Worker ↔ Worker:  双方都加入同一 Matrix 房间, 消息走 Twunel; 大文件落 MinIO (节省
token)
Worker ↔ Human:   Matrix 房间 (管理员是房间成员, 可以随时介入)
Worker → 工具:     Worker 容器 —consumer key→ Higress —route→ MCP server
(external 或 in-cluster)
Manager → 控制器:  Manager 容器内的 hiclaw CLI —HTTP→ controller —CR write→
etcd/kine

```

设计决策与哲学

- [声明式 Agent 运维](#): Agent 是 CR 不是对象。免费拿到 K8s 的 declarative / reconcile / self-healing / RBAC——这是 HiClaw 与 langgraph / autogen / crewai 这类「代码中编排 Agent」框架的根本不同。

- IM 协议作为协作平面 *lingua franca*：选 Matrix 而非自研 Web UI/事件总线——每个 Agent 是 IM 用户，每场协作是 IM 房间，人类天然就是房间成员，可观测性与人在回路同时拿到，避免两个大坑。代价：所有 Agent 都要做 Matrix 适配。
- **凭据零暴露**：Worker 永远拿不到真实 API key / GitHub PAT / OSS AK，只持 [Higress](#) consumer key。结果：MCP 工具市场和 skills 市场可以大胆装载社区作品——即便 Worker 被 prompt injection 攻陷，攻击者拿到的也只是可被即时 revoke 的 consumer key。
- 运行时三选一可插：WorkerSpec.Runtime 字段切换 OpenClaw (claude-code 系) / QwenPaw (Qwen Code 系，省钱) / Hermes (autonomous coding agent, 自主)。最近 commit d3e33e8 把默认 runtime 从 OpenClaw 切到 QwenPaw，价格敏感倾向明显。
- 容器后端双轨：WorkerBackend 接口 + docker.go / kubernetes.go 两实现，让"单机 Docker Desktop 跑 5 个 Worker"与"K8s 集群跑 500 个 Worker"共用同一 controller 二进制。
- 嵌入式 K8s (kine + 内嵌 controller-manager)：单机模式跑 kine (SQLite-backed etcd) + 同进程的 controller-manager，于 127.0.0.1:2379。用户感觉装了 Docker，骨子里是 K8s API——CRD / Reconcile / Helm 在无真 K8s 环境也成立。
- schema-less 模板字段：AccessEntry.Scope 用 *apiextensionsv1.JSON 而非严格 struct，支持 \${self.name} / \${self.kind} / \${self.namespace} 运行期变量。CRD 不绑定特定 cloud provider；解析延迟到 accessresolver。
- hiclaw CLI 注入 Manager / Worker 容器：Manager Agent 不引用 Go SDK，通过 fork CLI 操作 controller HTTP API——让 skill (markdown 文档) 可以直接生成 hiclaw worker create ... 这种命令字符串。
- MCP via Nacos 包市场：Worker 能力通过 WorkerSpec.Package 字段 (nacos:/// http(s):/// file:///) 按需加载。Manager 侧的 hiclaw-find-worker skill 就是做 Nacos 搜索 + 导入。Worker 模板像 docker 镜像一样可分发。

关键组件深入：Provisioner

internal/service/provisioner.go (1154 行 / 29 公开方法) 是 controller 的脏活集中地，可分三组：

1. Matrix 编排 (ProvisionWorker / ProvisionTeamRooms / EnsureRoomMember / ReconcileRoomMembership ...) ——通过 Tuvunel admin API 完成 user / room / membership / alias 全生命周期；MatrixUserID(name) 与 roomAliasLocalpart(kind, name) 是命名规则集中点，保证幂等。
2. Higress 凭据 (EnsureManagerGatewayAuth / EnsureWorkerGatewayAuth / RefreshCredentials / DeleteCredentials) ——用 alibabacloud-go/apig-20240327/v6 在 Higress 上为每个 Agent 注册 consumer + key。

3. Manager 引导 (IsManagerLLMAuthReady / IsManagerJoinedDM / SendManagerWelcomeMessage) ——容器首启时检查 Manager 是否就绪 + 投递欢迎语 (按 language / timezone 渲染)。

横向拆分时机已到：按 Matrix / Higress / Credentials / Manager-bootstrap 四个域拆到独立文件，Provisioner 退化为外观。

与同类对比

维度	HiClaw	autogen	langgraph	crewai
形态	K8s operator + 容器化	Python library	Python library	Python library
Agent 表达	CR (YAML)	Python class	StateGraph node	Crew/Agent class
部署单元	Pod / Container	进程	进程	进程
协作面	Matrix 房间	In-process bus	StateGraph 边	Crew 内调度
人在回路	默认 (房间成员)	需自接 UI	需自接 UI	需自接 UI
凭据模型	网关托管	Agent 持真 key	Agent 持真 key	Agent 持真 key
跨 Agent 文件	MinIO 桶	自实现	自实现	自实现

最大差异：HiClaw 把 Agent 当 K8s 一等公民资源，所以是"可被 reconcile 自愈 / 可被 helm 上线 / 可被 RBAC 限定"的东西。其余框架本质是"在你进程里跑一段 Python"，做企业级运维要重头补。代价：学习曲线高、每 Worker 一个容器有 RAM / 镜像开销。

演进信号

- 近 30 天最活跃: install/hiclaw-install.sh (20) · internal/service/provisioner.go (12) · api/v1beta1/types.go (12) · internal/config/config.go (11) → 主线是"装上 / 升级体验" + "controller 核心模型稳定"
- 关键 PR: #780 默认 runtime → QwenPaw · #749 --keep-all 升级 · #783 helm uninstall 残留清理 · #785 manager 容器 API 对齐 controller /api/v1 · #798 error() 多行安全
- v1.0 → v1.1 阶段，下一步看点: Hermes runtime 主线化、多租户 RBAC、kine → 真 etcd 平滑迁移工具

相关页面

- [HiClaw](#)
- claude-code
- [higress](#) · matrix-protocol · mcp · k8s-operator
- [declarative-agent-management](#)
- [agent-credential-isolation](#)

- autogen · langgraph · crewai

Higress

Higress 是阿里系 AI Native API Gateway，基于 Envoy/Istio 控制面和多语言 WASM plugins。它在当前 wiki 中主要用于理解 AI gateway、模型路由、MCP/HTTP 策略和凭据治理，尤其是 [HiClaw](#) 背景中的网关能力。详见 [src-higress-architecture](#)。

架构边界

Higress 是通用 API Gateway 产品向 AI Gateway 扩展，不是只服务 LLM 的窄代理。它的优势来自成熟 API gateway 能力、服务发现、插件生态和 WASM 扩展。

关键设计

- cmd/higress、pkg/** 承载控制面和网关逻辑。
- plugins/** 提供 Go/Rust/C++/AssemblyScript 等多语言 WASM 插件。
- Envoy/Istio 基础设施负责通用 L7 流量治理。
- model-router、credential governance、MCP/LLM 插件把网关扩展到 AI 工作负载。

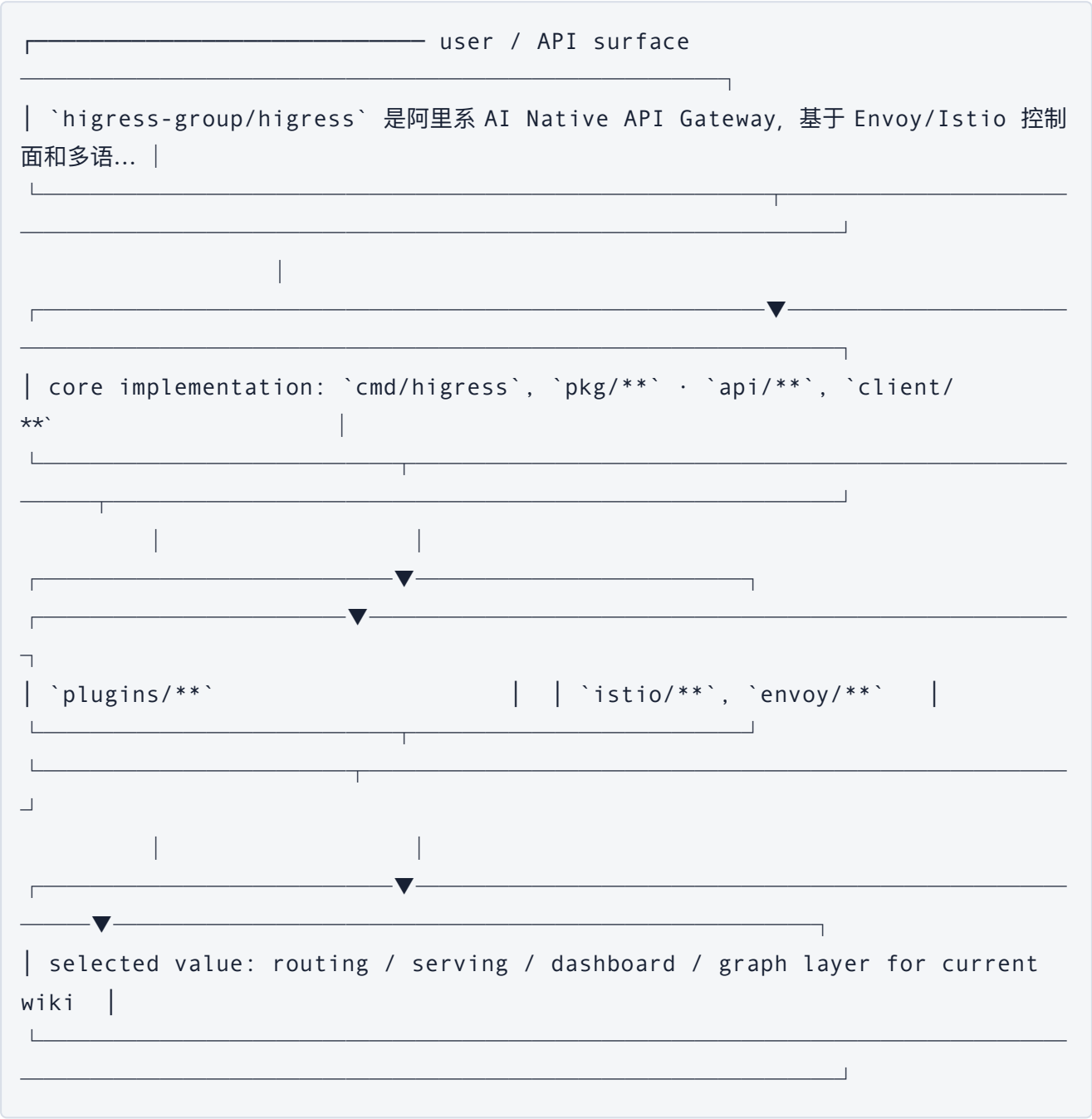
选型判断

需要传统 API gateway 和 AI gateway 合一时看 Higress；需要 GenAI 专用 Envoy Gateway 方案看 [Envoy AI Gateway](#)；需要 Kubernetes Gateway API 原生通用网关看 [kgateway](#)；需要 agent-side LLM/MCP/A2A 出口治理看 [agentgateway](#)。

Higress 架构与设计思路分析

higress-group/higress 是阿里系 AI Native API Gateway，基于 Envoy/Istio 控制面和多语言 WASM plugins。P1 中它的价值是 AI gateway/模型路由/凭据治理背景，尤其和 HiClaw 的凭据托管、MCP/LLM 网关能力相关。

核心架构图



模块分层

层/目录	责任
cmd/higress, pkg/**	Higress 控制面与网关逻辑。
api/**, client/**	API 与客户端。
plugins/**	WASM 插件：Go/Rust/C++/AssemblyScript。
istio/**, envoy/**	上游控制面/数据面依赖。

关键数据流

1. 用户配置 route/model-router/plugin。
2. 控制面生成 Envoy/Istio 配置并下发。
3. WASM 插件在数据面处理 AI auth、model route、MCP/HTTP 策略。

设计决策

- 插件化和服务发现能力强，适合传统 API gateway + AI gateway 融合。
- model-router 保留原始模型名选项说明多 provider/model 映射是活跃问题。
- 仓库较大，应聚焦 plugin/model-router/control plane，不必全读 Istio vendor。

对比定位

和 kgateway 相比，Higress 更偏 API gateway 产品和插件生态；和 Envoy AI Gateway 相比，范围更宽但 GenAI 专注度较低。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

Inference Routing

Inference routing 指 LLM/模型服务请求进入 serving fleet 后，如何选择 provider、模型、endpoint、pod、prefill/decode worker 或 KV cache 命中路径。

在 M4 中的位置

Routing 位于流量入口与 engine 执行之间。llm-inference-serving-project-map 的 D1/D2 分别展示职责边界与请求路径，区分 Gateway、Proxy/EPP/Router 和 Model Server。路由负责选择请求去向，engine 负责执行推理；选型时不能把两者当作替代品。

四类路由

类型	代表	决策信号
Gateway/API endpoint picking	gateway-api-inference-extension, llm-d-router	endpoint health、metrics、InferencePool、目标模型
语义/模型能力路由	semantic-router , RouteLLM	prompt semantic、成本/质量、guard、模型能力
KV/cache aware routing	llm-d-kv-cache, dynamo, kthena, aibrix	prefix overlap、KV block location、worker load、LoRA/cache affinity
Batch-to-serving dispatch	llm-d-batch-gateway	batch job、per-model plan、processor concurrency、下游 endpoint capacity
Edge/provider governance	envoy-ai-gateway , ai-gateway	auth、provider、quota、rate limit、audit、fallback

选型提示

不要把所有 router 混为一类。llm-d-router / Gateway API router 更靠近 Kubernetes endpoint；semantic router 更靠近 prompt/model selection；KV-aware router 更靠近 serving runtime 和 cache locality，llm-d-kv-cache 则把 KV event/index/scoring 这层单独拆出来。dynamo 的补充是：selection host 可以是 Dynamo Frontend、GAIE EPP 或 standalone router，但 KV overlap 与 active load 的选择模型保持一致。

llm-d-inference-sim 对 routing 选型很有用：它不会给出真实 GPU 性能，但能在无 GPU 环境中模拟 OpenAI/vLLM API、KV block/cache events、TTFT/ITL 和 fake metrics，从而验证 endpoint picking、KV-aware scoring 和 autoscaling 闭环。llm-d-batch-gateway 则说明 batch workload 的“路由”更多发生在 processor 的 per-model plan 和并发控制层，不应和在线低延迟 request routing 混在一起评估。

KV-aware 路由决策树

```
请求 → 协议/凭证/配额通过? — 否 → 拒绝或 fallback
      ↓ 是
      模型与能力匹配? ——— 否 → model router / cascade
      ↓ 是
      endpoint 健康? ———— 否 → 排除
      ↓ 是
      prefix/KV locality? → 计算 cache credit
      ↓
      综合 queue + KV + SLO + cost
      ↓
      选择 endpoint / prefill-decode pair
      ↓
      结果与 KV events 反馈回路路由
```

网关层与 Endpoint Router 的组合

```
Client
  | OpenAI / Anthropic request
  ▼
Envoy AI Gateway (可选)
  | auth · provider · quota · global rate limit
  ▼
Semantic Router / RouteLLM (可选)
  | model · capability · quality/cost · cascade
  ▼
Gateway API + Inference Extension
  | InferencePool / Endpoint Picker protocol
  ▼
llm-d Router EPP
  | KV · queue · priority · health · pod placement
  ▼
vLLM / SGLang / TRT-LLM endpoint
```

这几层可以组合，但不是必须全部部署。[envoy-ai-gateway](#) 解决 provider 与边缘治理，[semantic-router](#)/[routellm](#) 解决模型/能力选择，[gateway-api-inference-extension](#) 定义标准协议，[llm-d-router](#) 负责 Kubernetes serving endpoint 选择。

kgateway

kgateway 是通用 cloud-native API Gateway，并带 AI Gateway 能力。仓库核心是 Gateway API 资源、controller/deployer、Envoy xDS、plugins、policies、SDS、安全和 conformance/e2e。详见 [src/kgateway-architecture](#)。

架构边界

kgateway 的主线是 Kubernetes / Gateway API 原生通用网关，AI Gateway 是其 policy/plugin 能力分支。它不是只面向模型 API 的窄代理。

关键设计

- api/v1alpha1、pkg/kgateway、pkg/xds 负责 Gateway API 扩展、controller 和 xDS 生成。
- pkg/plugins、pkg/pluginsdk 提供扩展机制。
- pkg/deployer 和 pkg/sds 管 Envoy 部署和 Secret Discovery。
- ReferenceGrant / cross-namespace 引用治理是安全边界重点。

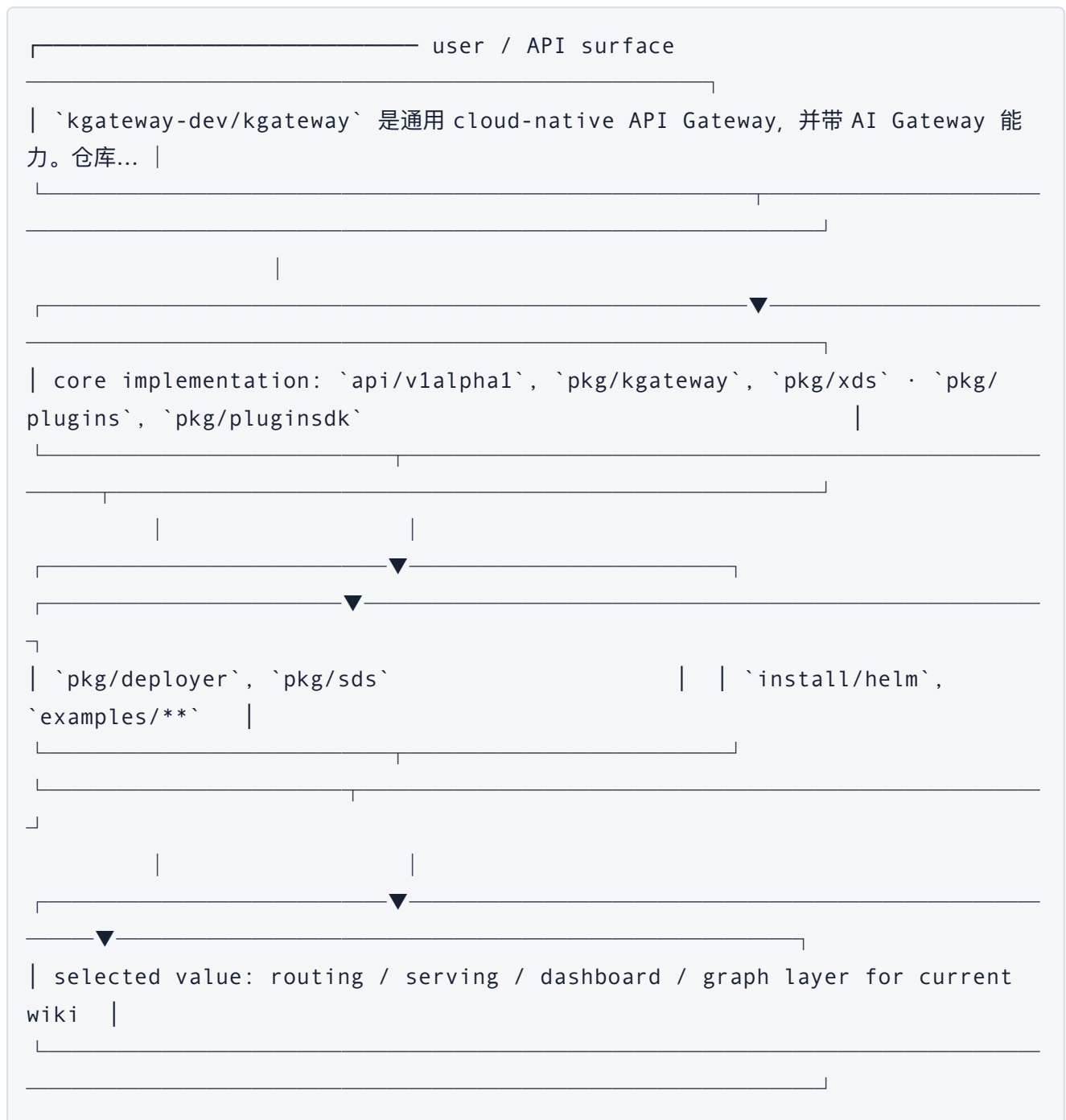
选型判断

已有 Gateway API / Envoy 体系并想统一通用 API 与 AI gateway 时看 kgateway。GenAI 专用治理看 [Envoy AI Gateway](#)；产品化插件网关看 [higress](#)；agent protocol unified gateway 看 [agentgateway](#)。

kgateway 架构与设计思路分析

kgateway-dev/kgateway 是通用 cloud-native API Gateway，并带 AI Gateway 能力。仓库核心是 Gateway API 资源、controller/deployer、Envoy xDS、plugins、policies、SDS、安全和 conformance/e2e；最近 reference grant mode 也说明跨 namespace 引用治理是主路径。

核心架构图



模块分层

层/目录	责任
api/v1alpha1, pkg/kgateway, pkg/xds	Gateway API 扩展、controller 和 xDS 生成。
pkg/plugins, pkg/pluginsdk	插件系统。
pkg/deployer, pkg/sds	Envoy 部署和 Secret Discovery。
install/helm, examples/**	安装和示例。

关键数据流

1. Gateway API/自定义 policy 被 controller watch。
2. kgateway 生成 Envoy 配置/xDS 并部署/更新代理。
3. plugins/policies 处理 auth、traffic、AI 等增强能力。

设计决策

- 先做通用 Gateway，再把 AI 场景做成 policy/plugin。
- reference grant 强化跨 namespace 安全边界。
- 适合已有 Gateway API/Envoy 体系的团队。

对比定位

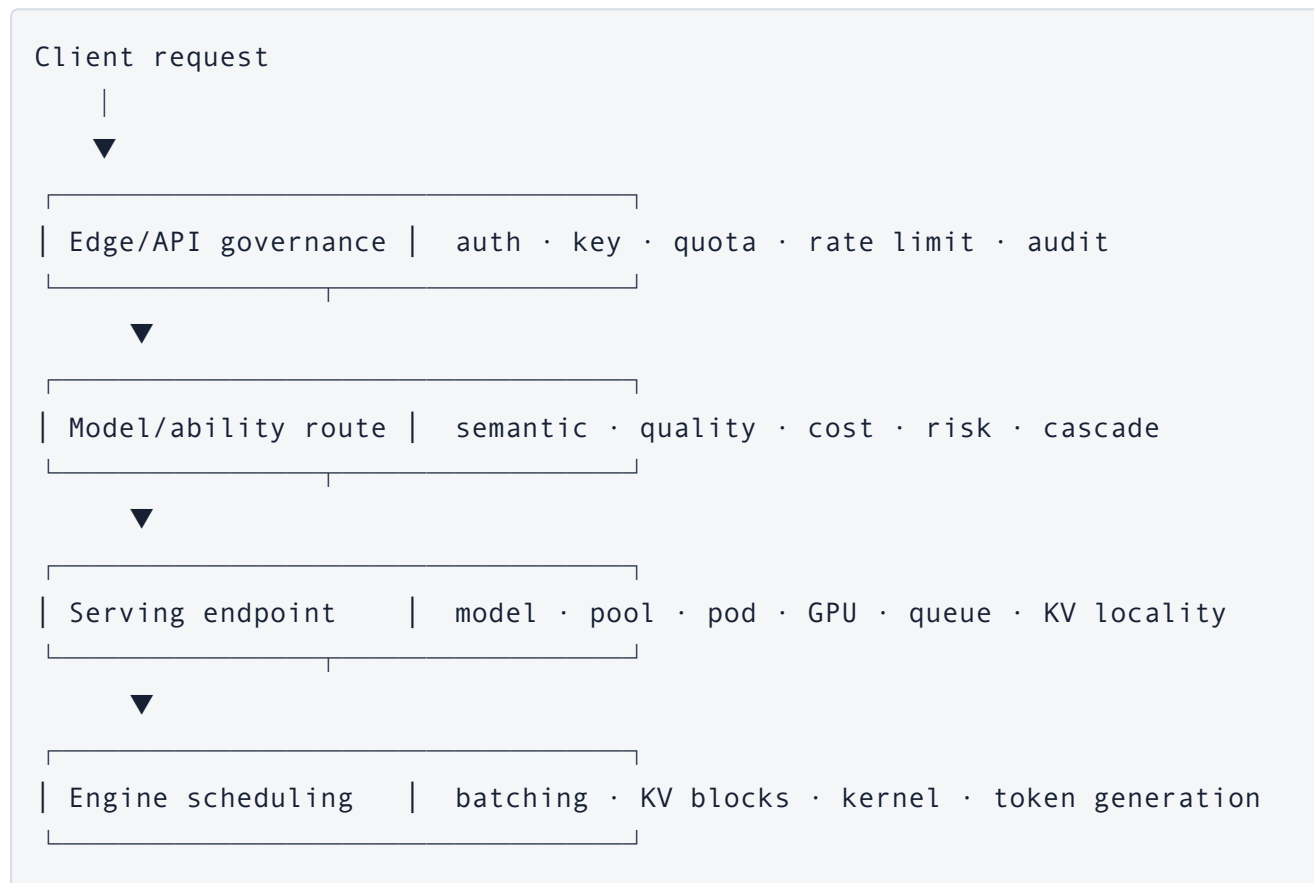
和 Envoy AI Gateway 相比，kgateway 范围更宽；和 Higress 相比，它更 Kubernetes/Gateway API 原生。

相关链接

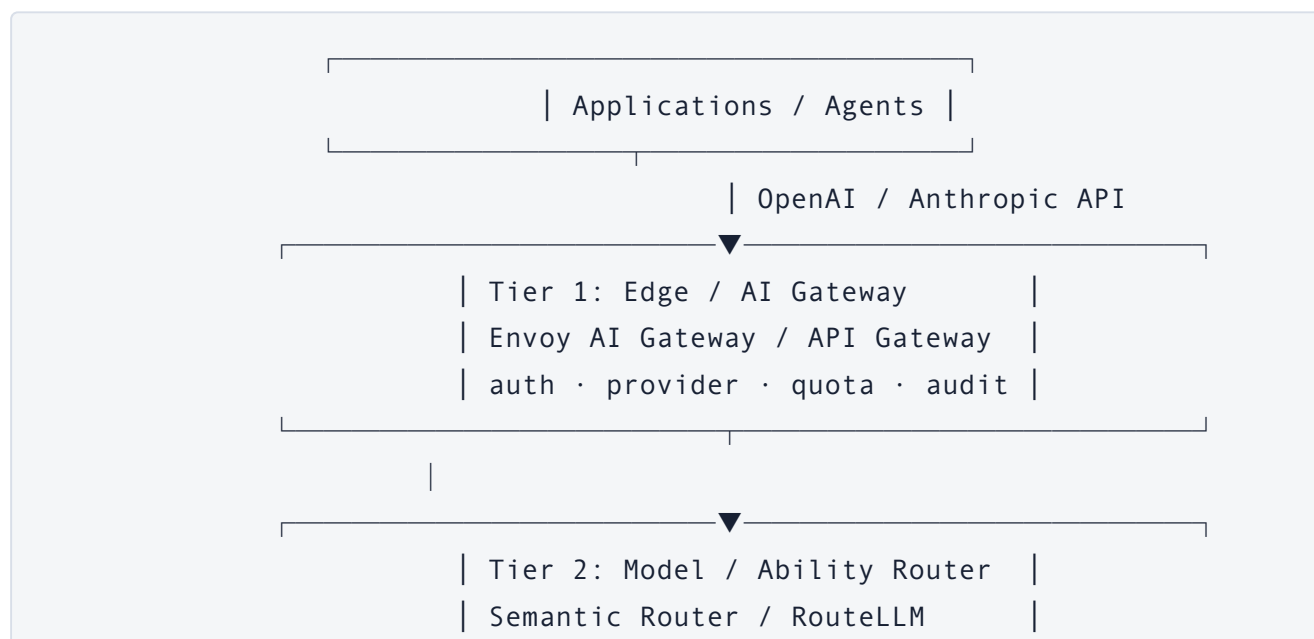
- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

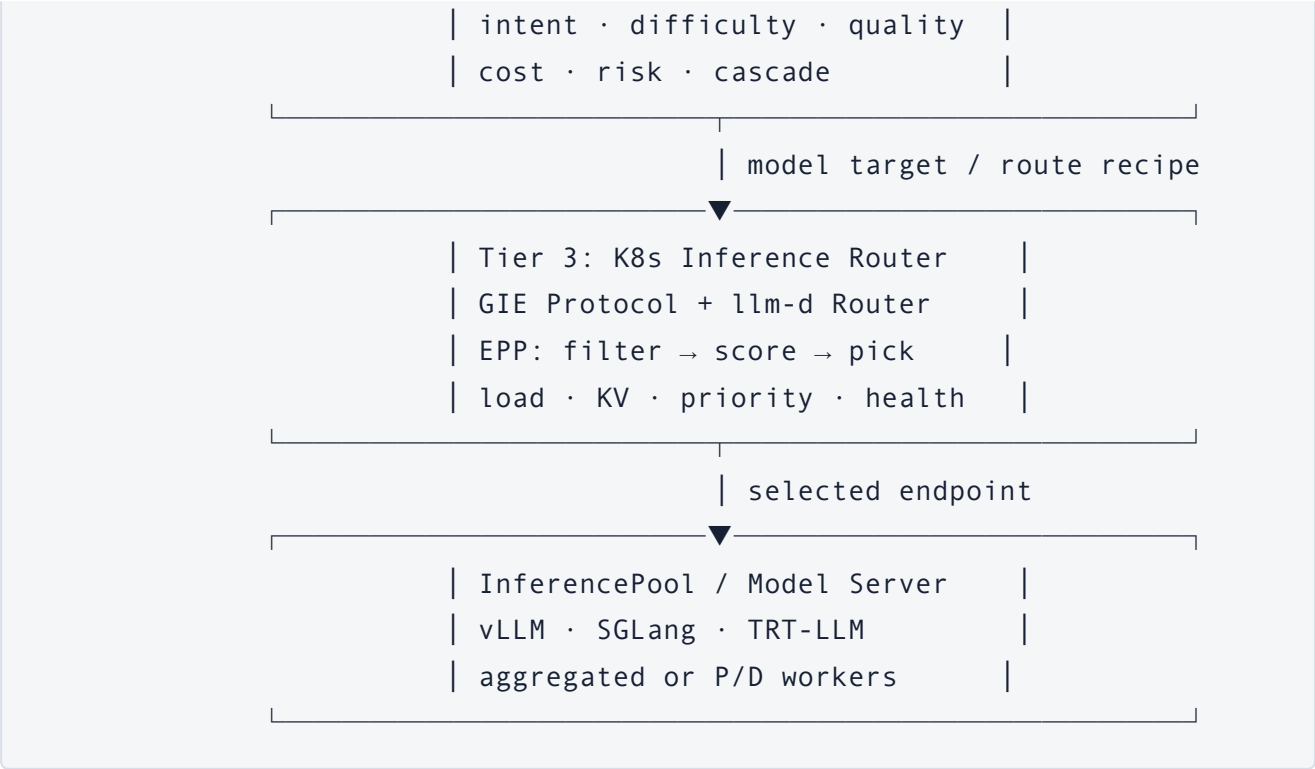
LLM 网关与路由层对比

先分清四类决策



统一架构图



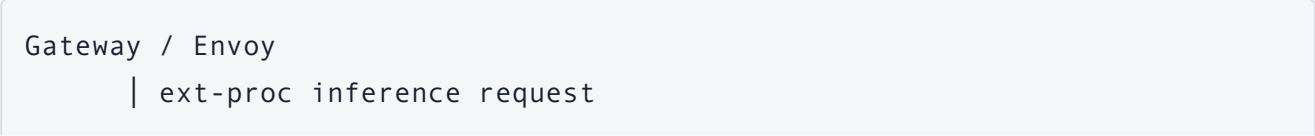


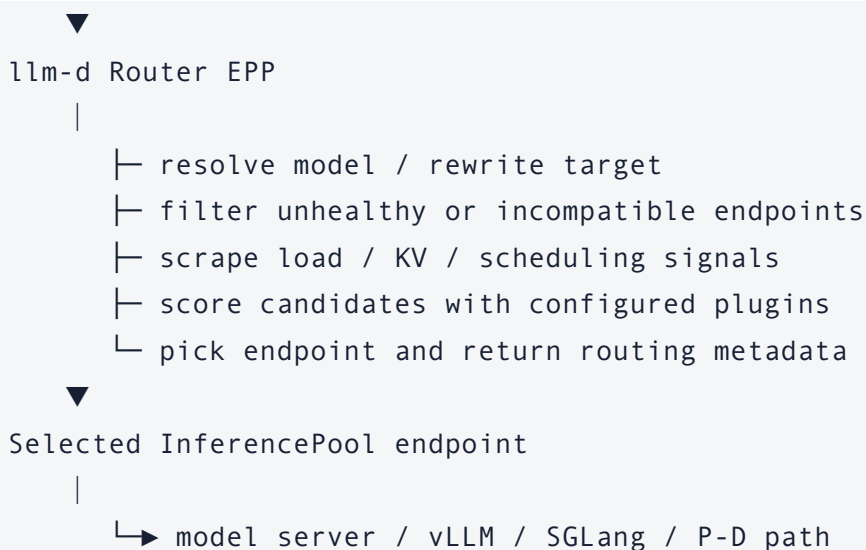
项目定位

项目	决策层	核心输出	Kubernetes 边界
llm-d-router	endpoint/pod/ worker	selected endpoint + routing metadata	EPP/ext-proc、Gateway API、 InferencePool
semantic-router	model/ability/ recipe	model、cascade、 capability path	可部署在 K8s，但不是 GIE 协 议本身
routellm	strong/weak model	cheaper 或 stronger model	SDK/OpenAI-compatible server，不是 operator
gateway-api-inference- extension	inference endpoint protocol	inference endpoint pick	Gateway API 的推理扩展
envoy-ai-gateway	edge/provider governance	provider/model upstream	Envoy Gateway + two-tier gateway

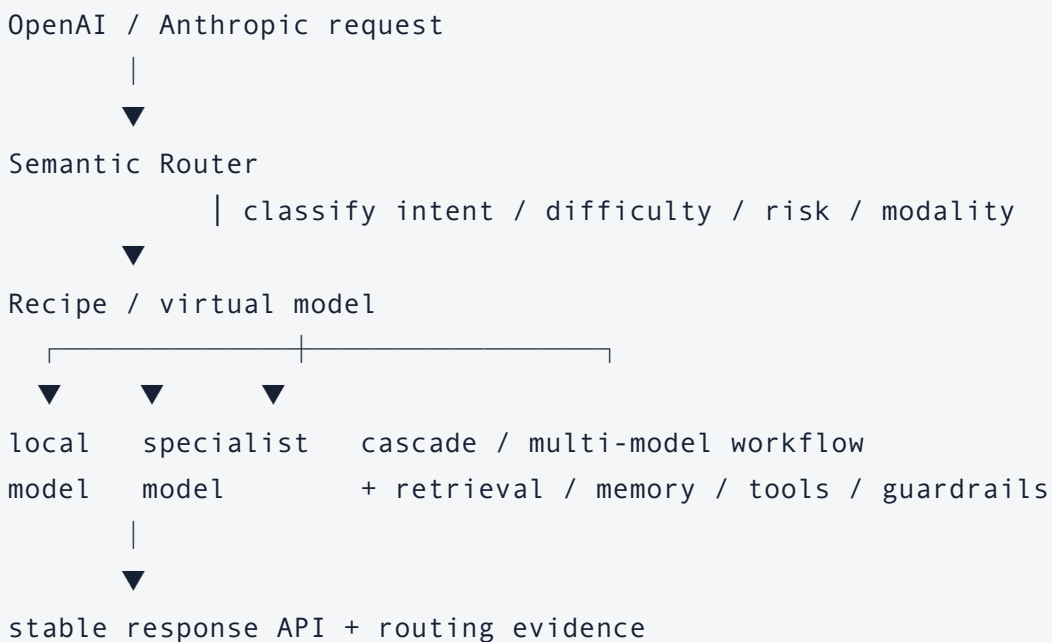
五条请求流程图

llm-d-router

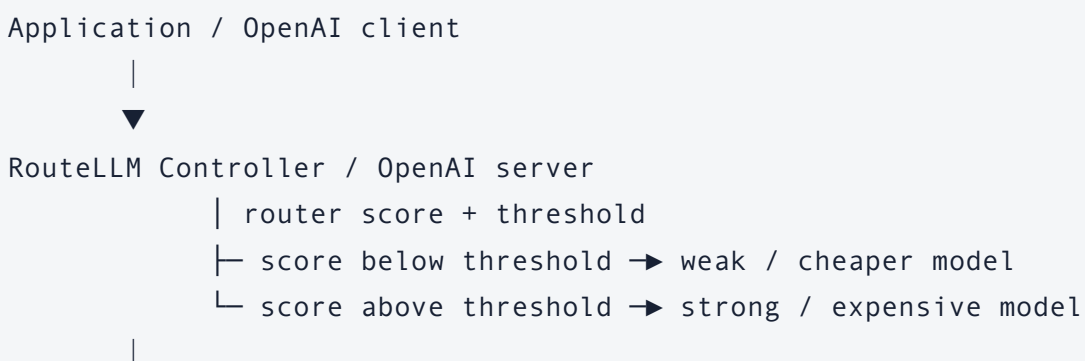


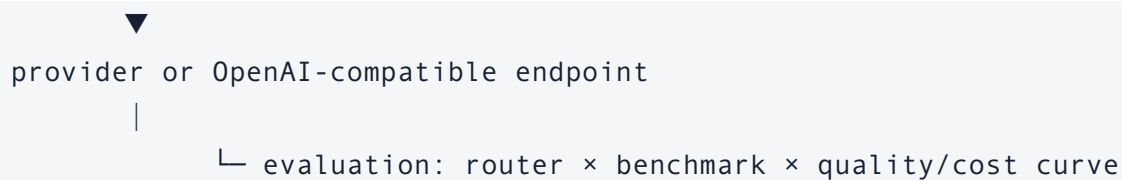


Semantic Router

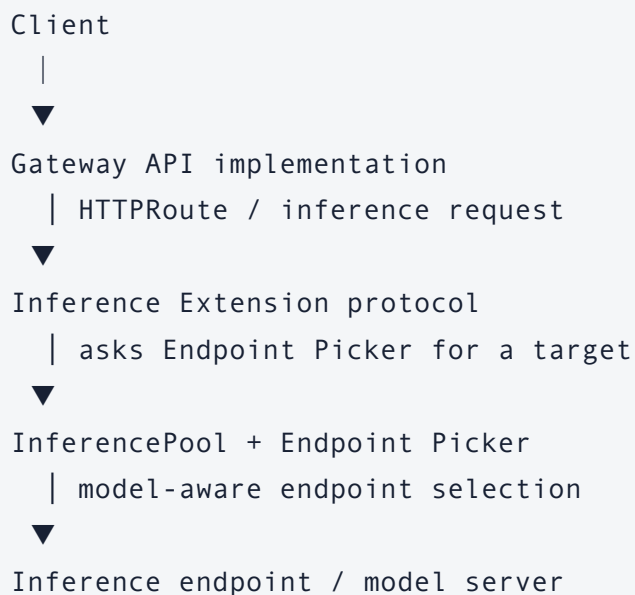


RouteLLM





Gateway API Inference Extension



Envoy AI Gateway



选型结论

- provider、key、quota、限流和审计: [envoy-ai-gateway](#)。
- 语义、风险、能力和质量-成本模型选择: [semantic-router](#) / routellm。

- KV、队列、GPU、健康和 InferencePool endpoint 选择: gateway-api-inference-extension / llm-d-router。
- 多层组合时，必须记录每一层的 route decision、模型名、endpoint、策略版本和失败原因。

相关页面

- [inference-routing](#)
- [ai-gateway](#)
- llm-inference
- [src-k8s-gateway-routing-comparison](#)

NemoClaw

NemoClaw 是 NVIDIA 为 [OpenShell](#) sandbox 内 always-on AI Agent 提供的 TypeScript CLI 控制面。它把 guided onboarding、gateway、provider inference、sandbox 生命周期、agent setup、network policy、凭证迁移和 e2e 验证收束成一个命令面。详见 [src-nemoclave-architecture](#)。

架构边界

NemoClaw 不是新的推理引擎，也不是新的多 Agent framework。它更像 [OpenShell](#) 的 host-side 编排层：在宿主机上规划 gateway、provider、sandbox、OpenClaw/Hermes runtime、policy preset 和 messaging channel，然后把结果应用到 sandbox。

关键设计

- 命令层保持很薄，`src/commands/**` 主要负责参数解析。
- `onboard` 是可恢复状态机，串起 `preflight`、`gateway`、`provider inference`、`sandbox`、`agent setup` 和 `policy`。
- Provider 凭证迁移到 `gateway`，agent 通过 `inference.local` 调模型，符合 [agent-credential-isolation](#)。
- Policy 默认 `deny-by-default`，再叠加 `selected presets` 和 `messaging channel` 所需网络权限。
- Messaging channel 通过 `manifest` 编译成 `credential`、`network policy`、`agent render`、`health check` 等计划。

选型判断

需要把 OpenClaw/Hermes 这类个人 agent 放进可审计 sandbox 并接入多 provider 推理时看 NemoClaw。需要底层 runtime enforcement 时看 [openshell](#)；需要 Kubernetes 原生 sandbox 时看 [agent-sandbox](#)；需要统一治理 LLM/MCP/A2A 出口时看 [agentgateway](#)。

NemoClaw 架构与设计思路分析

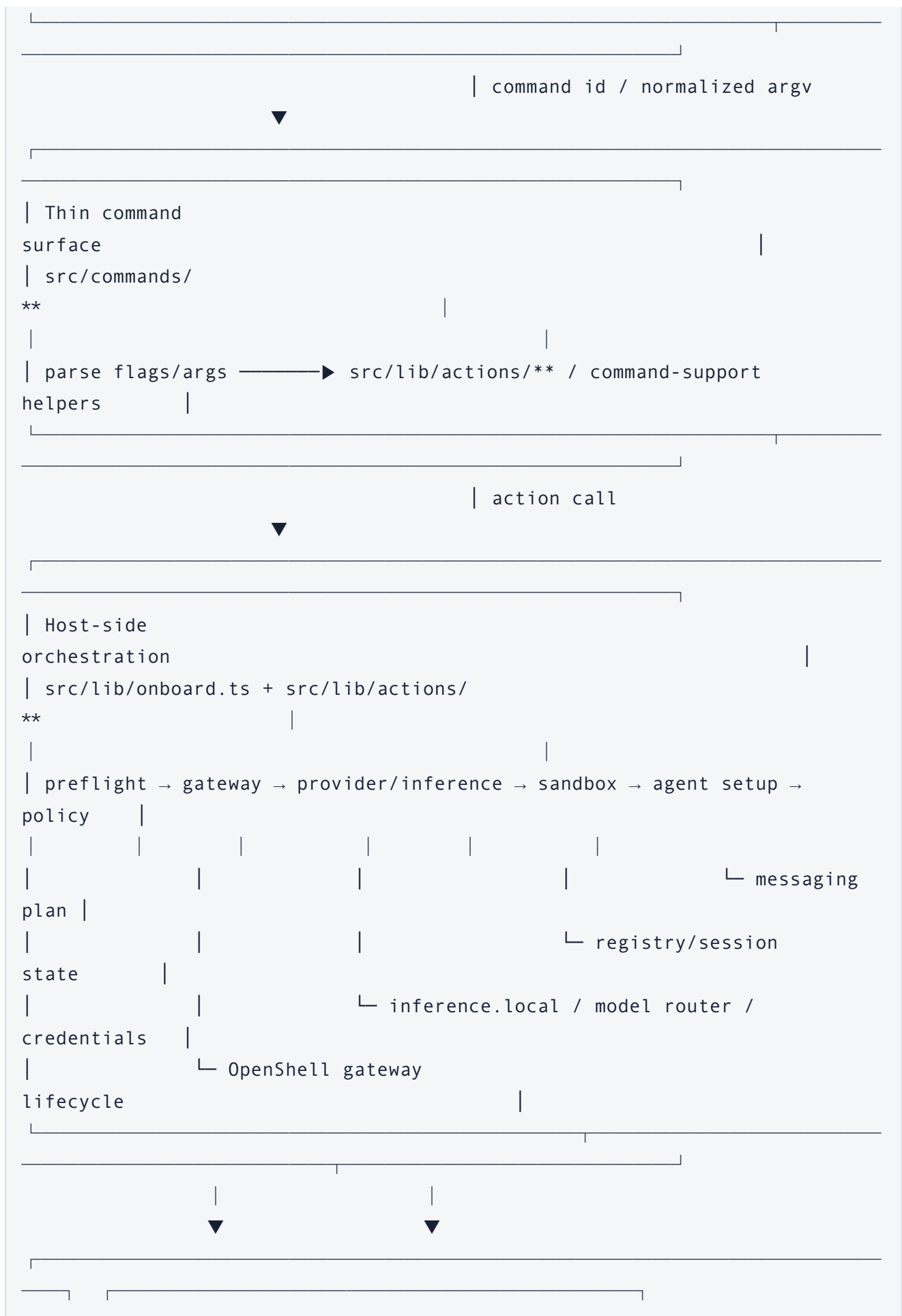
原文: raw/nemoclave-architecture-analysis.md · 仓库: <https://github.com/NVIDIA/NemoClaw> · 分析版本 HEAD 3c0340a

一句话定位

[NemoClaw](#) 是 NVIDIA 为 [OpenShell](#) sandbox 内运行 always-on AI Agent 提供的 TypeScript CLI 控制面：它把 guided onboarding、gateway、OpenClaw / Hermes agent 配置、routed inference、网络策略、凭证迁移、sandbox 生命周期和 e2e 验证收束到一个命令面。它更接近 [agent-sandbox](#) 与 [agentgateway](#) 之间的 host-side 编排层，而不是新的推理引擎；推理侧通过 `inference.local` 接入 NVIDIA/OpenAI/Anthropic/Gemini/vllm/Ollama 等 provider。

核心架构图





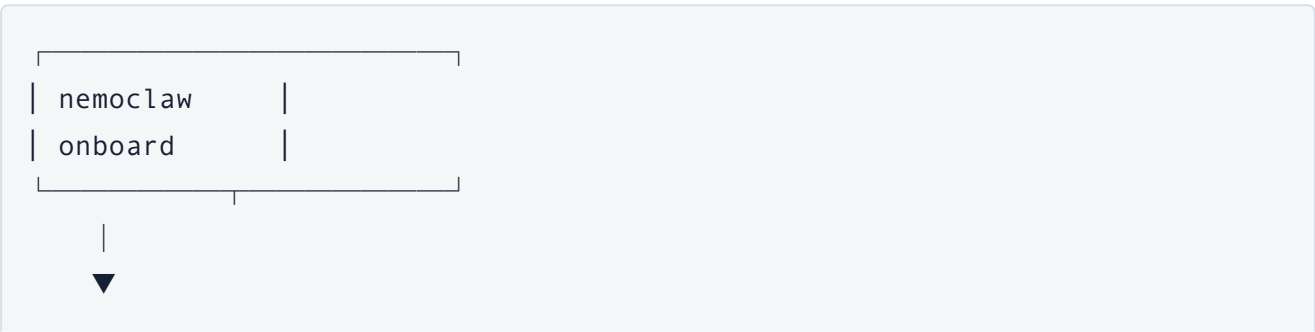
模块分层

层 / 模块	职责
CLI 入口与公共语法	兼容 nemoclaw <sandbox> <action>, 做 argv normalization、route translation、sandbox 名检查, 再转给 oclif。
命令层	src/commands/** 只解析 flags/args 并调用 action, 不承载产品行为。
Action / onboard 编排层	把命令变成 preflight、gateway、provider/inference、sandbox、agent setup、policy 的 host-side workflow。
领域规划层	provider 映射、policy/lifecycle 分类、messaging manifest 编译等尽量保持纯函数或可测试规划逻辑。
外部边界适配	封装 OpenShell、Docker、HTTP、shell 子进程和 privileged sandbox exec。
状态与凭证	~/ .nemoclaw JSON registry/session/shields 状态; 凭证只在进程 env staging, gateway 是 system of record。
安全与策略	baseline deny-by-default policy、policy presets、shields up/down/status、redaction/hash。
Messaging 扩展	channel manifest → serializable plan → OpenShell provider/policy/agent config applier。
推理与模型路由	inference.local 统一 sandbox 内出口, host/gateway 侧映射到 cloud 或 local provider。
Blueprint 与测试	OpenShell sandbox 资产、policy preset、runtime scripts、unit/e2e/e2e-scenario 验证矩阵。

核心分层约束是"薄命令 + 厚编排 + 纯规划 + 外部边界适配"。这与 ai-agent-plugin-patterns 里的接口化、边界清晰、可迁移能力包方向一致, 也和 [agent-credential-isolation](#) 的凭证边界设计相互呼应。

关键数据流

Onboard 主流程



```
| src/lib/onboard.ts:onboard |
```

- ```
| 1. branding/env/options: agent, non-interactive, resume, fresh |
```
- ```
| 2. validate sandbox name + usage notice + provider hint |
```
- ```
| 3. acquire onboard lock + load/create resumable session |
```
- ```
| 4. stage legacy credentials into process.env for gateway migration |
```
- ```
| 5. select agent: OpenClaw / Hermes |
```
- 



---

```
| state handlers |
```

```
| handlePreflightState |
```

```
| └ Docker/OpenShell/GPU/DNS/platform checks |
```

```
| handleGatewayState |
```

```
| └ start/reuse/recover named OpenShell gateway |
```

```
| handleProviderInferenceState |
```

```
| └ provider/model/credential route; model router/local proxy |
```

```
| handleSandboxState |
```

```
| └ create/reuse/recreate sandbox, persist registry/session |
```

```
| handleAgentSetupState |
```

```
| └ install/sync OpenClaw or Hermes runtime config |
```

```
| handlePoliciesState |
```

```
| └ apply baseline + selected policy presets |
```

---



---

```
| OpenShell gateway + sandbox side effects |
```

```
| openshell gateway start/reuse |
```

```
| openshell provider create/update --credential <ENV_KEY> |
```

```
| openshell sandbox create/connect/exec |
```

```
| openshell policy set |
```

---

```
| ~/.nemoclave/sandboxes.json + session files |
```

Onboard 是可恢复的 host-side 状态机，不是一次性安装脚本。它支持 resume/fresh/non-interactive、全局 lock、step 记录和失败恢复；近期提交也集中在把 provider inference、sandbox branch、OpenClaw setup 等分支抽成 FSM result。

## Messaging channel 编译与应用

```
| channel manifests |
| telegram/slack/... |
```

```
| registry.list()
```



```
| MessagingWorkflowPlanner |
| - de-duplicate configured channels |
| - reject unsupported channels for selected agent |
| - build ManifestCompilerContext |
```



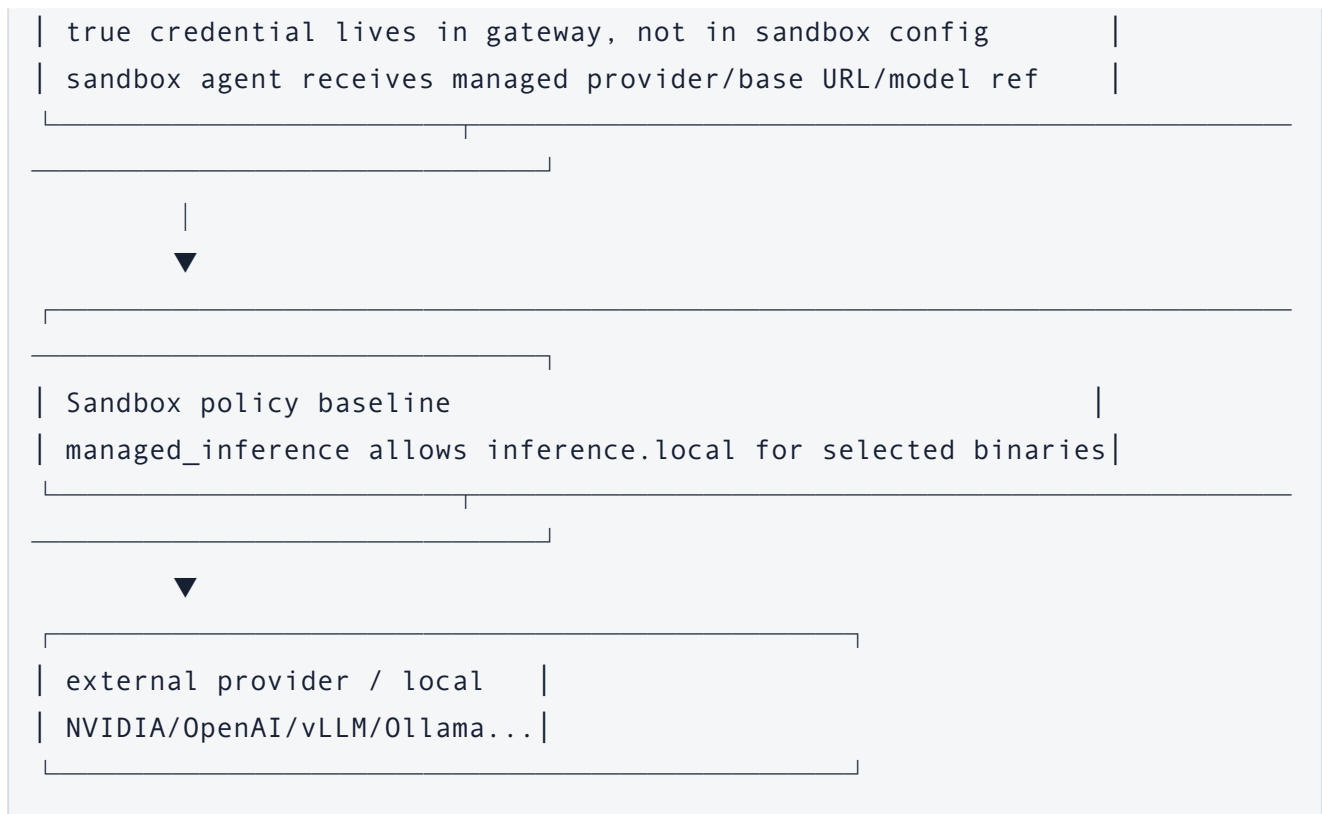
```
| ManifestCompiler.compile |
| |
| resolve manifests |
| compile per-channel inputs + enrollment/reachability hooks |
| fan out into: |
| credentialBindings |
| networkPolicy |
| agentRender |
| buildSteps |
| stateUpdates |
| healthChecks |
```



Messaging 子系统的关键是 declarative manifest compiler: channel 不直接侵入 onboard 主流程, 而是声明 secret/config input、policy preset、agent render、hook phase 和 health check, 再编译成 serializable plan。

## 推理路由与凭证边界





这里的设计重点是 [agent-credential-isolation](#): sandbox 看到的是 `inference.local` 和 `managed provider/model ref`, 真正 provider credential 由 [OpenShell](#) gateway 托管。local Ollama/vLLM 还有专用 token env, 避免误用 host OpenAI key。

## 设计决策与哲学

- 命令面兼容产品语法, 但不复制业务层: public dispatcher 负责 `nemoclaw <sandbox> <action>` 到 oclif command 的转换, 产品行为仍在 actions/onboard。
- Onboard 是可恢复状态机: resume/fresh/non-interactive、session、lock、step result 和 failure marking 都是一等流程, 而不是 shell script 串联。
- Declarative assets + host orchestrator: `nemoclaw-blueprint/**` pin 住 sandbox image digest、OpenShell/OpenClaw 版本、inference profile、router 和 policy; TypeScript 负责消费和编排。
- 凭证由 gateway 托管: NemoClaw 只做当前进程 env staging 与 legacy migration, 新凭证不写本地明文文件。
- 网络默认最小化: baseline policy 只开放核心 agent、managed inference 和必要服务; GitHub、messaging、package index 等走用户显式 policy preset。
- Shields 只能 host-side 控制: sandbox 不能自己降级或升级 shields; host 通过 privileged path 做 config lock、policy 切换和 hash seal。
- Messaging 走 manifest compiler: 新增 channel 的主要扩展面是 manifest + hook, 而不是修改 onboard 主流程。
- 本地 JSON 状态优先: `~/.nemoclaw registry/session` 降低部署复杂度, 但用 advisory lock、resume 和 recovery 补齐一致性。

## 关键组件深入解读

### Onboard 编排器

`src/lib/onboard.ts` 是复杂度中心，但它更像 coordinator：初始化运行模式、校验 sandbox 名、获取 consent、抢 lock、加载/创建 session、迁移 legacy credentials，再顺序调用 `preflight/gateway/provider/sandbox/agent/policy` 状态处理器。近期 git 历史显示维护方向是在保留一个端到端入口的同时，把巨型分支拆成 FSM result 和可测试状态模块。

### Messaging manifest compiler

Messaging compiler 对每个 channel manifest 做 input resolution、credential availability、enrollment/reachability hook，然后 fan out 成 credential binding、network policy、agent render、build step、state update 和 health check。Applier 接受 serializable `SandboxMessagingPlan`，说明 plan 是跨 host/OpenShell/sandbox 边界传递的 contract，而不是运行时对象。

## 相关页面

- [agent-sandbox](#)
- [agentgateway](#)
- [agent-credential-isolation](#)
- cloud-native-security
- vllm
- ai-agent-plugin-patterns
- NVIDIA
- [OpenShell](#)
- OpenClaw
- Hermes

# Open Cowork

Open Cowork 是 Electron desktop agent host，用 Claude/OpenAI-compatible chat、Skills、MCP、WSL2/Lima sandbox、GUI automation 和 Feishu/Slack remote control，把本地 coding agent 能力包装成桌面协作入口。详见 [src-open-cowork-architecture](#)。

## 架构边界

它是桌面端 agent host，不是 trace viewer、token analytics 或独立 LLM serving 平台。与 cc-connect 相比，Open Cowork 更偏本机桌面/sandbox/GUI 操作；cc-connect 更偏把已有本地 agent session 接到 IM。

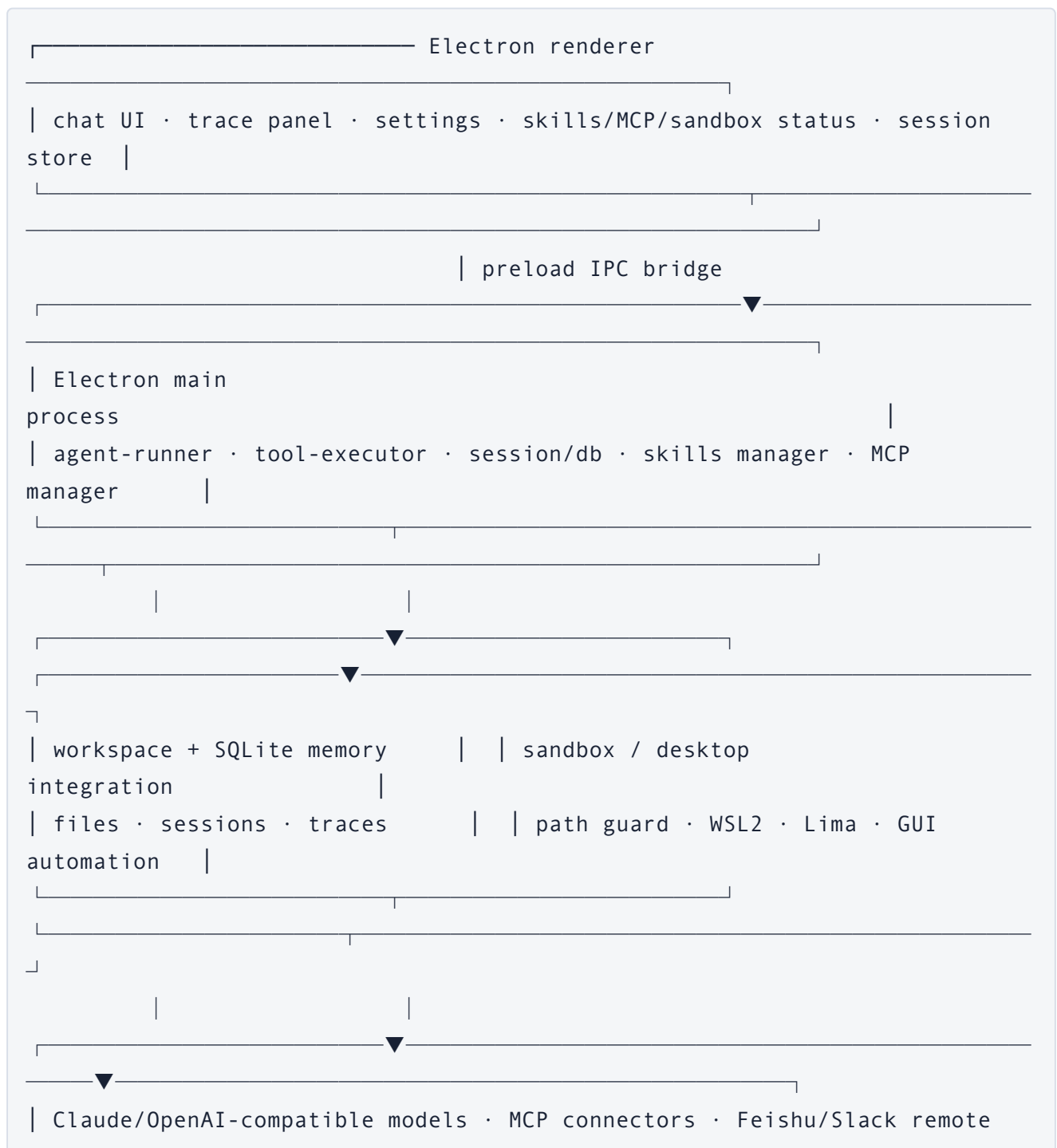
## 选型判断

适合需要桌面端聚合 agent、skills、MCP 和 sandbox 的场景。不适合只做 telemetry，此时看 loongsuite-pilot；也不适合只做请求取证，此时看 claude-tap。

# Open Cowork 架构与设计思路分析

OpenCoworkAI/open-cowork 是桌面端 agent host，而不是 terminal coding agent。它用 Electron main/preload/renderer 包装模型 API、Claude/Pi-compatible agent runner、SQLite/session、Skills、MCP、workspace 文件管理、WSL2/Lima 沙盒和远程控制，让非开发者通过 GUI 使用 AI 自动化文件、文档和桌面应用。

## 核心架构图



input |

## 模块分层

| 层/目录                                                    | 责任                                                |
|---------------------------------------------------------|---------------------------------------------------|
| src/main/claude/agent-runner.ts                         | agent 执行核心，测试覆盖 history rebuild 和 SDK session 恢复。 |
| src/main/sandbox/**                                     | path guard 与 WSL/Lima sandbox 相关逻辑。               |
| src/main/skills/**, .claude/skills                      | 技能加载、安装、校验、启停。                                    |
| src/main/memory/**, src/main/session/**, src/main/db/** | SQLite/session/memory 管理。                         |

## 关键数据流

- 用户在 GUI 选择 workspace 并输入任务，renderer 通过 preload IPC 调用 main process。
- agent-runner 组装模型、历史、技能、MCP 工具和 workspace 上下文，产生 trace/tool events。
- 文件/命令工具先过 workspace path guard；若 WSL2/Lima 可用，bash 路由到 VM，否则回退本机受限执行。

## 设计决策

- 桌面 app 优先降低使用门槛，代价是 Electron/IPC/OS sandbox/installer 复杂度。
- 安全模型是多级：path guard 是基础，WSL2/Lima 是增强；README 也提醒仍需审查高风险操作。
- Skills 处理文档/PPT/XLS/PDF 这类桌面生产力任务，区别于 coding-agent 纯代码 workflow。

## 对比定位

和 Multica 相比，Open Cowork 是单用户桌面入口；和 Codex/Pi 相比，它不是终端执行 loop，而是 GUI host；和 OpenShell/agent-sandbox 相比，它有 VM 级隔离但不是 K8s/runtime primitive。

## 相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

# OpenKruise Agents 架构与设计思路分析

原文: raw/openkruise-agents-architecture-analysis.md · 仓库: <https://github.com/openkruise/agents> · 分析版本 HEAD 0e58df8 (2026-06-12)

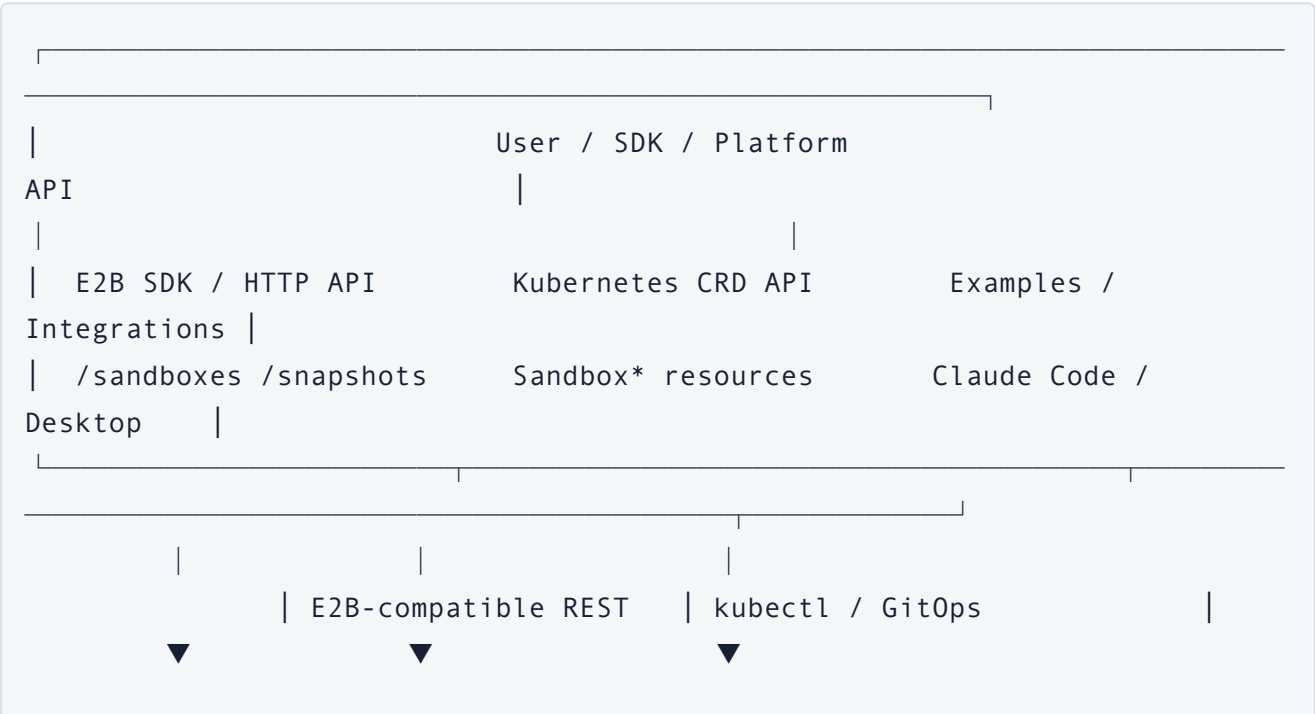
## 一句话定位

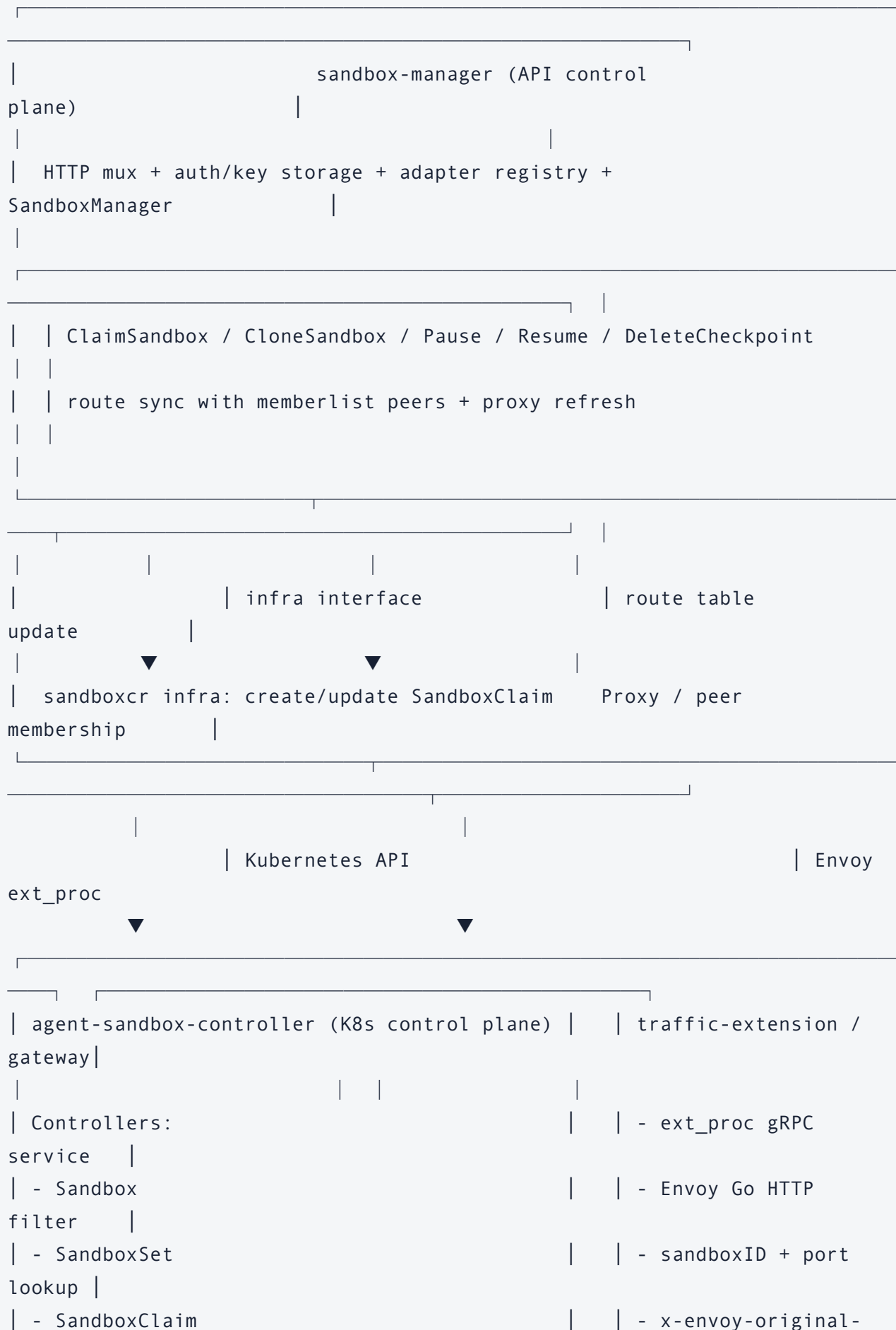
openkruise/agents 是 OpenKruise 面向 AI Agent sandbox lifecycle management 的 Kubernetes 原生平台层。它不只是一个 Sandbox CRD controller，而是把四类能力放在同一个工程里：

- 用 Sandbox / SandboxSet / SandboxClaim / SandboxTemplate 表达可暂停、可预热、可领取、可升级的 Agent sandbox。
- 用 agent-sandbox-controller 管 Pod/PVC/template/hash/status/rolling update/warm pool 等生命周期。
- 用 sandbox-manager 暴露 E2B-compatible API，把“创建 sandbox / clone checkpoint / pause / resume / route”变成产品 API。
- 用 traffic-extension / sandbox-gateway 接入 Envoy，把外部请求按 sandbox identity 路由到具体 Pod IP 和端口。

它和 [agent-sandbox](#) 的关系更像“上层发行版 + 兼容实现 + 产品化控制面”：README 明确说明会保持与 SIG agent-sandbox API 的兼容，同时在 SIG API 可用前内置 sandbox API 和实现。

## 核心架构图







## 模块分层

| 层 / 模块        | 职责                                                                                                                                                  |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| CRD API       | 定义 Sandbox、SandboxSet、SandboxClaim、SandboxTemplate，覆盖 pause、persistent contents、runtimes、claim、template、upgrade、dynamic volume mount。               |
| K8s 控制面       | controller-runtime manager；注册 Sandbox/SandboxSet/SandboxClaim/SandboxUpdateOps/SecurityTokenRefresh controller、webhook、field index、metrics cleanup。 |
| API 控制面       | 暴露 E2B-compatible API；把 request adapter、auth/key storage、infra backend、memberlist peer 和 route proxy 组合成 sandbox 管理服务。                              |
| Sandbox infra | 定义抽象 Infrastructure / Sandbox 接口；sandboxcr 实现用 K8s CRD 完成 claim、clone、pause/resume、checkpoint、CSI mount、identity token。                             |
| 路由数据面         | Envoy ext_proc 或 Go HTTP filter；从请求解析 sandboxID/port，查 route registry，写入 original-dst 元数据并做 peer 同步。                                                |
| Runtime 扩展    | agent-runtime sidecar、CSI volume mount provider、identity provider 和 token propagation 扩展点。                                                          |
| 示例与集成         | code interpreter、desktop、OpenClaw、Claude Code 等场景，展示如何把热池、E2B SDK、sandbox image 和 session 复用组合起来。                                                   |

## 关键对象

### Sandbox

Sandbox 是单个运行环境的期望状态。重要字段包括 paused、persistentContents、shutdownTime / pauseTime、runtimes、lifecycle、upgradePolicy、template / templateRef / volumeClaimTemplates。它把暂停、恢复、持久文件系统、runtime sidecar、动态存储和升级策略纳入同一个 K8s 对象。

### SandboxSet

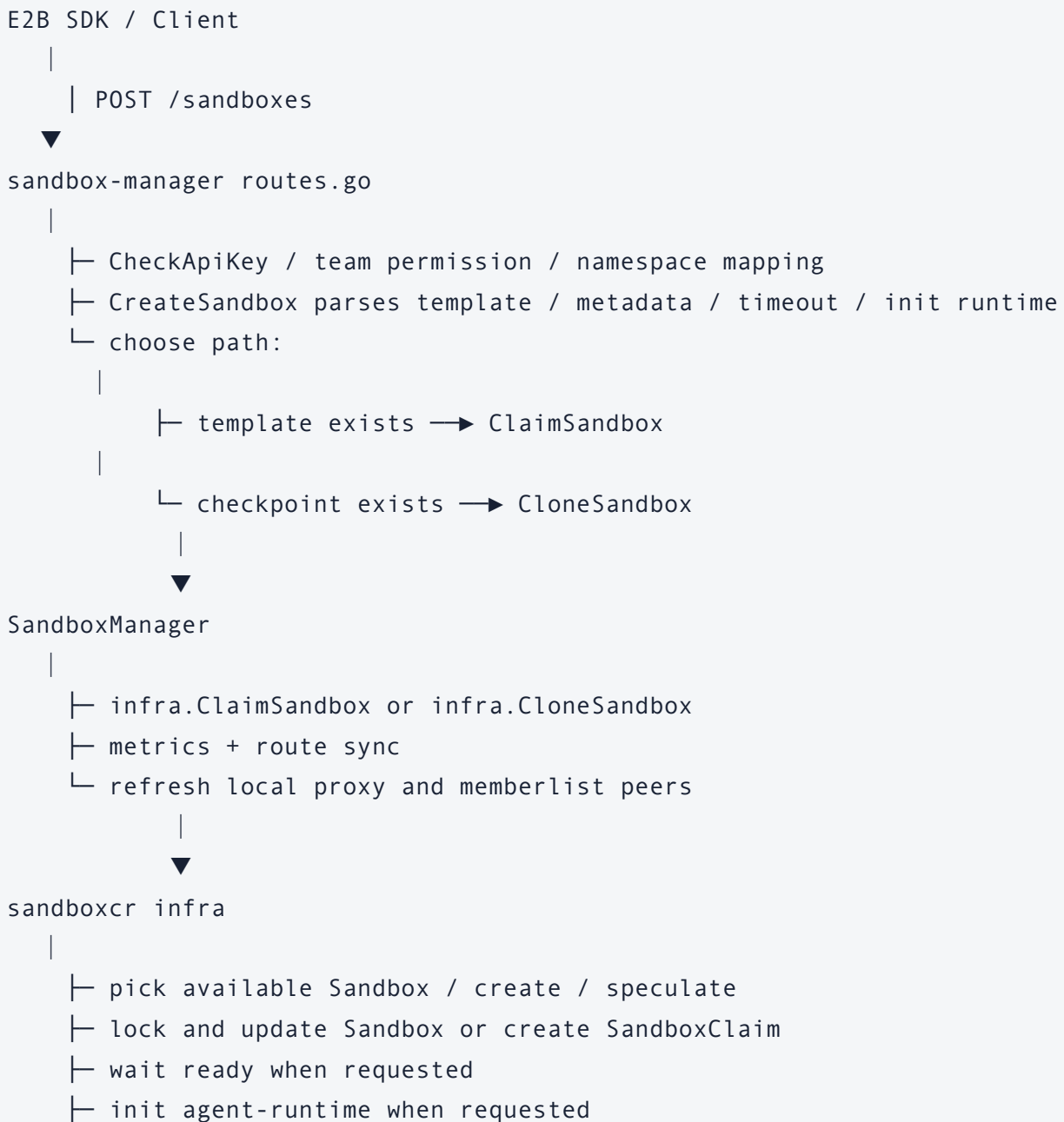
SandboxSet 维护一组“未被 claim 的可用 sandbox”，本质是 warm pool 和滚动更新控制器。replicas 表示需要保持的 unused sandboxes 数量，status 跟踪 available、updated、current/update revision，labels/annotations 则承载 pool、template、claim、owner、hash、lock、restore-from、init-runtime request、memberlist url 等协议状态。

## SandboxClaim

SandboxClaim 是把“用户请求一个 sandbox”变成控制面协议的关键对象。它通过 `templateName` 选择 `pool/template`，用 `createOnNoStock` 控制无库存时是否冷创建，用 `inplaceUpdate` 支持 `image/CPU` 更新，用 `dynamicVolumesMount` 支持请求时 CSI 挂载，并用 `reserveFailedSandbox` 支持失败现场保留。

## 关键数据流

### E2B API 创建 sandbox

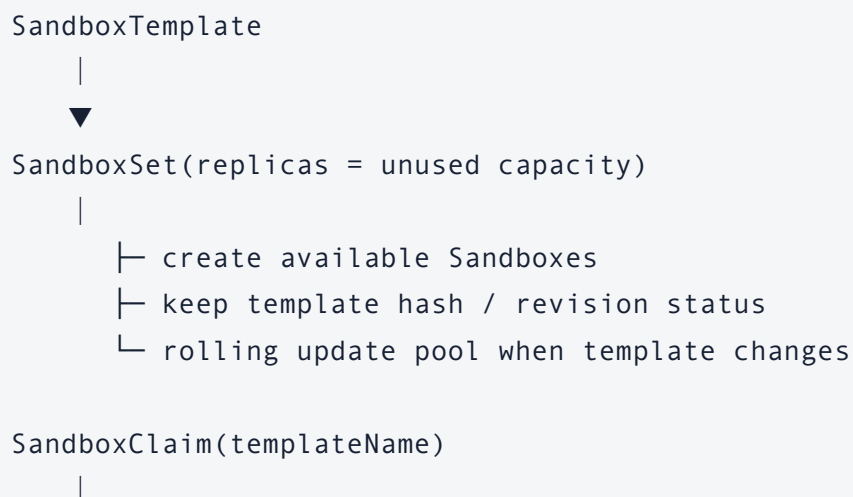


- └ issue and propagate identity token when feature enabled
- └ mount CSI dynamic volumes when requested

## 请求路由到 sandbox



## Warm pool / claim 生命周期



```

└─ select available Sandbox from matching SandboxSet
└─ lock and mark claimed / owner / timestamp
└─ apply labels, annotations, env vars and inplace update
└─ optionally initialize runtime and dynamic CSI mount
└─ wait ready -> Completed

```

## 设计决策

- 把 warm pool 做成声明式资源，而不是进程内队列：SandboxSet 让“保持 N 个可用 sandbox”由 K8s controller 持续收敛；SandboxClaim 则把领取动作显式化，便于处理并发、超时、失败保留和 owner/label 不变量。
- E2B API 是产品入口，CRD 是平台入口：开发者可以通过 E2B SDK 创建/暂停/恢复 sandbox，平台团队也可以通过 Sandbox\* CRD 做 GitOps、RBAC、namespace、多租户和控制面调试。
- 数据面和控制面解耦：sandbox-manager 负责创建、clone、pause、resume 和 route sync；Envoy ext\_proc / Go filter 只负责把请求映射到已知 sandbox route。高频请求不直接打 Kubernetes API。
- runtime 扩展是可插拔而不是硬编码：runtimes 字段和 agent-runtime、csi、traffic-proxy config 让 sidecar、动态存储、流量代理成为可声明能力；identity provider 接口也允许社区默认 token 与企业级 provider 分开演进。
- 身份安全还在工程化演进中：最近提交把 secret-backed key storage 的 ticker 刷新改成 informer-driven refresh，说明 E2B API key / token 生命周期已经成为真实控制面问题。

## 与同类项目的区别

| 项目                                 | 主要抽象                                                 | 与 OpenKruise Agents 的区别                                                                                                                              |
|------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">agent-sandbox</a>      | Sandbox CRD 原语                                       | 更偏 SIG 级基础 API；OpenKruise Agents 把 warm pool、E2B API、route proxy、identity/CSI/runtime 扩展也产品化进同一发行版。                                                  |
| <a href="#">agentcube</a>          | AgentRuntime / CodeInterpreter session orchestration | AgentCube 用 Router + WorkloadManager 把 agent-sandbox 包成 invocation API；OpenKruise Agents 直接提供 E2B-compatible API 和更完整的 sandbox lifecycle/runtime 扩展。 |
| <a href="#">substrate</a>          | WorkerPool / ActorTemplate 高密度 actor substrate       | Substrate 更强调 actor multiplexing、wake routing、snapshot；OpenKruise Agents 更贴近 K8s CRD + E2B sandbox 管理。                                               |
| <a href="#">agentscope-runtime</a> | AgentApp / Runner / deployer                         | AgentScope Runtime 是 Python app 服务化壳；OpenKruise Agents 是 Kubernetes sandbox lifecycle substrate，可作为 workspace/E2B 后端。                                |
| <a href="#">OpenShell</a>          | Gateway + supervisor + policy proxy                  |                                                                                                                                                      |

| 项目 | 主要抽象 | 与 OpenKruise Agents 的区别                                                                                      |
|----|------|--------------------------------------------------------------------------------------------------------------|
|    |      | OpenShell 的强项是 sandbox 内 runtime enforcement 和 policy;<br>OpenKruise Agents 的强项是 K8s 生命周期、热池、E2B API 和路由控制面。 |

## 采用建议

- 需要“在 Kubernetes 上提供 E2B-like sandbox 服务”时，优先看 [openkruise-agents](#)。
- 只想学习最小 CRD 原语时，先看 [agent-sandbox](#)；OpenKruise Agents 需要同时理解 controller、manager、gateway、runtime sidecar 和 identity/CSI。
- 已经有 Agent framework、想接远端 workspace/sandbox backend 时，重点验证 E2B API 兼容范围、template 管理、timeout、文件/命令 API 和路由行为。
- 高风险不可信代码执行仍要额外验证 runtimeClass、NetworkPolicy、seccomp/AppArmor、egress gateway、token revoke 和 audit；OpenKruise Agents 提供扩展点，但不是自动替代强 runtime enforcement。

## 风险与待验证点

- API 仍处在快速演进期，README/roadmap 显示 pool、storage、network、runtime、scheduling、observability、SDK 都有大量后续事项。
- SandboxClaim 并发领取、create-on-no-stock、speculate、lock 和 expectations 是正确性关键，需要用压力测试验证。
- 路由表同步依赖 manager/gateway peer 状态，生产要验证实例重启、网络分区和 route stale 时的 fail-close 行为。
- checkpoint/hibernation 的 memory/filesystem/GPU memory 能力要按具体 runtime 和存储后端验证，不能只看字段设计。

# OpenShell

OpenShell 是 NVIDIA 面向 autonomous AI Agent 的安全私有运行时。它把 agent 放进可审计、可连接、可恢复的 sandbox 中运行，并把 Gateway 控制面、sandbox supervisor enforcement、OPA/Z3 policy pipeline、provider credential 管理和 `inference.local` 推理路由拆成清晰边界。详见 [src-openshell-architecture](#)。

## 架构边界

OpenShell 不是 Agent framework。它不负责定义 ReAct loop 或多 Agent 工作流，而是负责让这些 agent 在受控环境中执行：Gateway owns desired state，Supervisor owns runtime enforcement。

## 核心组件

- Gateway control plane：管理 sandbox、provider、policy、settings、inference、session 和 compute orchestration。
- Runtime driver：把 Docker、Podman、Kubernetes、VM/libkrun 等后端收敛成统一 compute driver。
- Sandbox supervisor：在 sandbox 内准备 filesystem、process limits、network namespace、TLS、proxy、SSH 和 agent child。
- Policy proxy：用 OPA 做 L4/L7 网络裁决，绑定进程身份并做 SSRF 防护。
- `inference.local`：由 sandbox proxy 拦截并路由到 provider，避免 agent 直接持有真实模型凭证。

## 选型判断

适合把本地或托管 coding agent 放到强安全边界里运行。和 [agent-sandbox](#) 相比，OpenShell 更偏完整安全 runtime；和 [substrate](#) 相比，它不追求大规模 actor snapshot multiplex；和 [agentgateway](#) 相比，它治理运行时与网络 enforcement，而不是集群入口网关。

# OpenShell 架构与设计思路分析

原文: raw/openshell-architecture-analysis.md · 仓库: <https://github.com/NVIDIA/OpenShell> · 分析版本 HEAD 97986d9

## 一句话定位

[OpenShell](#) 是 NVIDIA 面向 autonomous AI Agent 的安全私有运行时：它不是新的 Agent framework，而是把 Agent 放进可审计、可配置、可连接、可恢复的 sandbox 里运行。它的关键手段是把 Gateway 控制面、sandbox 内 Supervisor enforcement、OPA/Z3 policy pipeline、provider credential 管理和 inference.local 推理路由拆成清晰边界；这也解释了它为什么能和 [NemoClaw](#)、[agent-sandbox](#)、[agentgateway](#) 互补。

## 核心架构图





```

child | |
| | - starts as root | spawn | - non-root
user | |
| | - loads policy/config | - filesystem/process/network
limits | |
| | - injects credentials | - ordinary egress forced through
proxy | |
| | - starts proxy + SSH |

```

```

| | - opens outbound | CONNECT /
HTTP |
| | session to gateway | ← relay/config/logs

```

```

| |
| |
| |
| |
| |

```

```

| | Policy
proxy | |
| | - bind process identity via /proc/net/tcp + binary
hash | |
| | - evaluate OPA network
action | |
| | - reject SSRF/internal destinations unless explicitly
allowed | |
| | - terminate TLS for L7 inspection and credential rewrite where
enabled | |
| | - intercept https://inference.local and route via openshell-
router | |

```

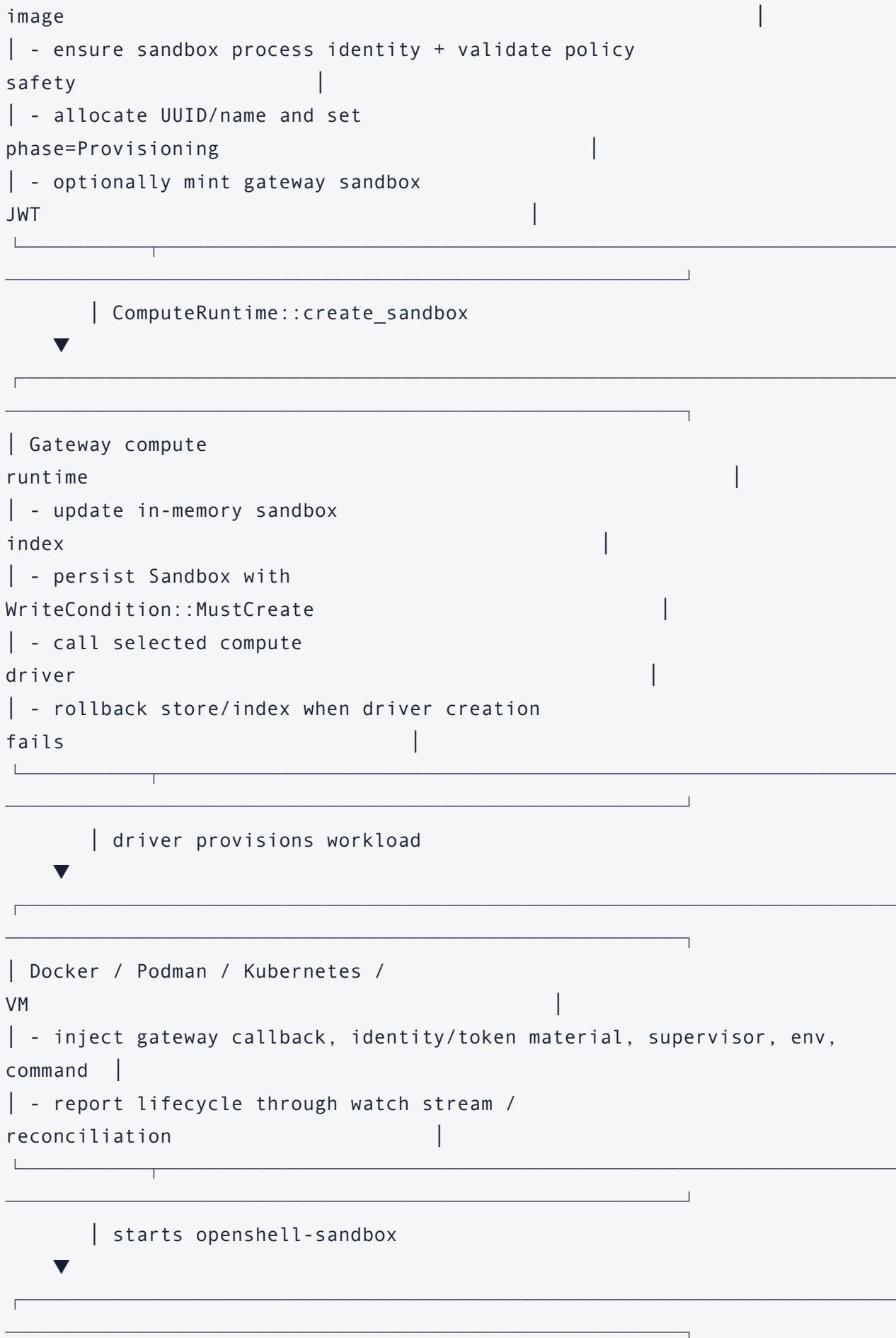
## 模块分层

| 层 / 模块               | 职责                                                                                                                       |
|----------------------|--------------------------------------------------------------------------------------------------------------------------|
| 用户入口                 | CLI/SDK/TUI 解析 gateway context、sandbox name、auth token，把用户意图变成 Gateway API 调用。                                           |
| Gateway 控制面          | API、认证、持久状态、policy/settings/provider/inference 配置、compute orchestration、relay 协调。                                        |
| 持久化                  | protobuf object store，SQLite/Postgres 后端，resource_version CAS；生产写路径必须显式使用 WriteCondition 或 update_message_cas。           |
| Compute runtime      | 抽象 Docker/Podman/Kubernetes/VM driver；负责 create/delete/watch/reconcile/startup resume。                                   |
| Sandbox supervisor   | sandbox 内安全边界：加载 policy/settings、准备 filesystem/process/netns/TLS、启动 proxy/SSH、拉取 provider env、启动 agent child、热更新 policy。 |
| Policy proxy         | 用 OPA 做 L4/L7 网络决策，用 /proc/net/tcp 绑定调用进程身份，做 SSRF 防护、TLS/L7 inspection、凭证 rewrite、inference.local 路由。                   |
| Provider / inference | provider discovery、credential 注入、OpenAI/Anthropic/NVIDIA 等模型路由；sandbox 内 inference.local 不走普通 OPA network policy。        |

OpenShell 的分层约束很强：Gateway owns desired state，Supervisor owns runtime enforcement。Gateway 不做 agent egress 的 per-request 裁决，真正的 安全 enforcement 放在 sandbox 内，那里能看到进程身份、文件系统、网络命名空间、TLS 流和 runtime credentials。

## 关键数据流







创建路径体现了 OpenShell 的一致性优先：Gateway 先验证 spec/policy、生成 sandbox identity 和 phase，再由 compute runtime 以 MustCreate 写入 store，随后才调用具体 driver；driver 失败时会回滚 store/index，避免留下“半创建”的 sandbox。Supervisor 启动则是另一条安全 pipeline：加载 policy/OPA、拉 provider env、准备 filesystem/netns/seccomp/TLS、启动 proxy/SSH、建立 outbound session，最后才 spawn restricted agent child。

## 设计决策与哲学

- 控制面和 enforcement 分离：Gateway 存储和分发策略，但请求时裁决放在 sandbox 内 Supervisor/proxy/kernel controls，因为那里能绑定 binary identity、TLS/L7 和 runtime credentials。
- Supervisor 主动连 Gateway：sandbox 通过 outbound ConnectSupervisor 持有 live session，Gateway 的 connect/exec/file sync 通过 relay 协调，不要求每种 runtime 暴露可直连地址。
- driver 薄、语义厚：Docker/Podman/Kubernetes/VM driver 只翻译平台操作；sandbox lifecycle、phase、policy/settings/provider/inference 是 OpenShell 自己的语义层。
- 持久状态按对象存储 + CAS：状态对象统一进 protobuf object store，生产写路径通过 CAS 或 MustCreate，降低并发 Gateway handler/reconciler 互相覆盖的风险。
- 网络策略绑定进程身份：proxy 通过 /proc/net/tcp 解析 socket owner，把 binary path/hash/ancestor/cmdline 放进 OPA input，避免一个 sandbox 内所有进程共享同一网络权限。
- 推理路由作为特殊本地能力：https://inference.local 由 sandbox proxy TLS terminate 并转给 openshell-router，避免 agent 直接持有 provider credentials 访问模型端点；这和 [agent-credential-isolation](#) 的思想一致。
- policy proposal 默认人工审批：denial 可以生成 draft policy chunk，但 Gateway 会用 Z3 prover 检查 policy delta；只有显式 auto 模式且 prover delta 为空才自动批准。

## 关键组件深入解读

### Gateway Compute Runtime

ComputeRuntime 是 Gateway 对 sandbox lifecycle 的核心封装。它把 Docker、Kubernetes、Podman、remote VM 都收敛成同一个 driver trait，Gateway 层只关心 create/delete/list/watch/validate。创建路径先持久化再调用 driver，失败回滚；删除路径先 CAS 设置 Deleting，清理 sandbox-owned records，再调用 driver delete。watch/reconcile 是长期后台任务：watch loop 应用 driver event，reconcile loop 定期比对 store 与 backend，发现 backend 缺失就走 orphan pruning。

近期 HEAD 97986d9 正好改了 startup resume：legacy 或未初始化 sandbox row 会 decode 成 proto 默认 Unspecified phase，之前 resume sweep 会跳过它，导致 Gateway 重启后既不恢复也不报错；最新修复把 Unspecified 视为应当运行，backend 存在则 resume，不存在则 mark Error。

### Supervisor Relay 与 Sandbox Proxy

OpenShell 的 connect/exec/file sync 不走 Gateway 直接拨 sandbox，而是由 Supervisor 先向 Gateway 建立 gRPC session。Gateway 想打开 relay 时，先等待 session，登记 pending channel，再向 Supervisor 发 RelayOpen；Supervisor 用 RelayStream 首帧 claim channel。这个顺序避免反向连接先到而 Gateway 还没有等待者的竞态。

Proxy 是数据面的核心：先解析 CONNECT，inference.local:443 进入 inference interception；其他目标先做进程身份绑定和 OPA policy evaluation，再做 SSRF 防护、TLS/L7 inspection、credential rewrite 或 raw relay。L7 和 inference 都在 L4 allow 之后发生，tls: skip 则显式退回 raw tunnel。

## 相关页面

- [NemoClaw](#)
- [agent-sandbox](#)
- [agentgateway](#)
- [cloud-native-security](#)
- [agent-credential-isolation](#)
- [mcp](#)
- [vllm](#)

# Plano

Plano 是 AI-native proxy / data plane，和 [agentgateway](#) 同处 agentic gateway / proxy 赛道，但更偏 Rust data plane + CLI/config/skills。详见 [src-plano-architecture](#)。

## 架构边界

Plano 不只是 provider adapter，也不是 Kubernetes Gateway API 控制面的完整替代。它的特点是把低延迟 proxy/data plane、配置 schema、CLI 运维入口和 skills/demos 组合起来，服务 model routing、guardrails 和 agent orchestration。

## 关键设计

- `crates/llm_gateway` 和 `prompt_gateway` 承载核心 Rust gateway/data plane。
- `cli/planoai` 是运维与使用入口。
- `config/**` 包含 Envoy template、schema 和测试配置。
- `skills/**` 把 agentic ops、部署、路由、观测固化成能力包。

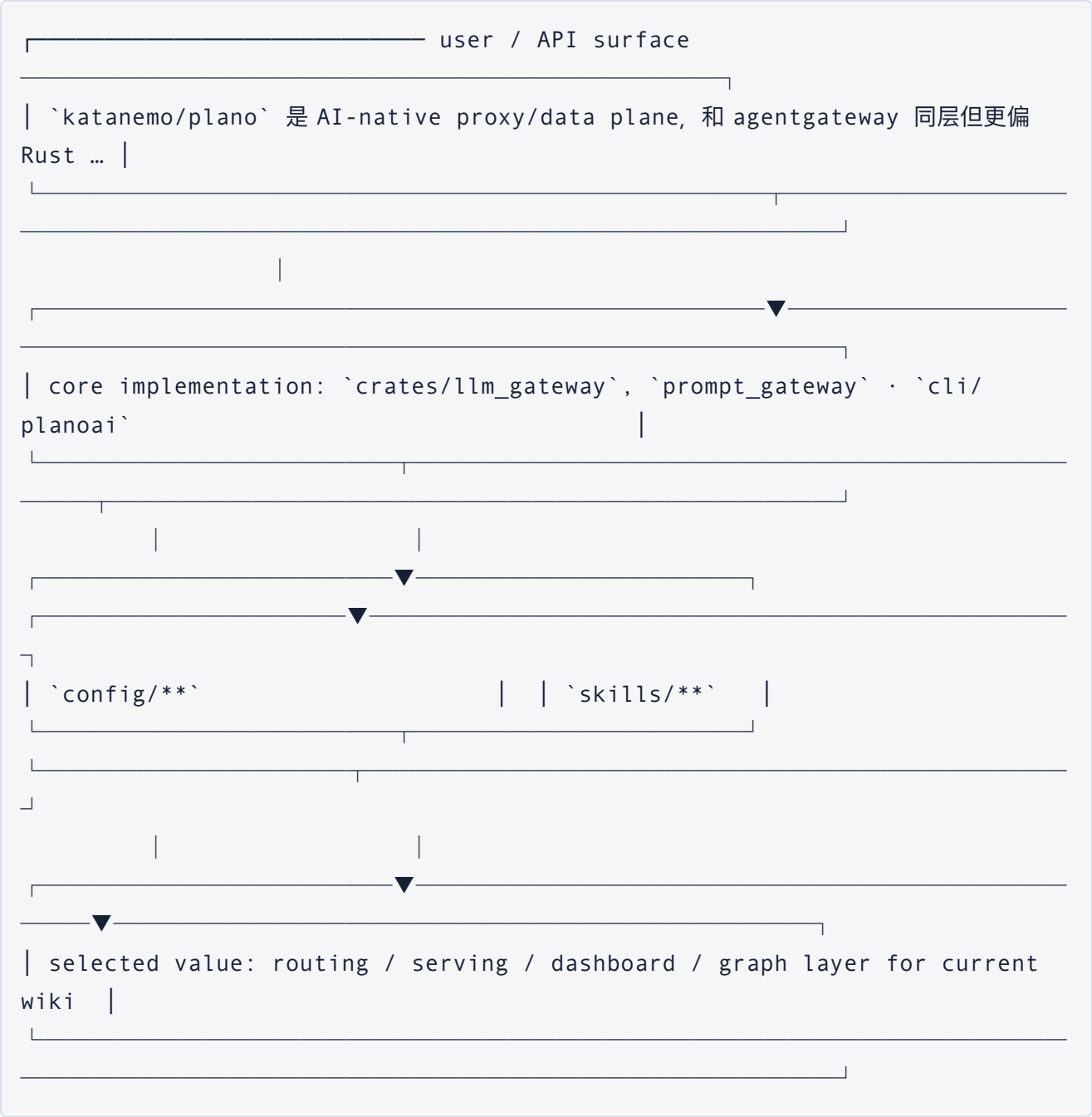
## 选型判断

想看自有 Rust AI proxy + skills-first 路线时看 Plano。需要 Gateway API / xDS / mesh 结合看 [agentgateway](#) 或 [kgateway](#)；需要 Envoy Gateway GenAI 专用治理看 [Envoy AI Gateway](#)。

# Plano 架构与设计思路分析

katanemo/plano 是 AI-native proxy/data plane，和 agentgateway 同层但更偏 Rust data plane + CLI/config/skills。仓库含 crates/llm\_gateway、prompt\_gateway、hermesllm、CLI、配置 schema、demos 和 skills，适合补 MCP/AI Gateway tooling map。

## 核心架构图



## 模块分层

| 层/目录                               | 责任                           |
|------------------------------------|------------------------------|
| crates/llm_gateway, prompt_gateway | 核心 Rust gateway/data plane。  |
| cli/planoai                        | CLI 运维入口。                    |
| config/**                          | Envoy template、schema 和测试配置。 |
| skills/**                          | 面向 agent/部署/路由/观测的技能包。       |

## 关键数据流

1. 用户通过 config/CLI 定义 gateway、routing、filter chains。
2. Rust data plane 处理 LLM 请求、路由、guardrails。
3. skills/demos 提供 agent orchestration 和运维入口。

## 设计决策

- 把 proxy/data plane 与 skills/docs 结合，服务 agentic ops。
- Rust 核心适合低延迟代理。
- 配置 schema 是使用体验关键。

## 对比定位

和 Envoy AI Gateway 相比，Plano 更自有 data plane/skills-first；和 agentgateway 相比，二者都可看作 agentic proxy，但生态和实现路线不同。

## 相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

# substrate

Agent Substrate 是运行在 kubernetes 之上的高密度 agent-like workload substrate。它把 Kubernetes 当作低频容量和模板控制面，把 Actor 的高频 resume/suspend、worker 分配、状态锁和路由唤醒放到 Redis/ValKey、ateapi、atelet、ateom-gVisor 和 atenet router 里完成。详见 [src-substrate-architecture](#)。

## 架构边界

Substrate 不是普通的 Agent framework，也不是只创建一个 Pod 的 sandbox CRD。它的核心目标是让大量 idle actor multiplex 到较少 warm worker 上，通过 golden snapshot、runtime checkpoint/restore、actor DNS 和 Envoy ext\_proc 唤醒降低激活延迟。

它和 [agent-sandbox](#) / [agentcube](#) 的区别在层级：[agent-sandbox](#) 提供有状态 Sandbox 原语，[agentcube](#) 做会话路由和 warm pool，Substrate 更底层，直接把 worker pool、actor template、snapshot 和路由恢复做成 substrate。

## 关键抽象

- WorkerPool：声明 worker 容量和运行模板，由 controller 物化为 privileged ateom pods。
- ActorTemplate：生成 golden actor，再 checkpoint 成 golden snapshot，供后续 actor 快速恢复。
- Actor：高频运行状态不进 K8s API，而是存 Redis/ValKey。
- atelet / ateom-gvisor：节点侧 supervisor 和 gVisor worker，负责 runsc create/start/checkpoint/restore。
- atenet：通过 actor hostname 触发 ResumeActor，再把请求转发到已分配 worker。

## 选型判断

适合研究或构建高密度、低空闲成本、可暂停恢复的 agent runtime。若只是给单个 agent 提供隔离，优先看 [agent-sandbox](#) 或 [OpenShell](#)；若需要团队级会话编排和 warm pool，优先看 [agentcube](#)；若要把 sandbox 出口流量纳入治理，再叠加 [agentgateway](#) / [ai-gateway](#)。

# vLLM Semantic Router

Semantic Router 是 vLLM 生态的 programmable Mixture-of-Models routing and control layer。应用通过稳定的 OpenAI/Anthropic-compatible endpoint 访问它，Router 根据请求特征选择或组合能力路径。

## 解决的问题

应用不应把“简单问题走便宜模型、复杂问题升级、敏感内容走安全路径、需要工具/检索/验证时组合能力”硬编码在业务代码中。Semantic Router 将 intent、difficulty、context、modality、identity、risk 和 system state 统一放在请求路径。

## 核心抽象

Recipe/virtual model 可以指向单模型、specialist、cascade 或 bounded multi-model workflow，并附加 retrieval、memory、tools、caching、guardrails 和 verification。

## 边界与选型

它决定 model/ability/recipe，不替代 Envoy 或模型 server，也不直接负责 K8s pod endpoint picking。需要成本/质量算法基线看 routellm；需要 KV、队列、InferencePool endpoint 看 llm-d-router。

详见 src-semantic-router-architecture、[src-k8s-gateway-routing-comparison](#)。

# 声明式 Agent 管理

把 Kubernetes Operator 模式套到 AI Agent 运维上的设计哲学：Agent 是资源（CR），不是对象（class）。

由 [HiClaw](#) 在 2026 年开源时首次清晰展现：

- Agent 用 CRD 声明（YAML），不是用 Python class 编排
- controller 持续 reconcile，让 actual state（容器、IM 房间、网关 consumer）向 desired state（CR）收敛
- 免费获得 K8s 的 self-healing / 多副本 / RBAC / Helm 上线下线 / kubectl 操作面
- 与 LangGraph / AutoGen / CrewAI 这类"在你自己进程里跑 Python"的框架根本不同——后者要做企业级运维需重头补

## Kubernetes Resource Model 对应

| K8s 模型         | Agent 管理类比  | 说明                                              |
|----------------|-------------|-------------------------------------------------|
| spec           | 期望 Agent 能力 | model、runtime、tools、skills、权限、协作关系              |
| status         | 实际运行状态      | 容器状态、IM 身份、room id、gateway consumer、ready/error |
| reconcile      | 自愈循环        | 外部资源被删或漂移后重新创建/修正                               |
| OwnerReference | 生命周期归属      | Team 删除时清理成员或 session 资源                        |
| finalizer      | 有序清理        | 先删 gateway consumer、Matrix user、storage，再删 CR   |

## 代表项目

| 项目                                | 声明式对象                                                            | 负责层级                                                                                |
|-----------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <a href="#">HiClaw</a>            | Worker / Team / Human / Manager                                  | 多 Agent 协作、IM 平面、凭据网关、容器后端                                                          |
| <a href="#">agent-sandbox</a>     | Sandbox / SandboxClaim /<br>SandboxTemplate /<br>SandboxWarmPool | 有状态单实例 Agent runtime 原语                                                             |
| <a href="#">openkruise-agents</a> | Sandbox / SandboxSet /<br>SandboxClaim / SandboxTemplate         | Agent sandbox lifecycle platform：warm pool、claim、E2B API、路由、runtime/CSI/identity 扩展 |
| <a href="#">agentcube</a>         | AgentRuntime / CodeInterpreter                                   | 把 agent-sandbox 包成会话、Router、PicoD、WarmPool API                                      |

| 项目     | 声明式对象                   | 负责层级                                |
|--------|-------------------------|-------------------------------------|
| kagent | agentic DevOps/K8s 操作入口 | 更偏“用 Agent 操作 K8s”，不是底层 runtime CRD |

## 什么时候适合

- Agent 是长寿命、有状态、有身份的运行单元；
- 需要多人/多 Agent 协作和可审计生命周期；
- 需要 RBAC、namespace、多租户、Helm/GitOps 交付；
- 需要自愈、滚动升级、资源配额、网络/存储策略；
- 需要把 Agent 与 gateway、storage、IM、sandbox 等外部资源绑定。

## 什么时候是反模式

- 只是在一个 Python 进程里编排短生命周期 workflow；
- 目标是本地个人自动化，不需要多租户或运维控制面；
- Agent 没有持久身份、状态、权限或外部资源；
- 团队还没准备好 Kubernetes operator 的调试和发布复杂度。

## 与插件设计原则的关系

声明式 Agent 管理把 ai-agent-plugin-patterns 里的“接口化”和“宿主边界”推到集群级：Agent 能力不是散落在 prompt、脚本和环境变量里，而是被 CRD、policy、secret、gateway backend 和 controller 状态明确表达。